

UE : Bases de données et développement back-end

- **AA** : Bases de données - Théorie

Antoine Malaise

Ecrit par Samuel Cremer

Bachelier en Informatique Orientation réseaux & télécommunications

Base de données - Théorie

Antoine Malaise
Dr Samuel CREMER, Ir Ing.

1

1

Plan du cours

1. Introduction aux BD
2. Modélisation
3. Langage SQL
4. Algèbre relationnelle
5. Calcul relationnel
6. Fonctionnalités avancées
7. Indexation et plan de requête
8. Big Data, NoSQL et NewSQL
9. Les SGBDR

2

2

1. Introduction

1.1. Définitions

3

3

Base de données : définitions

Définition générale :

*Ensemble **organisé** de données ayant un objectif commun*

Base de données = collection de données

Base de données $\neq$ logiciel informatique

Base de données informatisée :

*Un ensemble **structuré*** et **organisé** permettant le stockage de grandes quantités de données afin d'en faciliter l'exploitation*

* Le terme structuré peut être facultatif en fonction du paradigme utilisé

4

4

Base de données : définitions

SGBD (Système de Gestion de Base de Données) :

DBMS (DataBase Management System) :

Logiciel permettant la gestion et l'accès à des bases de données

SGBDR :

RDBMS :

*SGBD dédié aux BD relationnelles**

* Paradigme de base de données le plus utilisé, s'inspirant de l'algèbre relationnelle

5

5

Base de données : définitions

OLTP (Online Transaction Processing) :

(traitement transactionnel en ligne) :

*Contexte d'utilisation typique des activités opérationnelles
(e-commerce, production, système de réservation, etc.)*

Un système OLTP doit :

- Pouvoir répondre à un nombre élevé de transactions (ensemble solidaire de requêtes) par minute.
- Doit être performant tant en lecture qu'en écriture.
- Garantir l'ACIDité des transactions.

6

6

Base de données : définitions

Propriété ACID

Ensemble de propriétés qui garantissent qu'une transaction est exécutée de façon fiable

- Atomicité :** une transaction se fait au complet ou pas du tout
- Cohérence :** assure que chaque transaction amènera le système d'un état valide à un autre état valide
- Isolation :** Toute transaction doit s'exécuter comme si elle était la seule sur le système (pas de dépendance)
- Durabilité :** lorsqu'une transaction a été confirmée, elle demeure enregistrée même à la suite d'une panne

7

7

Base de données : définitions

OLAP (Online Analytical Processing) : (traitement analytique en ligne) :

*Contexte d'utilisation typique des secteurs décisionnels
(marketing, analyse financière, reporting, etc.)*

Un système OLAP doit :

- Pouvoir répondre rapidement à des requêtes d'interrogation souvent complexes (plus complexes qu'en OLTP)
- Traiter de "gros" volumes de données.
- Permettre des ingestions massives de données

8

8

1. Introduction

1.2. Bref historique

9

9

Avant les SGBD

Les fichiers plats (flat files)

```
line-0043660~P018850127500~10~4020~000117~29672~279~5~921~921~5~5618~115213V129411~1294.2520  
line-0043661~P018472109620~10~4020~000462~29734~282~6~517~517~5~1346~85274V10854~4~32724.4786  
line-0043662~P018448171044~10~4020~000227~29994~282~6~482~482~5~10456~125295V113958~4~28932.55  
line-0043663~P018009186870~10~4020~000510~29565~276~4~1073~1073~5~4322~45303V122691~419.4589  
line-0043664~P015617121983~10~4020~000397~29890~280~1~876~876~5~95559V14001~24432.6089~42  
line-0043665~P016588191572~10~4020~000146~29580~283~1~849~849~5~15232~35568V178804~14352.7713  
line-0043666~P016006173883~10~4020~000511~30052~276~4~1074~1074~5~13349~105623V169217~5056.46  
line-0043667~P015428113898~10~4020~000446~29774~277~3~429~429~5~10770~58480V15303~46107.082~5  
line-0043668~P014732180295~10~4020~000514~29614~282~6~529~529~5~1566~85017V18045~4~35944.1562  
line-0043669~P01412160936~10~4020~000578~29747~283~1~895~895~5~15538~25877V180261~714.7043~7  
line-0043670~P01438416310~10~4020~000504~29566~278~3~810~810~5~18017~105888V193490~6122.082  
line-0043671~P013978119376~10~4020~000200~29890~283~1~855~855~5~13693~25954V170960~8128.7876  
line-0043672~P01386215353~10~4020~000119~30067~282~6~464~464~5~3900~85931V120862~4124.182~5  
line-0043673~P013775141242~10~4020~000618~29844~275~2~821~821~5~1417~35931V17271~3746.2015~35  
line-0043674~P012760141756~10~4020~000083~29596~282~6~458~458~5~19204~24081V199804~4~2624.382  
line-0043675~P01241218464~10~4020~000470~29827~277~3~631~631~5~11886~124123V141402~8716.3102  
line-0043676~P011861145059~10~4020~000017~29811~275~5~755~755~5~18153~36168V194127~14203.458  
line-0043677~P011048174786~10~4020~000479~29824~278~6~856~856~5~14030~16302V172763~4~7793.1105  
line-0043678~P010817150168~10~4020~000203~29889~281~4~1021~1021~5~5093~66363V126658~9799.5243  
line-0043679~P010527142759~10~4020~000480~29761~278~6~525~525~5~5439~126365V128492~4~1316.0575  
line-0043680~P01730130087~10~4020~000491~29889~281~4~1049~1049~5~4653~126370V13462~11397.45  
line-0043681~P01189177803~10~4020~000423~29661~279~5~955~955~5~5301~16608V127707~13787.5434  
line-0043682~P01564124200~10~4020~000456~29759~275~2~868~868~5~76~46712V13373~1875.7229~371.8
```

Développement d'outils spécifiques

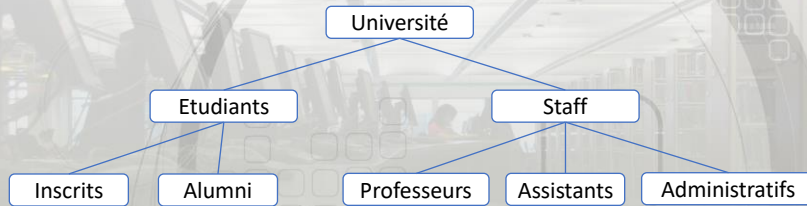
Logiciels rigides

Coûts

10

10

Bases de données hiérarchiques



Chaque nœud ne peut avoir qu'un seul parent

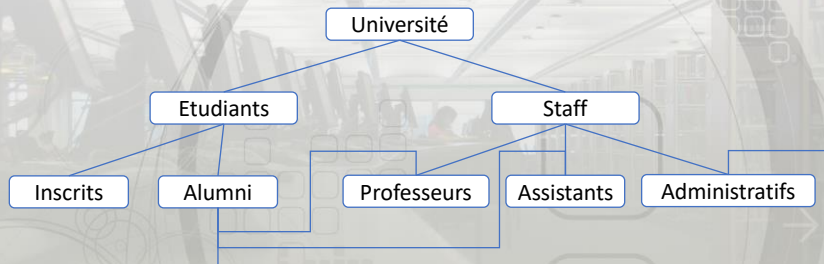
Exemple : IMS (Information Management System)
initialement à destination du programme Apollo
toujours d'actualité



11

11

Bases de données réseaux



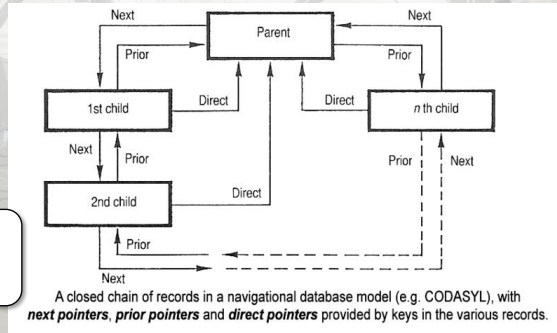
Exemple : IDS (Integrated Data Store)
base de "l'approche" CODASYL (consortium à la base du COBOL)



12

12

Bases de données réseaux : IDS



<https://en.wikipedia.org/wiki/Database#/media/File:Codasy1B.png>

Navigation à la manière
d'une liste doublement chaînée fusionnée avec un arbre

Paradigme souvent délaissé
car estimé trop complexe

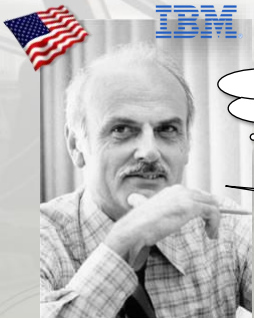


Les paradigmes hiérarchiques et réseaux sont
regroupés sous l'appellation :
Navigational DBMS

13

13

Edgar Frank Codd



en.wikipedia.org/wiki/Edgar_F._Codd

CODASYL n'apporte pas
de réelle facilité pour la
recherche d'information

Je propose le **modèle relationnel** !

Codd, Edgar F. (1970). "A Relational Model of Data for Large Shared Data Banks". *Communications of the ACM*. 13 (6): 377–387.

C'est trop nul ton truc !

LoL

mmm...
pas si nul...

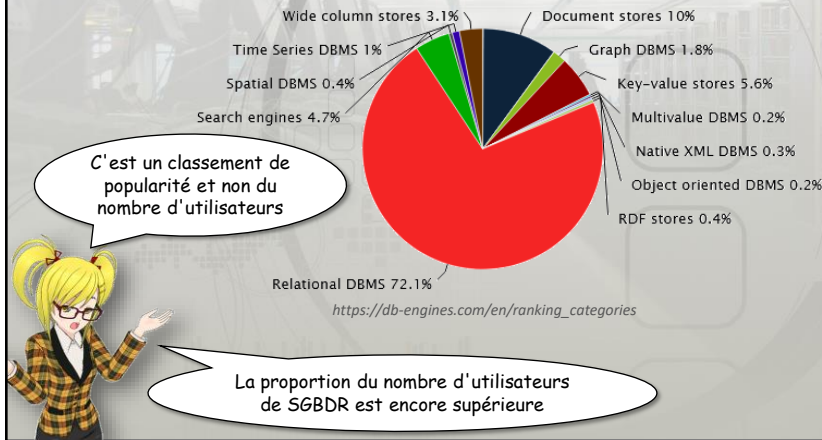


14

14

Bases de données relationnelles

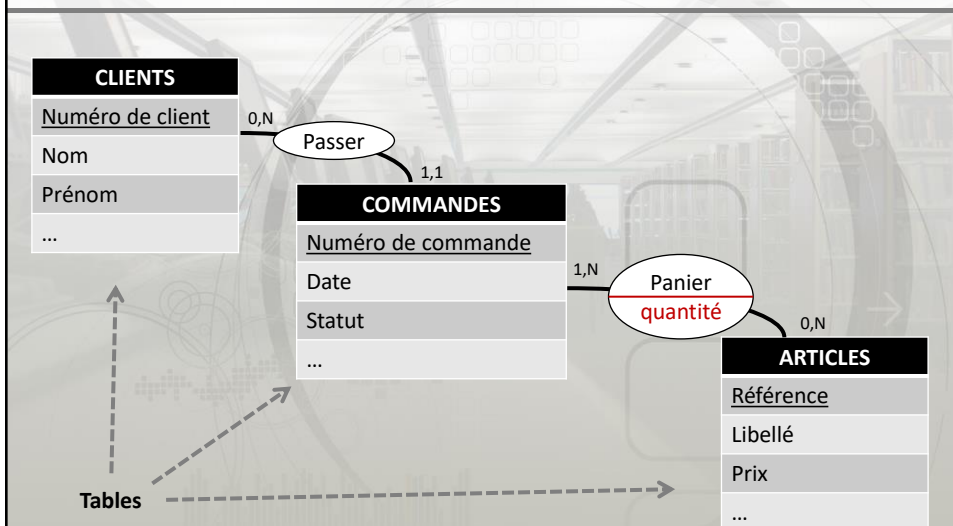
Classement de popularité des différents paradigmes de BD (septembre 2018)



15

15

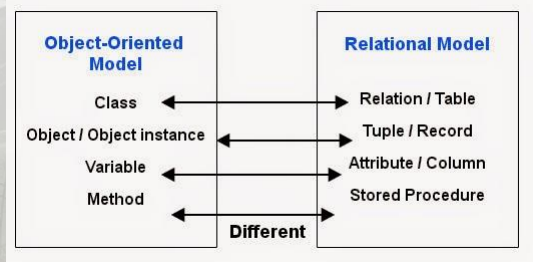
Le modèle relationnel



16

16

Bases de données orientées objets



tinyurl.com/y7p7jlbh

Avantages :

- Meilleure intégration avec les langages OO (plus facile pour le développeur OO)
- Plus grande flexibilité (types et relations)

17

17

Bases de données orientées objets



18

18

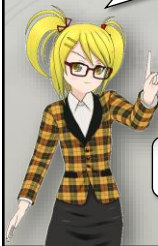
Bases de données orientées objets



Inconvénients :

- Cette flexibilité rend beaucoup de fonctionnalités "simples" (un tri par exemple) plus complexes à mettre en œuvre
- Les performances (pour de vrais cas d'utilisation)
- Les SGBDO ont quasi disparu

Popularité de 0,2 %
en 2022



Notez que certains SGBDR
permettent une approche OO

18 systems in ranking, September 2018

Rank			DBMS	Database Model	Score		
Sep 2018	Aug 2018	Sep 2017			Sep 2018	Aug 2018	Sep 2017
1.	1.	1.	InterSystems Caché	Multi-model	2.77	+0.51	+0.03
2.	2.	4.	Versant Object Database	Object oriented DBMS	1.27	+0.04	+0.16
3.	3.	2.	Db4o	Object oriented DBMS	1.23	+0.03	-0.19
4.	4.	3.	ObjectStore	Object oriented DBMS	1.21	+0.17	+0.09
5.	5.	5.	Matisse	Object oriented DBMS	0.97	+0.05	+0.29
6.	6.	9.	GemStone/S	Object oriented DBMS	0.52	+0.00	+0.02
7.	8.	6.	Objectivity/DB	Object oriented DBMS	0.39	+0.05	-0.23
8.	9.	8.	ObjectDB	Object oriented DBMS	0.37	+0.04	-0.13
9.	7.	7.	Perst	Object oriented DBMS	0.37	+0.00	-0.15
10.	11.	11.	Starcounter	Object oriented DBMS	0.31	+0.06	+0.01
11.	10.	10.	Jade	Object oriented DBMS	0.30	-0.02	-0.10
12.	12.	12.	InterSystems IRIS	Multi-model	0.15		
13.	12.	12.	VelocityDB	Multi-model	0.14	+0.01	+0.03
14.	13.	13.	Quera	Object oriented DBMS	0.10	-0.00	+0.02
15.	14.	14.	KandaDB	Object oriented DBMS	0.06	-0.01	+0.01
16.	15.	15.	godb	Object oriented DBMS	0.05	-0.02	-0.03
17.	16.	16.	Versant FastObjects	Object oriented DBMS	0.04	+0.02	-0.03
18.	17.	17.	OrigoDB	Multi-model	0.00	±0.00	±0.00

19

19

Bases de données semi-structurées



```
<exempleXML>
  <titre>Mes supers albums</titre>
  <album>
    <artiste>Rammstein</artiste>
    <titre>Mutter</titre>
    <annee>2001</annee>
  </album>
  <album annee="2017">
    <artiste>Ultra Vomit</artiste>
    <titre>Panzer Surprise !</titre>
  </album>
</exempleXML>
```

Avantages :

- Absence de schéma
- Mieux adapté au stockage de documents

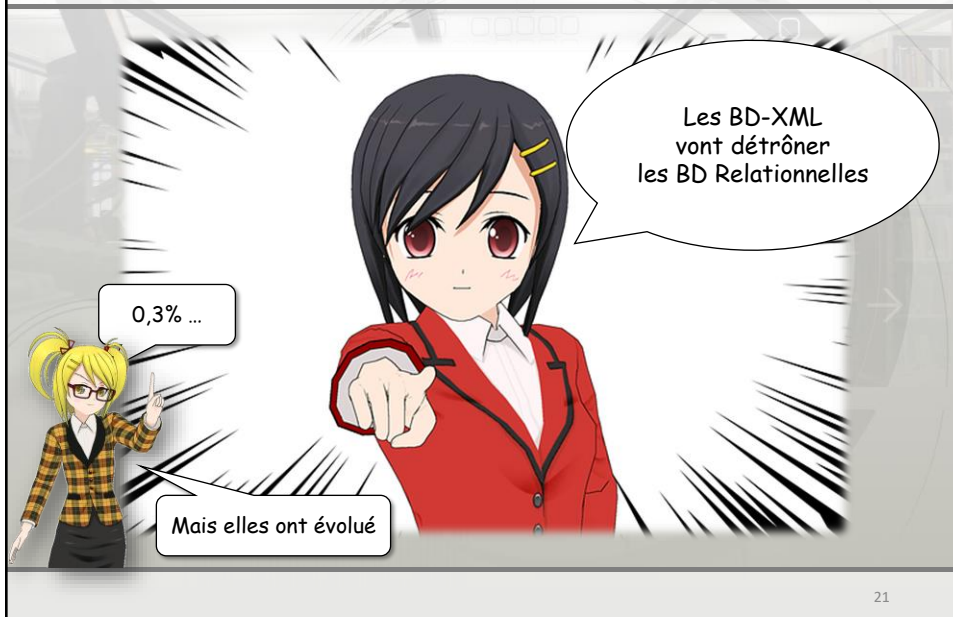
Inconvénients :

- Performances des recherches (nécessite de parser)
- Fonctionnalités peu nombreuses

20

20

Bases de données semi-structurées



21

21

Bases de données NoSQL

Not only SQL

À l'origine, destinées à manipuler des bases de données géantes (Google, Amazon.com, Facebook ou eBay)

Renoncent aux fonctionnalités classiques des SGBDR au profit de la simplicité.

Les performances "restent bonnes" avec la montée en charge (**scalabilité**) en multipliant le nombre de serveurs

Divers paradigmes :

- Key-Value Stores
- Document-Stores (évolution des BD-XML)
- Wide Column Stores
- Graph DBMS
- Etc.

On reparlera de ces systèmes dans un autre chapitre



22

22

NewSQL

	SGBDR	NoSQL	NewSQL
Fonctionnalités	✓	✗	✓
Scalabilité (horiz.)	✗	✓	✓

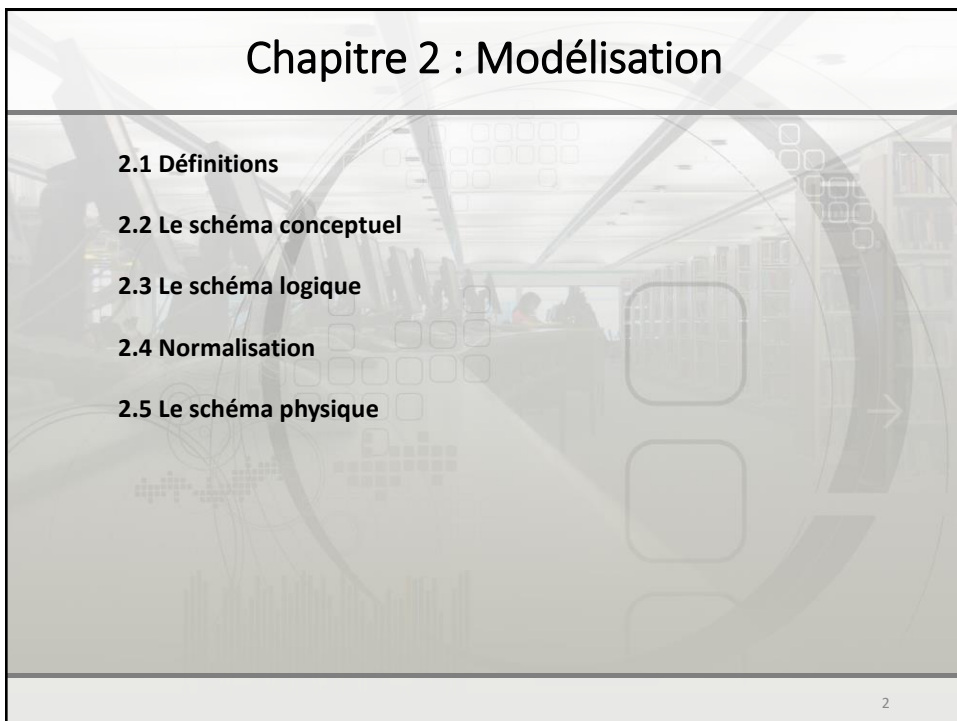
La définition de ce que doit être un système
NewSQL est relativement floue

Attention aux effets
d'annonce !

De nombreux systèmes se disent être de type NewSQL
sans cocher toutes les cases de ce que doit réellement
offrir un tel système



1



2



2. Modélisation

2.1. Définitions

3

3

Tuple (t-uple)

Néologisme basé sur le terme mathématique N-uplet : Table UPLEt

Un n-uplet est une collection ordonnée de n objets.

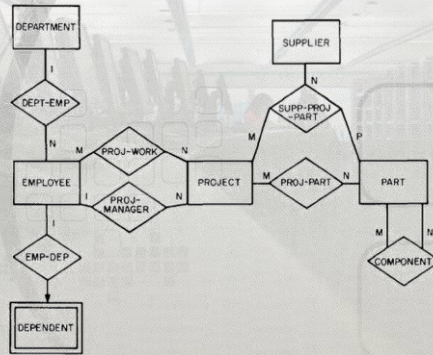
Un tuple est un enregistrement stocké dans une table (une entité)

4

4

ER Data Model

Entity-Relationship Data Model



Peter Pin-Shan Chen. 1976. The entity-relationship model—toward a unified view of data. ACM Trans. Database Syst. 1, 1 (March 1976), 9-36.

5

5

ER Data Model

Entity-Relationship Data Model

Entité-Relation ou Entité-Association

Outil de modélisation conceptuelle

Il n'y a pas de réel standard

Source d'inspiration de normes plus modernes :

- UML
- **MERISE**

Adapté aux bases de données relationnelles

6

6

MERISE

~~Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprises~~

Ce n'est pas un acronyme :

"Le merisier ne peut porter de beaux fruits que si on lui greffe une branche de cerisier : ainsi en va-t-il des méthodes informatiques bien conçues, qui ne produisent de bons résultats que si la greffe sur l'organisation réussit", R.Colletti et al., La méthode Merise : Principes et outils, t. 1, Paris, Éditions d'organisation, 1983, 328 p

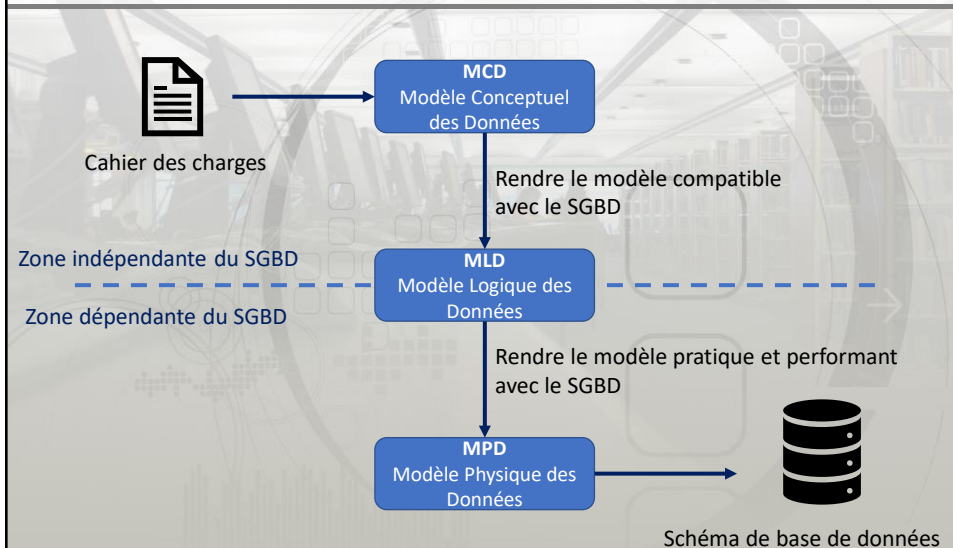
Méthode d'analyse, de conception et de gestion de projet (informatique)

Pour ce chapitre : seules les parties MERISE qui sont dédiées à la modélisation de données seront utilisées

7

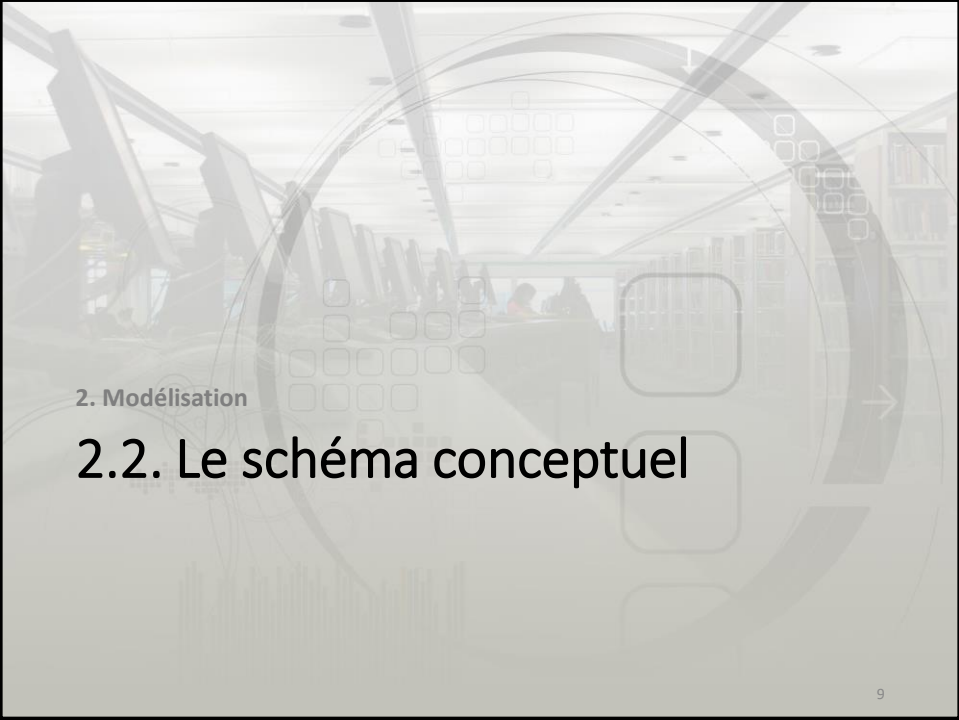
7

Processus de modélisation



8

8



2. Modélisation

2.2. Le schéma conceptuel

9

9

Vocabulaire

Entités :

- regroupements d'informations communes à un "individu"
- équivalent d'une "classe" en POO
- schématisés par un rectangle

Attributs :

- caractéristiques décrivant les entités
- vont stocker des valeurs
- représentés par une liste de mots explicites et concis

Association :

- Liaison logique entre une ou plusieurs entités
- Représentée par une ellipse (en MERISE) reliée aux entités concernées
- le nombre d'entités concernées par cette association = degré
- Doit être affublée de cardinalités aux extrémités de l'association afin d'en caractériser le type

10

10

Principe du MCD

Les entités

Les attributs (propriétés, champs)
Identifiants (CLE PRIMAIRE)

Les associations (liaisons, **relations***)

Le degré (2 entités sont impliquées
dans la relation, le degré vaut 2)

Les cardinalités (sont inversées en UML et ER-Model)

***En algèbre relationnelle, une relation décrit plutôt une collection de données**



11

11

Choix d'une clé primaire (PK, primary key)

Une propriété naturelle

Personne -> **email**

Être certain que la valeur soit unique
Éviter les modifications ultérieures

Une propriété artificielle

Client -> **ID_client**

Performances supérieures

Une propriété composée
Entreprise -> **Enseigne + localité**

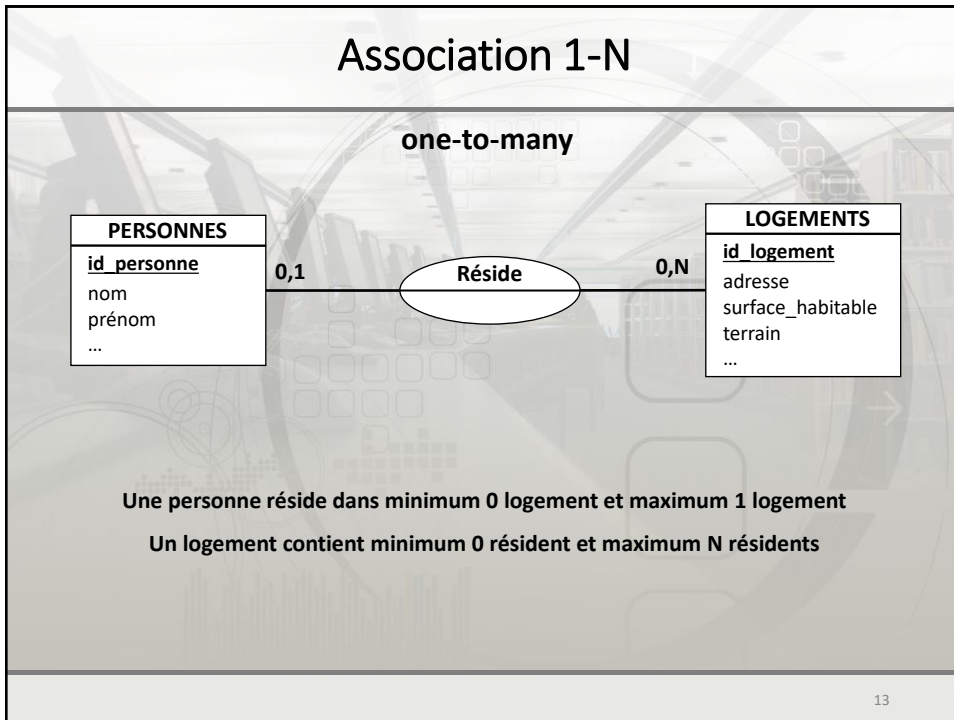
Attention aux **contraintes d'unicité**

Plusieurs tuples d'une même entité ne peuvent avoir la même valeur de clé primaire

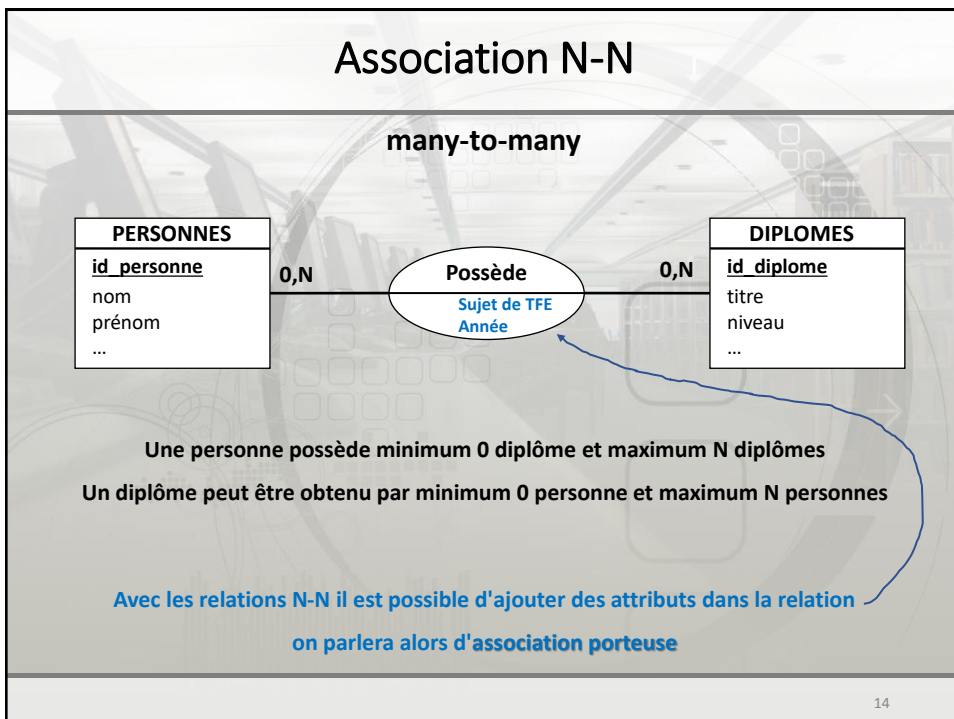
Les "multivalued attributes" ne peuvent pas être utilisés comme clé primaire

12

12



13



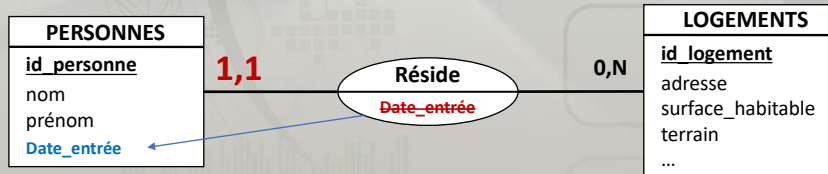
14

Attribut dans une association 1-N



Dans certains cas, il peut être avantageux de placer un attribut dans une association 1-N.

Cela n'est possible **que si cette association est optionnelle**



15

15

Association 1-1

one-to-one



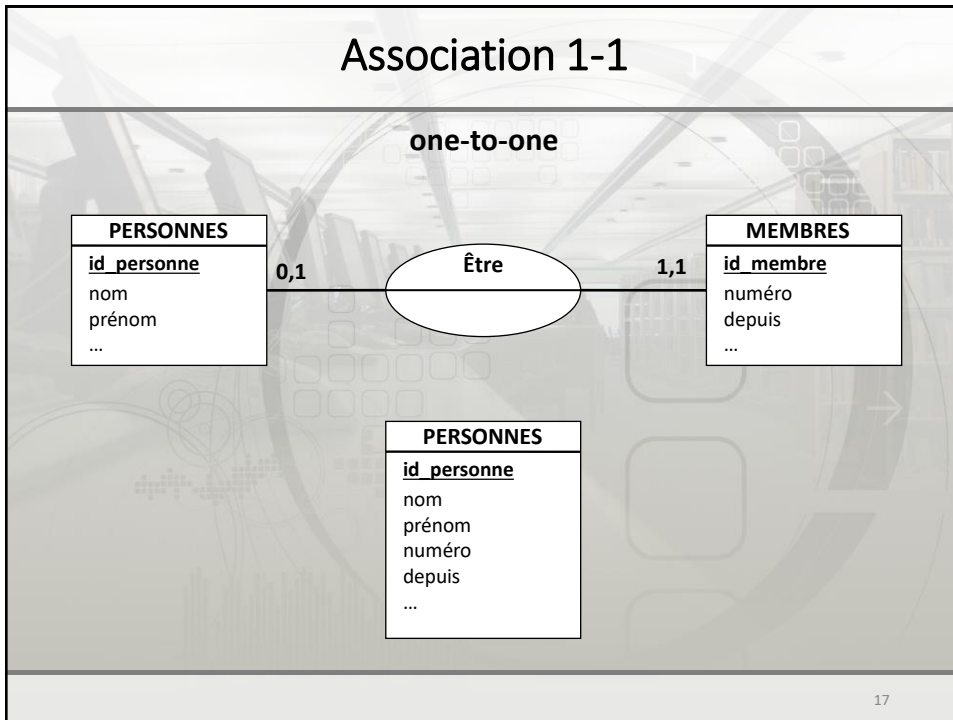
Une personne peut ou pas être membre du club

A un numéro de membre correspond d'office une personne

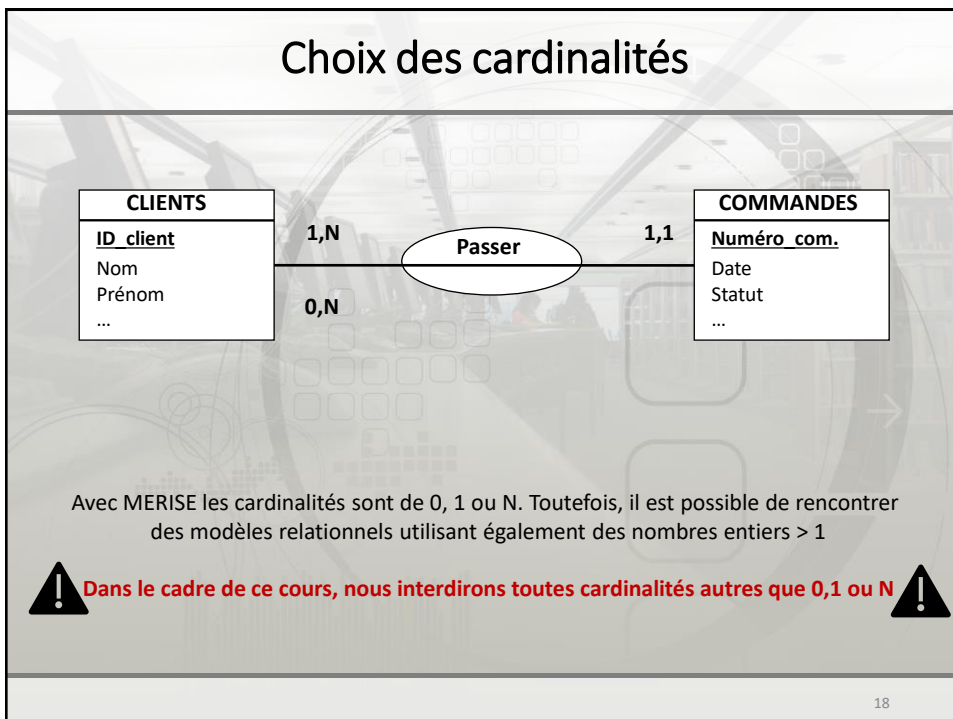
ATTENTION : les relations 1-1 peuvent être le signe d'une erreur de conception

16

16

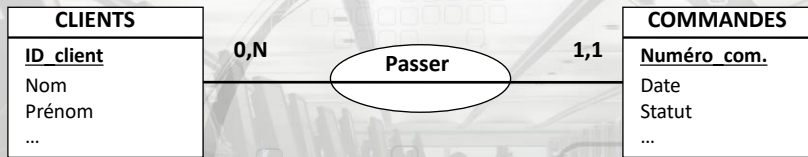


17



18

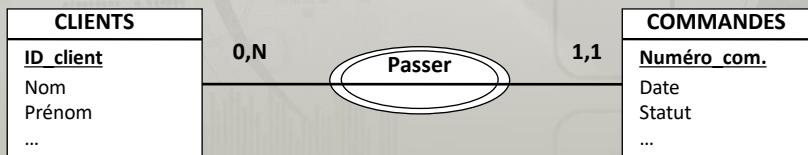
Indépendance / dépendance



CLIENTS -> Entité indépendante

COMMANDES -> Entité dépendante

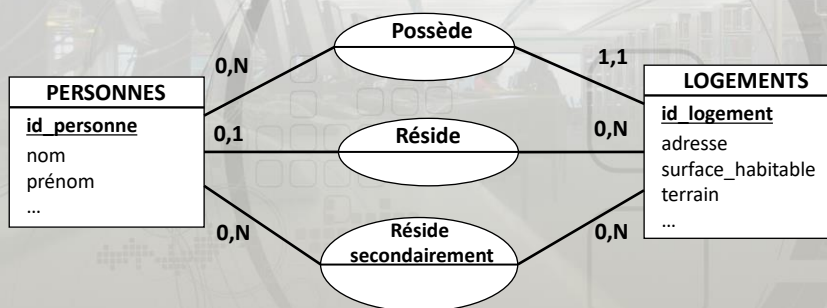
Certaines notations ER ajoutent une double bordure à l'association si elle implique une dépendance :



19

19

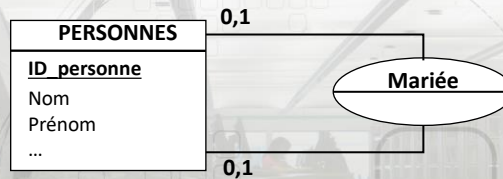
Association plurielle



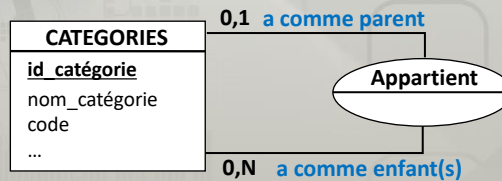
20

20

Association réflexive



Il est conseillé d'ajouter un supplément d'information (**rôle**) pour clarifier certaines associations :

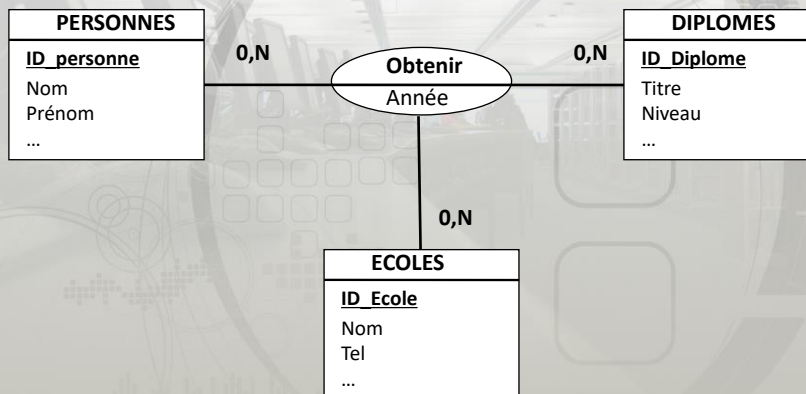


Le rôle fait partie des contraintes d'intégrité (tout comme les identifiants et les cardinalités)

21

21

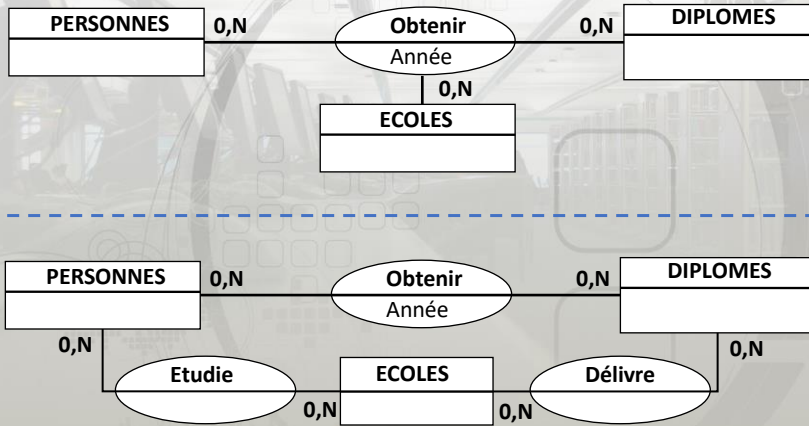
Association ternaire



22

22

Conversion : ternaire -> 3 binaires



23

23

Conversion : ternaire -> 3 binaires

OBTENIR			
Jean	Ingénieur	UMONS	2015
Sarah	Ingénieur	UMONS	2008
Jean	Electronicien	HEH	2012
Cindy	Informaticien	HEH	2012
Jean	Informaticien	UMONS	2005

ETUDIE	
Jean	UMONS
Sarah	UMONS
Jean	HEH
Cindy	HEH

OBTENIR		
Jean	Ingénieur	2015
Sarah	Ingénieur	2008
Jean	Electronicien	2012
Cindy	Informaticien	2012
Jean	Informaticien	2005

DELIVRE	
UMONS	Ingénieur
HEH	Electronicien
HEH	Informaticien
UMONS	Informaticien



Jean a obtenu son diplôme d'informaticien à l'UMONS ou à la HEH ?

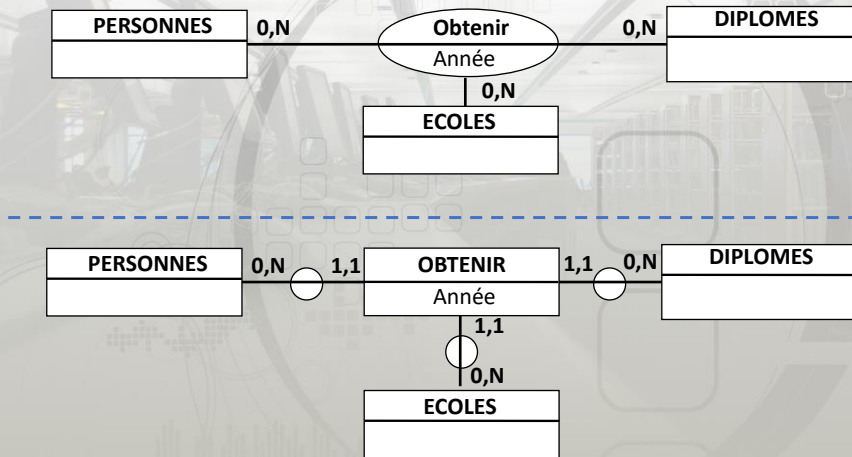
Une association ternaire n'est pas équivalente à 3 associations binaires !!!



24

24

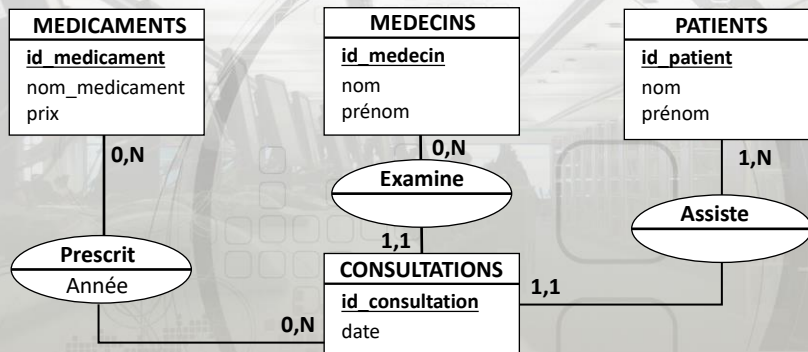
Conversion (OK) : ternaire -> 3 binaires



25

25

Exercice



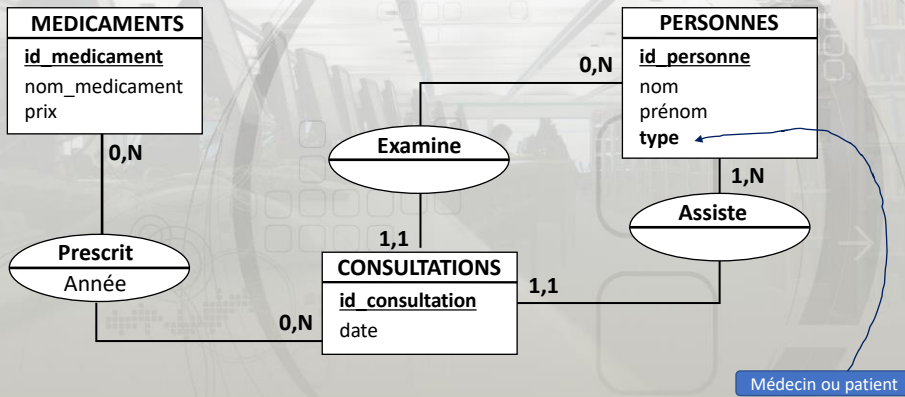
1. Un patient peut-il effectuer plusieurs visites?
2. Un médecin peut-il recevoir plusieurs patients dans la même consultation?
3. Peut-on prescrire plusieurs médicaments lors d'une même consultation?
4. Deux médecins différents peuvent-ils prescrire le même médicament?

OUI
NON
OUI
OUI

26

26

"Simplification"

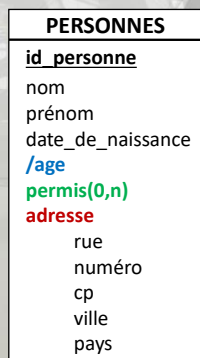


27

27

Les attributs spéciaux

Il est possible de rencontrer des modélisations ER (non MERISE) qui intègrent une notation particulière pour certains attributs :



age = attribut dérivé

calculé sur base de la date de naissance

Il est souvent préférable de ne pas stocker l'âge et de le calculer dynamiquement

permis = attribut multivalué
(multivalued attribute)

Également à éviter, sauf si vous êtes certain qu'aucune recherche ne se fera sur l'attribut

adresse = attribut composite
composé de sous-attributs

Supprimer l'attribut composite en promouvant au rang d'attribut les sous-attributs

28

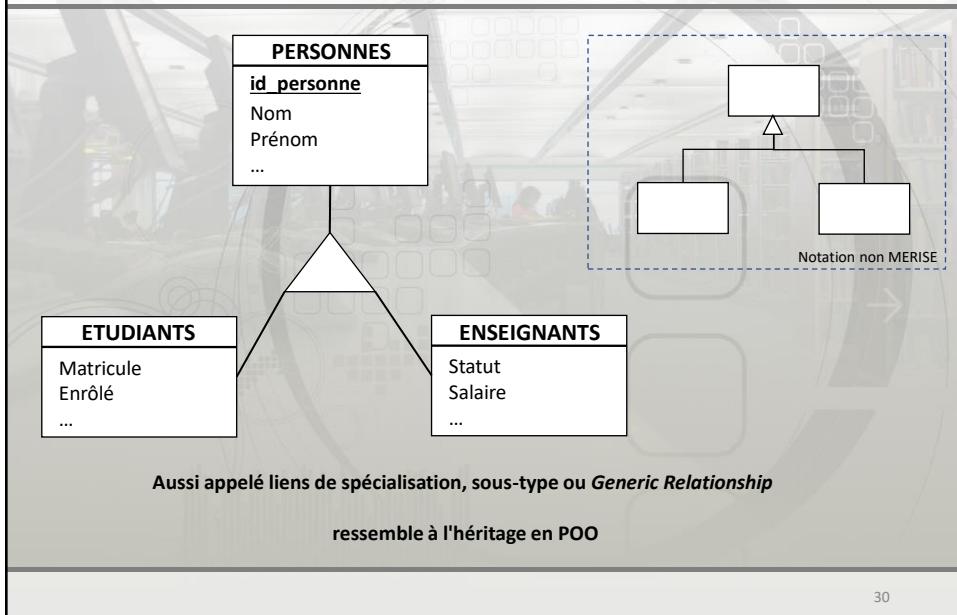
28

Attribut ou entité ?



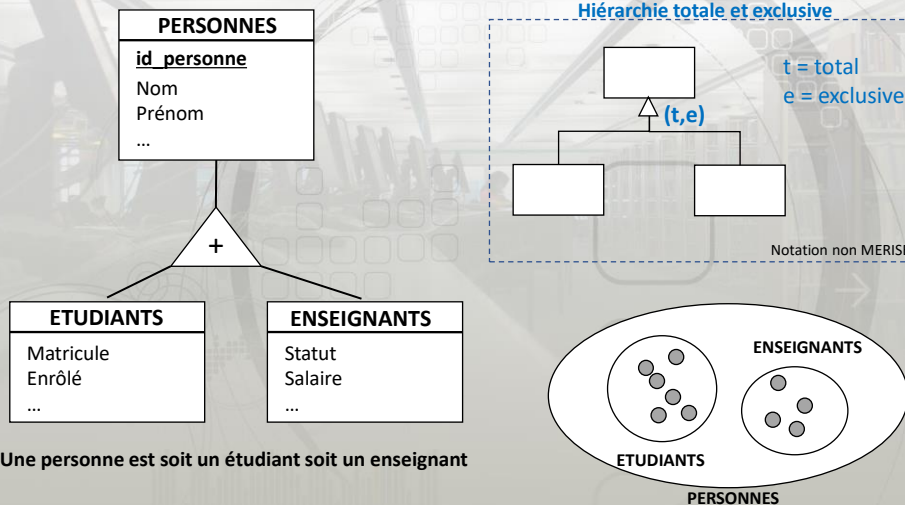
29

Liens (ou hiérarchie) de généralisation



30

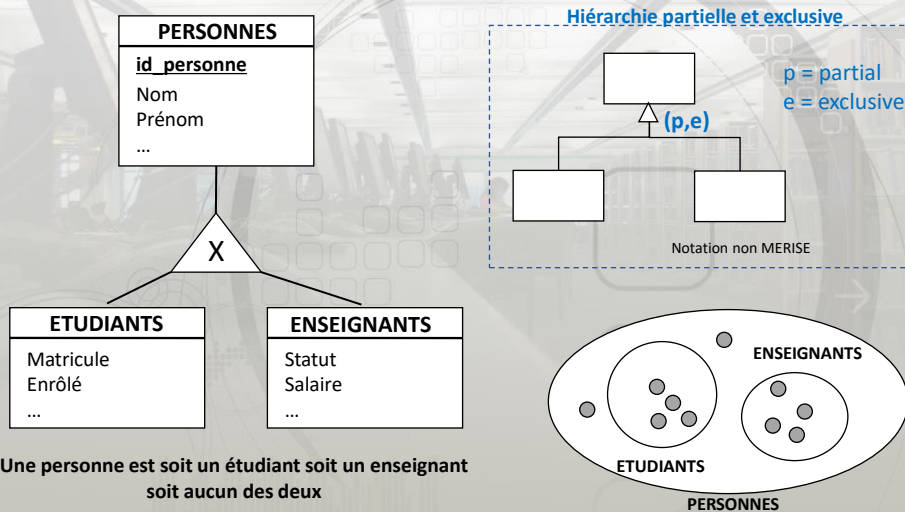
Contraintes sur les sous-types



31

31

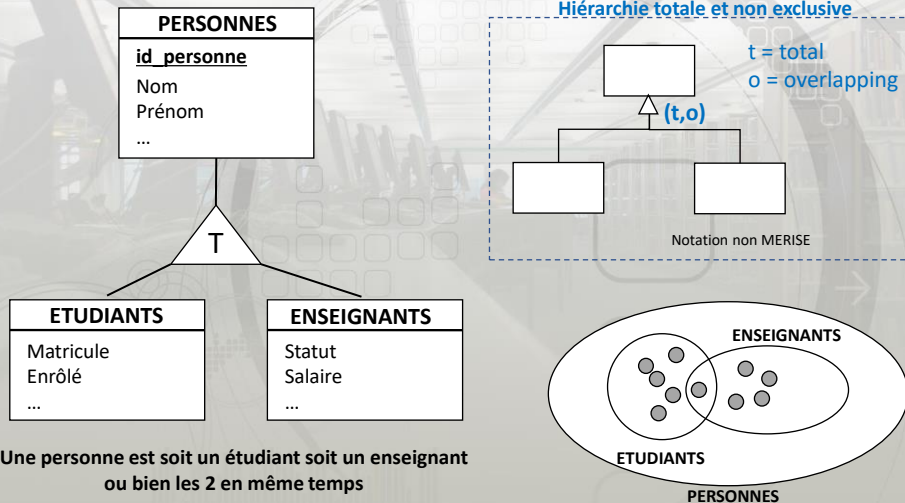
Contraintes sur les sous-types



32

32

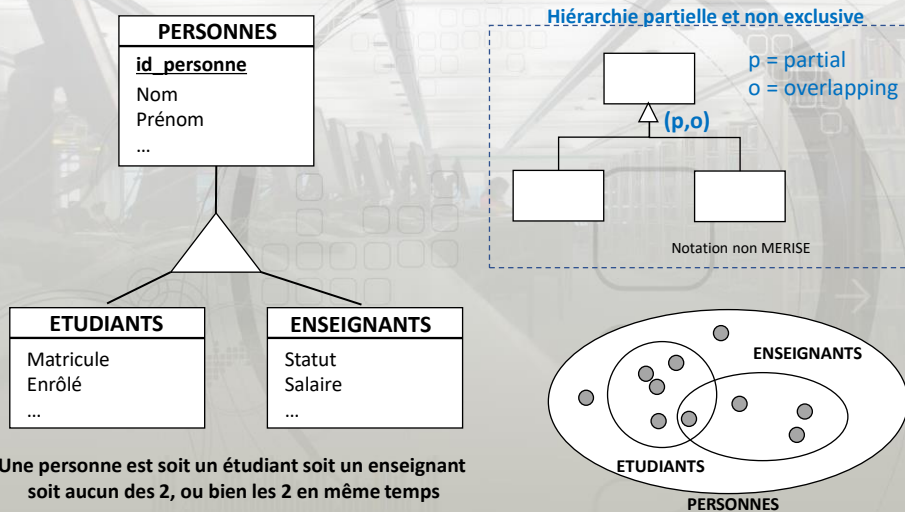
Contraintes sur les sous-types



33

33

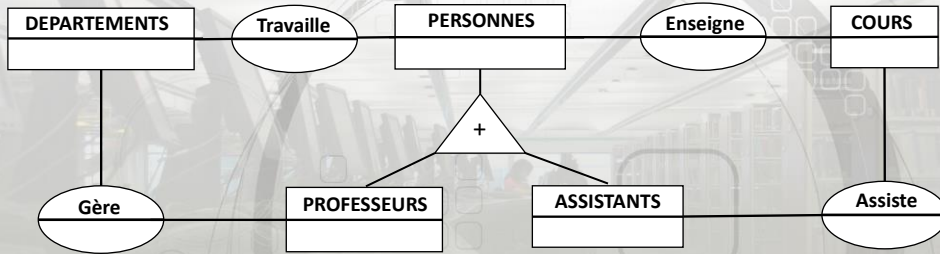
Contraintes sur les sous-types



34

34

Attention aux associations avec les sous-types



Il faut placer les associations au niveau approprié

Les professeurs et assistants travaillent tous les 2 dans des départements et enseignent des cours
Par contre, seuls les professeurs gèrent des départements et seuls les assistants assistent.

35

35

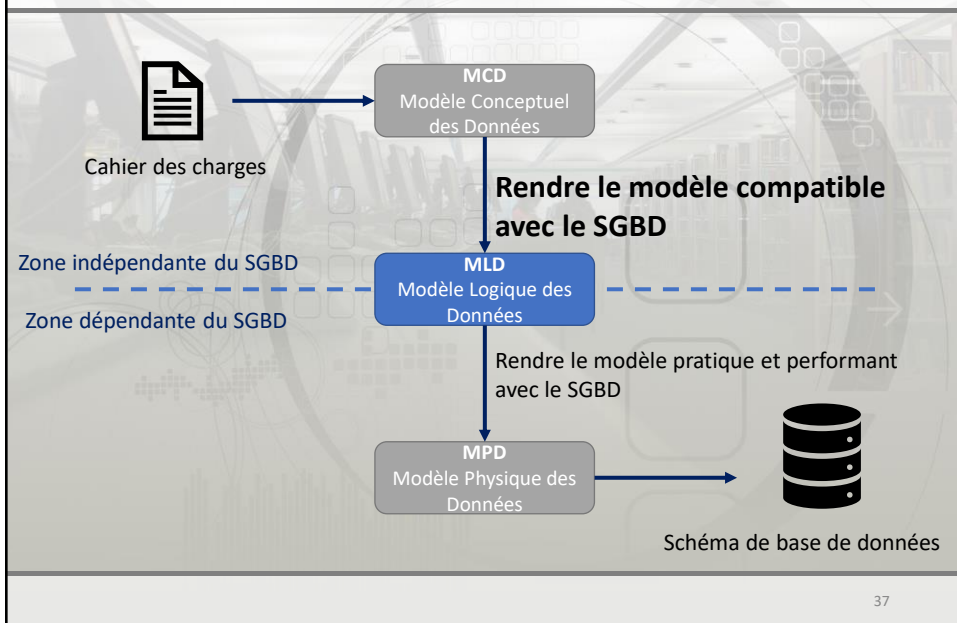
2. Modélisation

2.3. Le schéma logique

36

36

Processus de modélisation



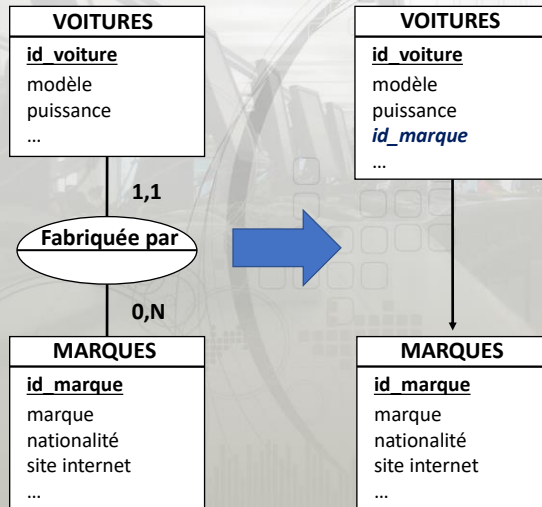
37

Le MLD c'est trop facile !



38

MCD -> MLD



Les entités deviennent des tables

Les attributs des colonnes/champs

Les identifiants deviennent des clés primaires (PK)

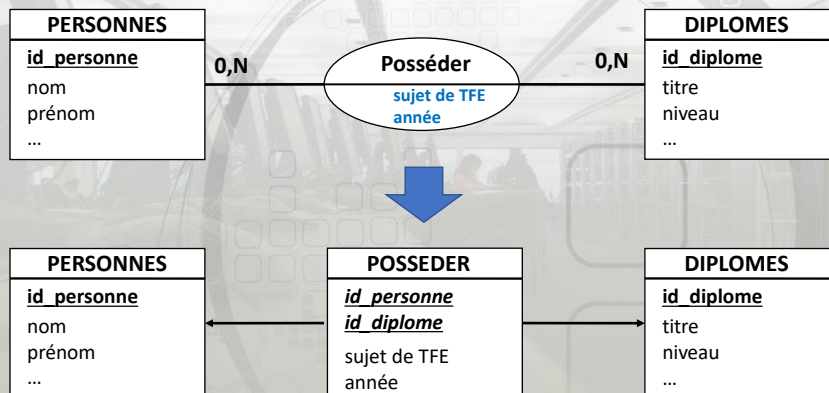
Avec les associations 1-N :

- apparition d'une clé étrangère
- FK : Foreign Key
- La clé est dans la table qui a la cardinalité maximale = 1

39

39

MCD -> MLD



Les associations N-N se transforment en une table de jointure

40

40

MCD -> MLD : conseils

Après une conversion MCD-> MLD MERISE, les noms des associations et les cardinalités disparaissent.

Cependant, je vous conseille de conserver les cardinalités et éventuellement certains noms d'association dans le but de clarifier le MLD

Il est courant de nommer les tables de jointure en utilisant les noms des associations qu'elles remplacent.

En fonction de la complexité du schéma, il peut être plus clair de nommer une table de jointure en utilisant les noms des tables jointes. Cela donnerait **personnes_diplomes** pour l'exemple précédent.



41

41

Table de jointure et PK

Généralement, 2 écoles s'affrontent :

Une clé primaire auto-incrémentée, et les clés étrangères dans la table sont de simples attributs

PERSONNES_DIPLOMES	
<u>id_personnes_diplomes</u>	
<u>id_personne</u>	
<u>id_diplome</u>	
sujet de TFE	
année	

Une clé primaire composée des clés étrangères

PERSONNES_DIPLOMES	
<u>id_personne</u>	
<u>id_diplome</u>	
sujet de TFE	
année	

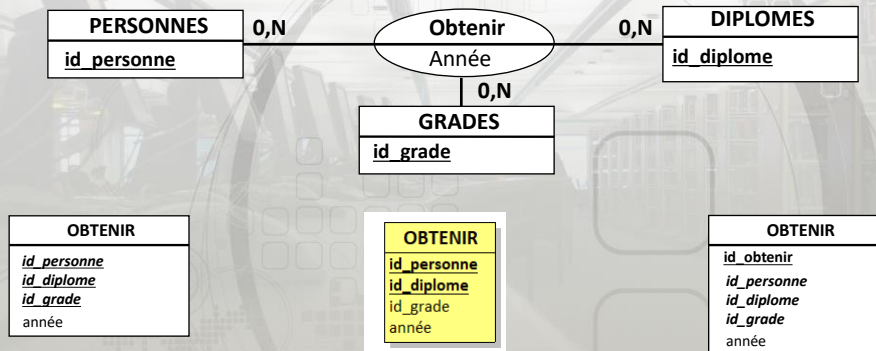
De cette façon l'indexation est plus efficace et la table plus compacte. Par contre, ce n'est pas toujours possible...



42

42

Table de jointure et PK



Si un grade est toujours renseigné pour tous les diplômes alors la PK peut être composée des FK

Si des diplômes peuvent être obtenus sans grade, alors il ne sera pas possible d'utiliser une PK composée. -> Les attributs qui composent une PK ne peuvent pas être NULL

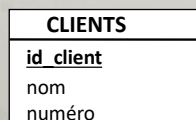
43

43

MCD -> MLD : associations 1 - 1



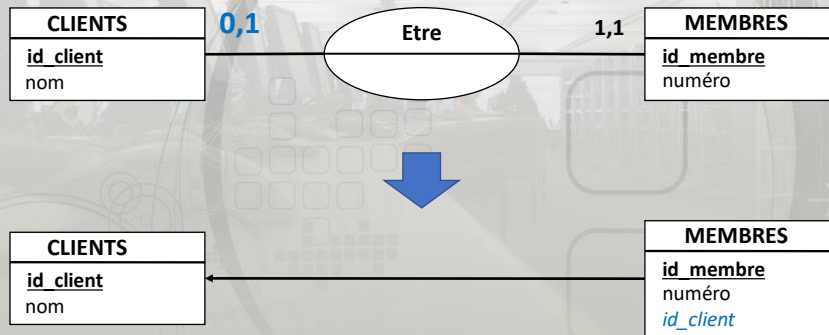
Si vous avez ce type d'association, c'est que votre MCD n'est pas correct



44

44

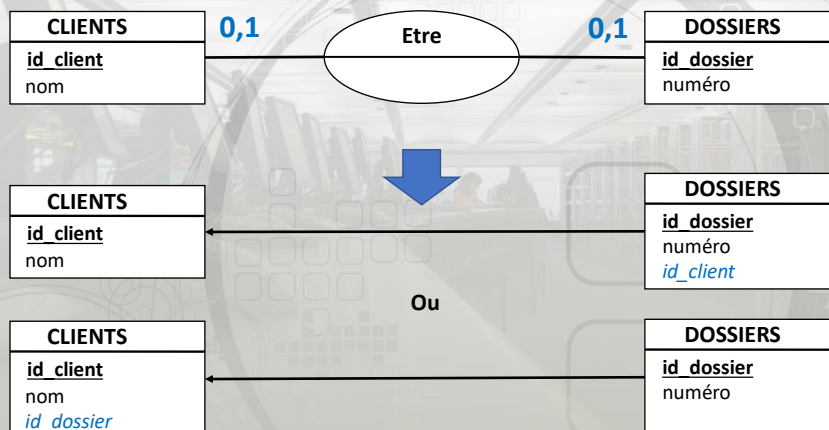
MCD -> MLD : associations 1 - 1



45

45

MCD -> MLD : associations 1 - 1



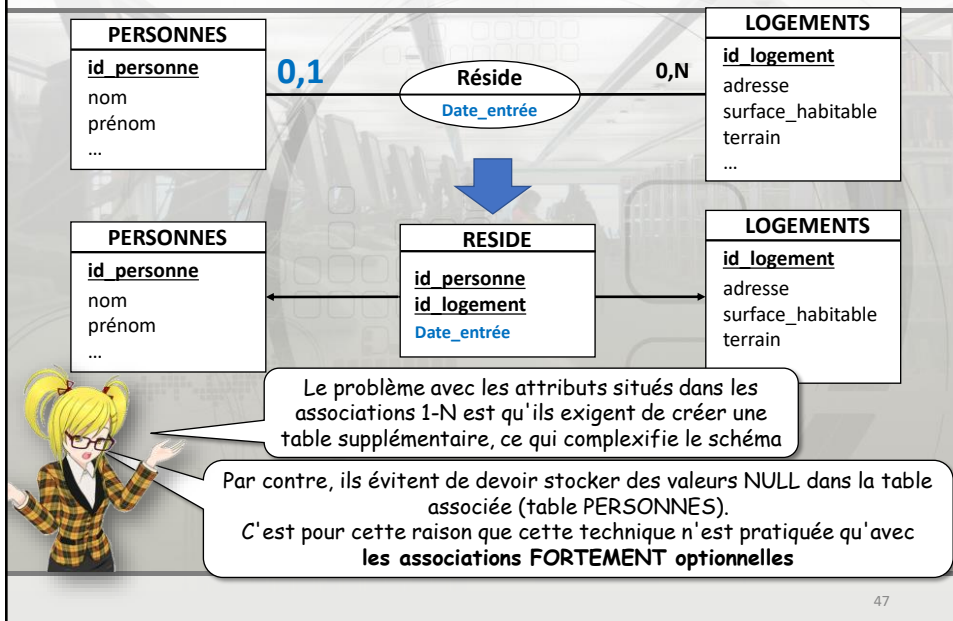
Choisir la solution qui génère le moins de NULL.

Si l'association est FORTEMENT occasionnelle, une table de jointure peut être envisagée

46

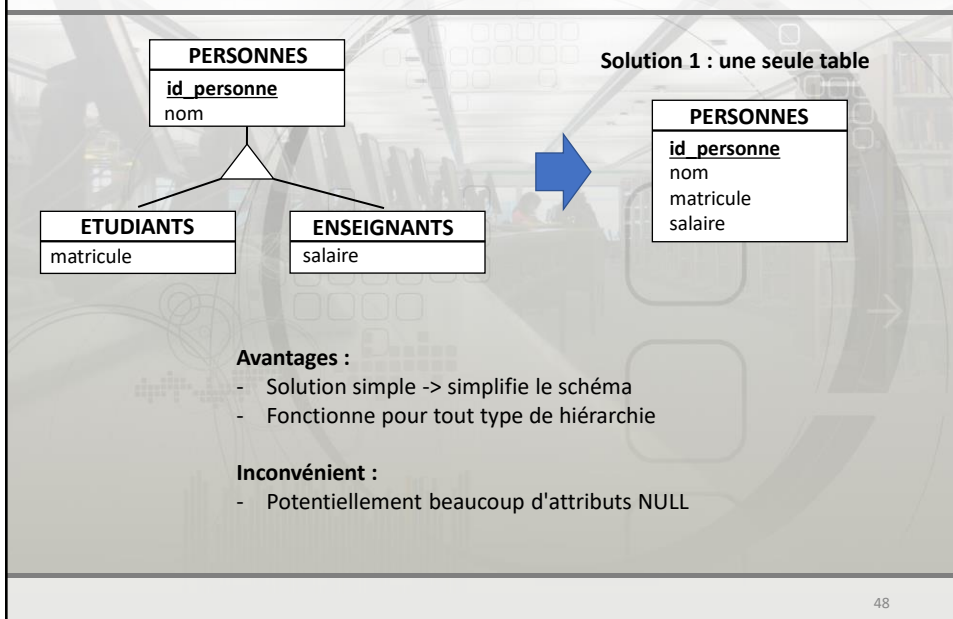
46

Attribut dans une association 1-N



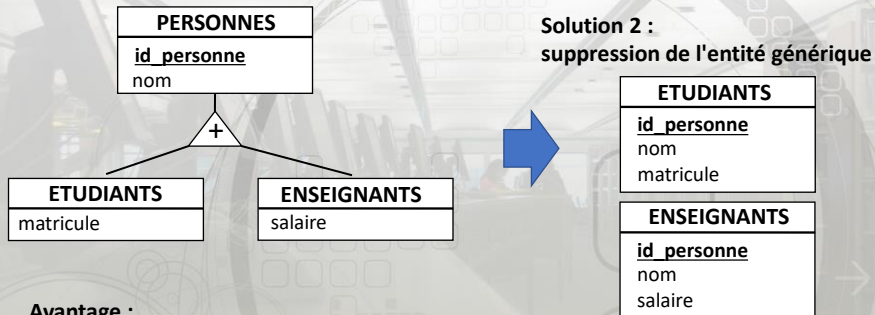
47

Traduction des hiérarchies de généralisation



48

Traduction des hiérarchies de généralisation



Avantage :

- ...

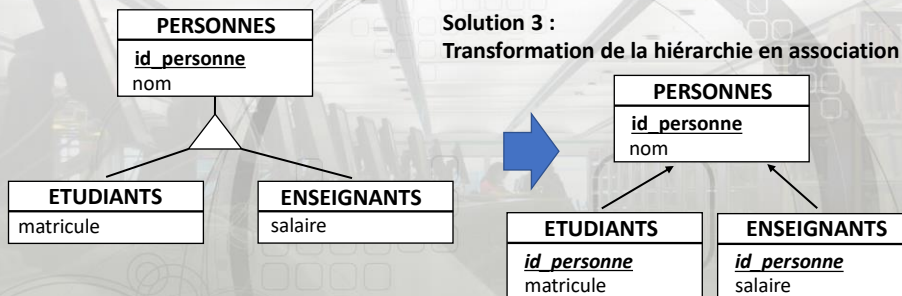
Inconvénients :

- Ne fonctionne qu'avec une généralisation totale et exclusive (+)
- Cela peut compliquer le système (if else if else if...)
- Perte de sémantique (la notion de PERSONNES)

49

49

Traduction des hiérarchies de généralisation



Avantages :

- Fonctionne pour tout type de hiérarchie
- Pas de perte de sémantique

Inconvénient :

- Le schéma résultant est plus complexe qu'avec l'approche 1

50

50

Traduction des hiérarchies de généralisation

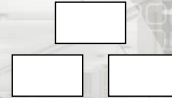
Solution 1



Solution 2



Solution 3



La solution 3 est souvent l'approche à privilégier sauf pour certains cas bien précis.

La solution 2 n'est applicable que sur les hiérarchies totales et exclusives et est à préférer uniquement quand la majorité des traitements accèdent simultanément aux attributs génériques et spécifiques (ce qui permet d'éviter des jointures et présente une meilleure compacité des données que la solution 1)

Si la majorité des traitements d'écriture se font simultanément sur des groupements de tuples et sur l'ensemble des attributs génériques et spécifiques, alors la solution 1 est conseillée (meilleures performances)

La solution 1 est aussi à privilégier s'il n'y a que peu d'attributs spécifiques



51

51

2. Modélisation

2.4. Normalisation

52

52

Dépendance fonctionnelle

ARTICLES
<u>id_article</u>
nom
référence
prix
numéro_catégorie
libellé_catégorie

Emmental 4578912 1,25 €	Brie 1256912 0,98 €	Pink Lady 4568941 0,85 €
4 Fromage	4 Fromage	7 Fruits

On dit qu'il existe une dépendance fonctionnelle entre un attribut A1 et un attribut A2, on note $A1 \rightarrow A2$ si, connaissant une valeur de A1, on ne peut lui associer qu'une seule valeur de A2.

On dit aussi que A1 détermine A2.

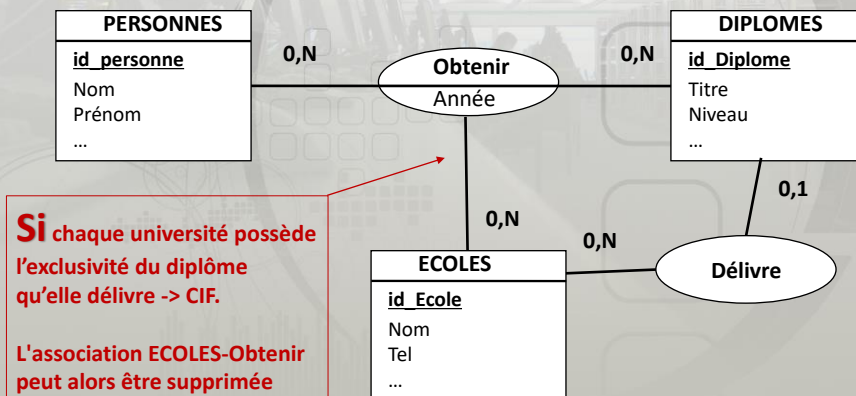
A1 est la source de la dépendance fonctionnelle et A2 le but.

53

53

Contrainte d'intégrité fonctionnelle

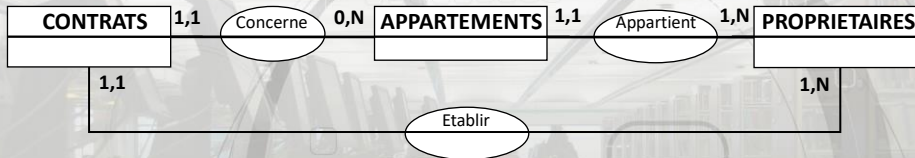
se définit par le fait que l'une des entités participant à l'association est complètement déterminée par la connaissance d'une ou de plusieurs autres entités participant dans cette même association



54

54

Association transitive



L'association qui relie l'entité « CONTRAT » et l'entité « PROPRIETAIRE » est une **association transitive**

Elle **doit être ôtée du modèle** car le propriétaire peut être retrouvé à partir des associations « Concerne » et « Appartient ».

Les associations transitives surchargent le MCD inutilement et peuvent être la porte d'entrée aux d'incohérences

55

55

La normalisation

Quoi :

- Règles de bonne pratique

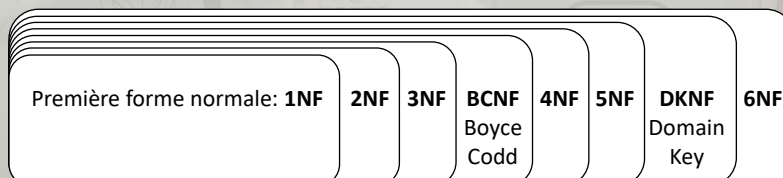
But :

- Améliorer la modélisation
- Éviter les redondances des données (éviter les problèmes de mise à jour)

Contexte :

- Principalement OLTP

Niveaux :



56

56

Première forme normale : 1FN

id	nom	email
1	M. Frank Dewit	f.dewit@gmail.com, zedewit@hotmail.com
2	M. Ivan Raoul	rivan@hotmail.fr
3	Mme Leen Breckx	Leen.breckx@gmail.com, brekyleen@live.fr

Les attributs doivent être atomiques :

- Supprimer les attributs composites en promouvant au rang d'attribut les sous-attributs
- Les attributs multivalués donnent naissance à de nouvelles entités

id	titre	prenom	nom
1	M.	Frank	Dewit
2	M.	Ivan	Raoul
3	Mme	Leen	Breckx

0,N

1,1

id	email
1	f.dewit@gmail.com
2	zedewit@hotmail.com
3	rivan@hotmail.fr
4	Leen.breckx@gmail.com
5	brekyleen@live.fr

57

57

Deuxième forme normale : 2FN

Respecter la 1FN

Les attributs doivent dépendre complètement de la clé et non partiellement

Ce cas se rencontre principalement avec les clés composées.

Exemple avec une entité qui contient les évaluations des films par des utilisateurs :

id_film	id_user	note	genre
1	1	3	Comédie
2	1	5	Action
2	8	4	Action
3	8	1	Aventure

Pas en 2FN !

L'attribut "genre" est propre au film (id_film) et ne dépend pas de l'utilisateur (id_user), cet attribut n'est pas dans la bonne entité.

58

58

Troisième forme normale : 3FN

Respecter la 2FN

Pas de dépendance fonctionnelle entre les attributs non clé

Exemple avec une entité qui contient les évaluations des films par des utilisateurs :

id_film	id_user	note	image_note
1	1	3	3etoiles.png
2	1	5	5etoiles.png
2	8	4	4etoiles.png
3	8	1	1etoiles.png

Pas en 3FN !

id_film	id_user	note
1	1	3
2	1	5
2	8	4
3	8	1

1,1 0,N

note	image_note
3	3etoiles.png
5	5etoiles.png
4	4etoiles.png
1	1etoiles.png

59

59

Forme normale de Boyce-Codd : BCNF

Respecter la 3FN

Les attributs non clé ne doivent pas être source de dépendance fonctionnelle vers une partie de la clé

Cela peut être interprété comme la contrainte inverse de la 2FN.

Exemple avec une entité contenant un historique des transports :

id_pays	id_user	date	transport	ville
1	1	10/10/2018	Voiture	Bruxelles
1	2	11/10/2018	Bus	Liège
3	3	11/10/2018	Avion	Boston
4	2	11/11/2018	Voiture	Paris

Dépendances fonctionnelles :

<u>id_pays, id_user</u>	-> date	OK
<u>id_pays, id_user</u>	-> transport	OK
ville	-> <u>id_pays</u>	Pas en BCNF

60

60

Quatrième forme normale : 4FN

Respecter la BCNF

Une relation est en 4FN si pour chaque dépendance multivaluée $X \twoheadrightarrow Y$ non triviale, X est une superclé de la relation

...

Exemple avec l'entité "livres" :

Les "formes normales" font partie de ces choses très complexes à formuler mais qui sont plutôt simples à mettre en œuvre



isbn	auteur	mot clé
1-12312-323-2	Marc	Tortue
1-12312-323-2	Michel	Tortue
1-12312-323-2	Marc	Natation
1-12312-323-2	Michel	Natation
8-12312-323-4	Alice	Hiver

Pas en 4FN !



isbn	auteur
1-12312-323-2	Marc
1-12312-323-2	Michel
8-12312-323-4	Alice

isbn	mot clé
1-12312-323-2	Tortue
1-12312-323-2	Natation
8-12312-323-4	Hiver

61

61

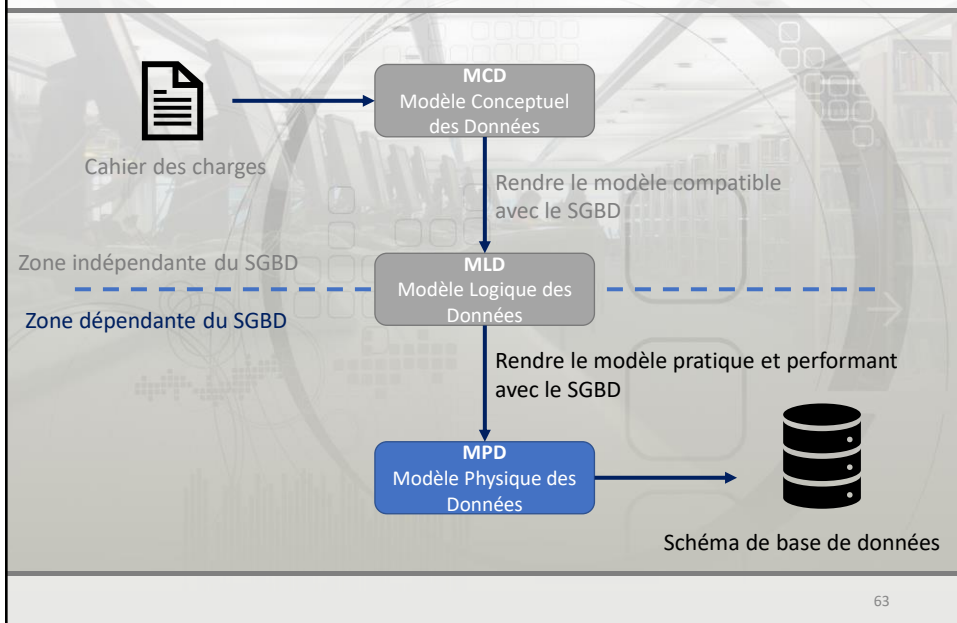
2. Modélisation

2.5. Le modèle physique

62

62

Processus de modélisation



63

Le modèle physique

Généralement, 2 écoles s'affrontent :

Le modèle physique c'est juste l'implémentation du MLD dans le SGBDR, donc pas besoin de le dessiner

Le modèle physique c'est :
- Préciser le format de stockage des attributs
- Dénormaliser la BD afin de rendre le système moins complexe et/ou plus performant
- Renseigner les index, vues, triggers, etc.

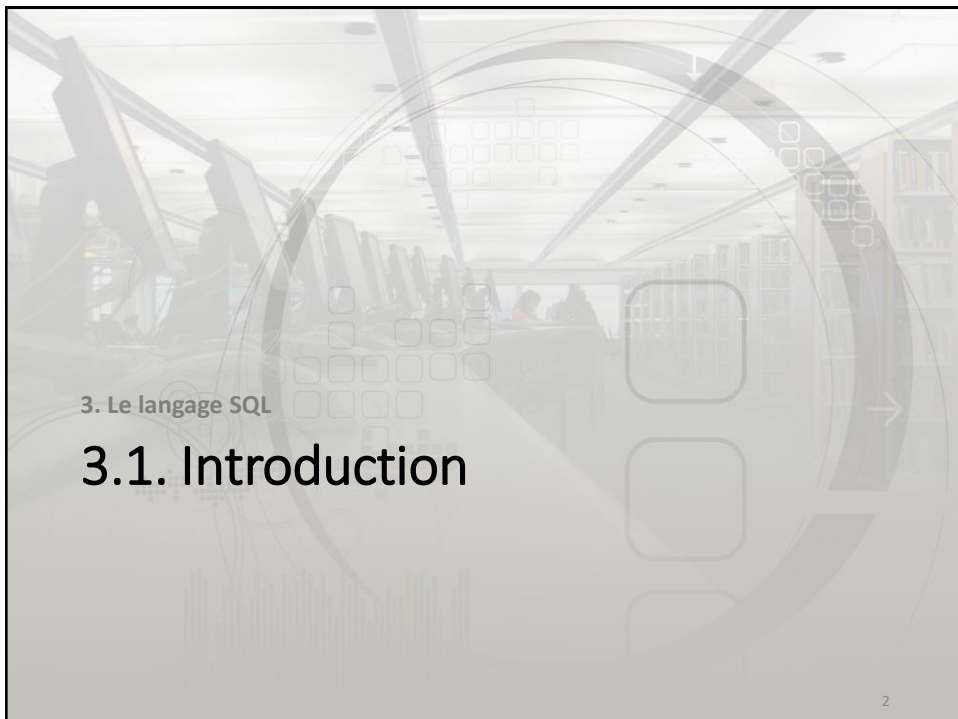
Donc il faut le dessiner !

Voyez le MPD comme un MLD plus détaillé

64



1



2

Le langage SQL

Structured Query Language

Est souvent prononcé "**SEQUEL**" en anglais

Exploiter les BD relationnelles

Langage de programmation **déclaratif**

Sa structure ressemble à des constructions de phrases

LE STANDARD INDUSTRIEL

3

3

Le langage SQL

SQL n'est pas un langage de programmation !

Si

SQL est un vrai langage de programmation. Il est mathématiquement complet depuis la norme 1999.

4

4

Mathématiquement complet ?

Le SQL permet la récursivité

Peut traiter tout problème théoriquement solvable en une seule requête*

Rares sont ceux qui font le tour du SQL

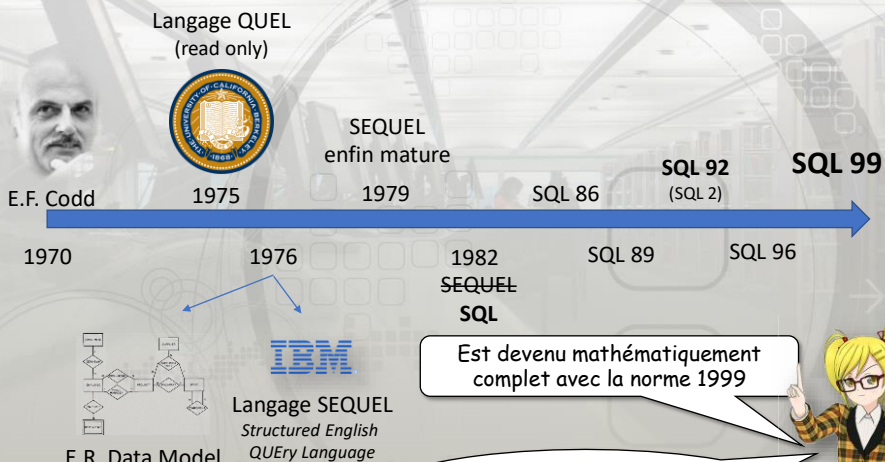
C'est pourtant un langage très puissant qui, une fois "maîtrisé", permet d'éviter de nombreuses lignes de programmation*.

*** à condition que la BD soit correctement structurée et alimentée**

5

5

Historique



6

6

Le standard SQL

Pas respecté par les SGBDR

Par exemple :

MySQL : +/-SQL 1992 (+ quelques fonctionnalités maison)

MariaDB : +/-SQL 1999 (+ quelques fonctionnalités maison)

SQLite : fait à sa mode (par exemple, tout est dynamiquement typé)

On parle de **dialecte SQL**

7

7

Préliminaires

Indépendance entre la vue logique de la BD et son implémentation physique

Les tables (données et structure) ne sont **pas naturellement ordonnées**

Le SQL s'inspire de **l'algèbre relationnelle** lui-même basé sur la **théorie des ensembles**

Les résultats sont récupérés **sous forme de table**

Essayez **d'abord d'obtenir une requête qui fonctionne** avant de l'optimiser.

Les performances obtenues dépendent de la **requête**, du **schéma**, et également des propriétés **statistiques des données**

8

8

Utilisation du SQL

Il existe 4 façons d'intégrer le SQL dans un logiciel :

Utilisation d'un connecteur de BD avec écriture des requêtes sous forme de chaînes de caractères.

Embedded SQL : intégration du langage SQL directement dans le code source de l'application.

Utilisation d'un ORM (Object Relationnal Mapping) qui permet de générer automatiquement le SQL via une couche d'abstraction Objet

Principe des bases de données épaisses : le SQL n'est pas dans l'application mais dans le SGBDR (utilisation de procédures stockées)

9

9

Connecteur de BD

Le connecteur se connecte :

- soit à un *driver* dédié à un SGBD particulier
- soit à un *middleware* plus générique (par exemple : ODBC) qui permet de se connecter à son tour à un *driver* dédié

Les requêtes sont écrites sous forme de chaînes de caractères envoyées comme paramètre de fonction/méthode au connecteur

```
PreparedStatement stmt = conn.prepareStatement(
    "SELECT MAX(salaire), AVG(salaire)" +
    " FROM employees");
rs = stmt.executeQuery();

maxSalary = rs.getBigDecimal(1);
avgSalary = rs.getBigDecimal(2);

rs.close();
stmt.close();
```

Exemple JAVA avec JDBC

10

10

Embedded SQL

Intégration du langage SQL directement dans le code source de l'application.

Dans ce cas, le SQL est analysé par le compilateur/interpréteur de l'application

```
#sql [ctx] {  
    SELECT MAX(salaire), AVG(salaire)  
    INTO :maxSalary, :avgSalary  
    FROM employes  
};
```

Exemple JAVA avec SQLJ

Ne pas confondre avec Embedded RDBMS où le SGBDR est embarqué dans l'application (l'application contient l'intégralité du SGBDR -> exemple : SQLite)

11

11

ORM : Object-Relational Mapping



12

12

ORM : Object-Relational Mapping

Mapping objet-relationnel

Couche logicielle entre l'application et le SGBDR permettant d'accéder à la base de données et son contenu en utilisant des objets.

Le SQL est généré automatiquement

Hibernate, Entity Framework et Doctrine sont les plus connus



Les ORM introduisent une couche logicielle supplémentaire. Les requêtes générées sont rarement optimisées ce qui conduit à d'inévitables soucis de performances pour les systèmes complexes.

Il y a des ORM qui ne gèrent pas le multithreading (ou mal).

Il y en a même où il faut recompiler toute l'application en cas de modification du schéma (même mineure)

13

13

Bases de données épaisses

La BD est accédée uniquement par des :

- **Vues :** une table virtuelle définie par une requête SQL stockée côté serveur
- **Procédures Stockées :** fonctions précompilées et stockées côté serveur. Elles peuvent contenir du SQL ainsi que du code impératif (procédural)

Rend le code de l'application agnostique du schéma de la BD
(bien plus qu'avec un ORM qui lui... **ne l'est pas**)



Cette approche est souvent la meilleure façon de procéder.

Par contre, elle est plus complexe à mettre en œuvre et requiert donc plus de "skill"

14

14

Subdivision du SQL

DDL
Data Definition Language

Création et modification de la BD
CREATE, ALTER, DROP, TRUNCATE

DML
Data Manipulation Language

Traitement des données
INSERT, UPDATE, DELETE, SELECT

DCL
Data Control Language

Gestion des droits d'accès
GRANT, REVOKE

TCL
Transaction Control Language

Contrôle la bonne exécution des transactions
SET TRANSACTION, COMMIT, ROLLBACK

SQL/PSM
Persistent Stored Module

Langage procédural pour les procédures stockées
BEGIN, END, IF THEN, ELSE, etc.

15

15

3. Le langage SQL

3.2. Data Definition Language

16

16

DDL : Data Definition Language

Ce sont les instructions SQL qui permettent :

- La création de la BD: `CREATE DATABASE ma_bd;`
- La création des tables : `CREATE TABLE ma_table (...);`
- Modification des tables : `ALTER ma_table ...`
- Effacer une base de données : `DROP DATABASE ma_vieille_bd;`
- Effacer une table : `DROP TABLE ma_vieille_table;`
- Effacer le contenu d'une table : `TRUNCATE TABLE ma_table_pleine;`

Il n'y a pas que les tables mais également les **index** (voir chapitre sur l'indexation), les **assertions**, **vues**, **fonctions**, **trigger** et **procédures** (voir chapitre sur les fonctionnalités avancées).

17

17

Création d'une table

```
CREATE TABLE ma_table (  
    id_table INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
    nom VARCHAR(100) NOT NULL,  
    prenom VARCHAR(100) NOT NULL,  
    date_de_naissance DATE  
);
```

Possible de rajouter des contraintes de valeur, des index, des clés étrangères, des commentaires, etc.

Attention aux mots réservés,
aux accents et aux espaces !

Si vous avez oublié quelque chose -> ALTER TABLE

Ce qui suit est valable avec MySQL et MariaDB.

Notez qu'avec MySQL et MariaDB : choix d'un **moteur de stockage**



18

18

Types de données textuelles

CHAR(taille)	Chaîne de caractère de taille fixe (max 255)
VARCHAR(taille)	Chaîne de caractère de taille variable (max 65 535)
TINYTEXT et TEXT	Sont équivalents à du VARCHAR(255) et VARCHAR(65535)
MEDIUMTEXT	Max $2^{24}-1$ caractères (16 777 215)
LONGTEXT	Max $2^{32}-1$ caractères (4 294 967 295)
BINARY(taille)	Équivalent à du CHAR mais pour des données binaires
VARBINARY(taille)	Équivalent à du VARCHAR mais pour des données binaires
BLOB	Binary Large Objects, max ~64ko
MEDIUMBLOB	Max ~16 Mo
LOBLOB	Max ~4 Go
ENUM('a','b',..., 'n')	Énumération de maximum 65 535 valeurs.
SET	Comme une énumération sauf que plusieurs valeurs peuvent être stockées.

Le type SET n'est pas très "Forme Normale" !



19

19

Attention à l'interclassement

Aussi appelé **collation**

Table de correspondance spécifique à un jeu de caractères.

Un jeu de caractères peut avoir plusieurs interclassements

Permet l'ordonnancement correct d'une liste de caractères

Indique, par exemple que le "é" vient après le "e", ou que les majuscules viennent avant les minuscules

Permet de savoir quand un caractère est "équivalent" à un autre.

Le caractère "e" est équivalent à "E", mais aussi à "é", "è" ou "ê".

Choix de l'interclassement en fonction des types de recherche et de tri les plus fréquents.

Avec MySQL/MariaDB, un des plus complet est : **utf8mb4_unicode_520_ci**

20

20

Les types numériques

TINYINT	Entier 8 bits	} Peuvent être signés (par défaut) ou non (UNSIGNED)
SMALLINT	Entier 16 bits	
MEDIUMINT	Entier 24 bits	
INT	Entier 32 bits	
BIGINT	Entier 64 bits	
FLOAT	Réel simple précision 32 bits	
DOUBLE	Réel double précision 64 bits	
BIT(taille)	Valeur binaire de <i>taille</i> bits (max 64)	
NUMERIC(taille)	Entier de <i>taille</i> chiffres	
DECIMAL(taille,n)	Réel de <i>taille</i> chiffres et de <i>n</i> chiffres après la virgule (inclus dans <i>taille</i>)	

Les valeurs de type NUMERIC et DECIMAL étant stockées sous forme de DCB, effectuer des opérations mathématiques avec peut avoir un impact non négligeable sur les performances



21

21

Les types temporels

DATE	Date au format 'aaaa-mm-jj' Bornes : 1000-01-01 et 9999-12-31
DATETIME	Date avec heure au format 'aaaa-mm-jj hh:mm:ss' Précision max = microsonde -> 'aaaa-mm-jj hh:mm:ss.uuuuuu' Bornes : 1000-01-01 00:00:00.000000 et 9999-12-31 23:59:59.999999
TIMESTAMP	Même format d'affichage que datetime mais d'un point de vue stockage c'est le temps écoulé de puis le 1 ^{er} janvier 1970. Attention à la borne max 2038-01-19 03:14:07
TIME	Durée au format 'HH:MM:SS' ou 'HH:MM:SS.uuuuuu', peut être négatif et est borné entre -838:59:59.000000 et 838:59:59.999999
YEAR	Année au format 'aaaa' , bornée entre 1901 et 2155

Il existe également des **types spatiaux** (GEOMETRY, POINT, LINESTRING, POLYGON, etc.) et le type **JSON** (ce qui peut transformer MySQL et MariaDB en "document-store")

22

22

3. Le langage SQL

3.3. Data Manipulation Language

23

23

Insertion des données



```
INSERT INTO ma_table VALUES ( 10 , 'Marc' , 'Dufour' , NULL , '1988-10-19' );
```



```
INSERT INTO ma_table (id, nom, prenom, email, date_naissance)  
VALUES ( 10 , 'Marc' , 'Dufour' , NULL , '1988-10-19' );
```

Si vous "hard-codez" votre requête dans un soft et qu'un attribut est rajouté à posteriori dans *ma_table*, la première requête ne sera plus fonctionnelle mais bien la seconde (à condition que les nouveaux attributs puissent être *NULL*)

Il est possible d'insérer plusieurs tuples avec une seule requête :

```
INSERT INTO ma_table (id, nom, prenom, date_naissance)  
VALUES ( NULL , 'Michel' , 'Dupont' , '1986-04-09' ),  
        ( NULL , 'Mike' , 'Ducanon' , '1989-07-10' ),  
        ( NULL , 'Marthe' , 'Dubonbon' , '1990-11-11' );
```



Une seule requête qui insère x tuples est plus rapide que x requêtes d'insertion mono-tuple

24

24

Insertion et clés dupliquées

num_chassis	voiture	couleur
abcdabcde13	Polo	grise
yzzzABC4442	Golf	rouge
BKxyOMzz13	Clio	bleue
xxxybbzzz14	Clio	jaune
zaeaFIW123	Punto	rouge

La requête suivante va provoquer une erreur (PK dupliquée) :
INSERT INTO ma_table **VALUES** ('xxxybbzzz14','Clio','noire');

Il est possible d'ignorer l'erreur et rien ne sera ajouté dans la table
INSERT IGNORE INTO ma_table **VALUES** ('xxxybbzzz14','Clio','noire');

Il est possible de transformer l'insertion en une modification en cas de PK dupliquée :
INSERT INTO ma_table **VALUES** ('xxxybbzzz14','Clio','noire')
ON DUPLICATE KEY UPDATE voiture='Clio', couleur='noire';

25

25

Suppression des données

Requête qui efface tous les enregistrements dans *ma_table* pour lesquels l'attribut *id* est inférieur à 100 :

DELETE FROM ma_table **WHERE** id < 100;

Attention à ne pas oublier le **WHERE**, sinon la table sera vidée : **DELETE FROM** ma_table;

26

26

DELETE vs TRUNCATE

Pour vider une table : **DELETE FROM ma_table** ou **TRUNCATE TABLE ma_table** ???

DELETE = DML

TRUNCATE = DDL

DELETE et **TRUNCATE** sont très différents. Contrairement à **DELETE**, **TRUNCATE** :

- ne déclenche pas les triggers (déclencheurs)
- ne peut pas être exécuté sur les tables munies de contraintes d'intégrité référentielle (gestion des clés étrangères liées à la table)
- réinitialise les compteurs d'auto-incrémentation

27

27

Mise à jour des données

```
UPDATE ma_table  
  SET nom='Roger',  
      prenom='Tartuffo'  
 WHERE id=17;
```



Attention à ne pas oublier la clause **WHERE** sinon tous les enregistrements de la table seront modifiés (sauf si c'était l'effet souhaité).

Il est possible d'accéder à l'ancienne valeur, exemple :

```
UPDATE articles SET prix = prix*0.8 WHERE id_article=42;  
A l'issue de cette requête, l'article 42 verra son prix diminué de 20%.
```

28

28

Rechercher des données

SELECT * FROM ma_table;

Cette requête récupère tout le contenu de ma_table

La clause SELECT est bien plus complexe qu'il n'y paraît.



Bien qu'il soit aisé de réaliser une application ne contenant que des requêtes SELECT "basiques", la "maîtrise" de la clause SELECT permet :

- d'économiser de nombreuses lignes de code
- une augmentation significative des performances

29

29

La clause SELECT

SELECT

liste de colonnes ou * pour toutes les colonnes

FROM table

WHERE prédicats

GROUP BY ...

HAVING ...

ORDER BY liste de colonnes;

WHERE = filtre de recherche

GROUP BY = regroupement des résultats

HAVING = filtre portant sur les regroupements des résultats

ORDER BY = tri

Ceci est une requête SELECT sur une seule table, ce qui est un cas de figure encore "simple"

30

30

L'opérateur DISTINCT

Table plats

id	type_plat	nom	prix
1	Sandwich	Poulet-curry	2.50
2	Burger	Royal-Cheese	5.10
3	Sandwich	Martino	3.00
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50

```
SELECT DISTINCT type_plat FROM plats;
```

type_plat
Sandwich
Burger
Pizza

ATTENTION
l'utilisation de l'opérateur
DISTINCT peut avoir un
impact non négligeable sur
les performances !



31

31

La concaténation avec CONCAT

Table plats

id	type_plat	nom	prix
1	Sandwich	Poulet-curry	2.50
2	Burger	Royal-Cheese	5.10
3	Sandwich	Martino	3.00
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50

```
SELECT id, CONCAT(type_plat, ' : ', nom)  
FROM plats;
```

id	CONCAT(type_plat, ' : ', nom)
1	Sandwich : Poulet-curry
2	Burger : Royal-Cheese
3	Sandwich : Martino
4	Burger : King-Bacon
5	Pizza : 4 fromages

```
SELECT  
    id, CONCAT(type_plat, ' : ', nom) AS 'plat'  
FROM plats;
```

id	plat
1	Sandwich : Poulet-curry
2	Burger : Royal-Cheese
3	Sandwich : Martino
4	Burger : King-Bacon
5	Pizza : 4 fromages

32

32

La concaténation avec CONCAT_WS

Table *plats*

id	type_plat	nom	prix
1	Sandwich	Poulet-curry	2.50
2	Burger	Royal-Cheese	5.10
3	Sandwich	Martino	3.00
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50

SELECT

id, **CONCAT_WS**(' : ', type_plat, nom) **AS** plat
FROM plats;

WS = With Separator

CONCAT_WS(separator,value1, value2, ... , valueN)

id	plat
1	Sandwich : Poulet-curry
2	Burger : Royal-Cheese
3	Sandwich : Martino
4	Burger : King-Bacon
5	Pizza : 4 fromages

33

33

Le tri avec ORDER BY

Table *plats*

id	type_plat	nom	prix
1	Sandwich	Poulet-curry	2.50
2	Burger	Royal-Cheese	5.10
3	Sandwich	Martino	3.00
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50

SELECT * FROM plats **ORDER BY** type_plat, nom

id	type_plat	nom	prix
4	Burger	King-Bacon	5.40
2	Burger	Royal-Cheese	5.10
5	Pizza	4 fromages	8.50
3	Sandwich	Martino	3.00
1	Sandwich	Poulet-curry	2.50

ATTENTION
l'utilisation de l'opérateur
ORDER BY peut avoir un
impact non négligeable sur
les performances !

Il est conseillé de trier les
données uniquement quand
c'est utile et de façon
réfléchie
(pas sur toutes les colonnes
si ce n'est pas nécessaire)



34

34

L'ordre du tri ORDER BY

Table *plats*

id	type_plat	nom	prix
1	Sandwich	Poulet-curry	2.50
2	Burger	Royal-Cheese	5.10
3	Sandwich	Martino	3.00
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50

SELECT * FROM plats ORDER BY ASC type_plat, DESC nom

id	type_plat	nom	prix
2	Burger	Royal-Cheese	5.10
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50
1	Sandwich	Poulet-curry	2.50
3	Sandwich	Martino	3.00

ASC peut être omis, c'est le sens par défaut

35

35

L'ordre forcé du tri ORDER BY

Table *plats*

id	type_plat	nom	prix
1	Sandwich	Poulet-curry	2.50
2	Burger	Royal-Cheese	5.10
3	Sandwich	Martino	3.00
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50

SELECT * FROM plats ORDER BY FIELD(type_plat, 'Sandwich', 'Burger', 'Pizza'), nom

id	type_plat	nom	prix
3	Sandwich	Martino	3.00
1	Sandwich	Poulet-curry	2.50
4	Burger	King-Bacon	5.40
2	Burger	Royal-Cheese	5.10
5	Pizza	4 fromages	8.50

La clause ORDER BY doit toujours être la dernière partie d'une requête SQL



36

36

La clause WHERE

SELECT * FROM ma_table **WHERE** *prédicats*

Les prédicats sont des expressions logiques renvoyant des valeurs booléennes

Les expressions logiques peuvent être définies en utilisant :

- Des opérateurs de comparaison
- L'opérateur LIKE
- L'opérateur BETWEEN
- L'opérateur IN
- NOT, AND et OR

Les opérateurs mathématiques (+, -, *, /, %) peuvent également vous servir, tout comme des appels de fonction.

37

37

Les opérateurs de comparaison

= < > <= >= <>

Table plats

id	type_plat	nom	prix
1	Sandwich	Poulet-curry	2.50
2	Burger	Royal-Cheese	5.10
3	Sandwich	Martino	3.00
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50

SELECT * FROM plats **WHERE** id<4 **AND** prix>=3;

id	type_plat	nom	prix
2	Burger	Royal-Cheese	5.10
3	Sandwich	Martino	3.00

Certain SGBD utilisent
!=
pour différent de

Et n'oubliez pas que
c'est un seul = pour
tester l'égalité

38

38

Les opérateurs de comparaison (exceptions)

ATTENTION aux attributs de type BOOL, en fonction du SGBD, ils ne peuvent pas toujours être utilisés comme prédicat seuls sans en tester leur valeur

id	type_plat	nom	prix	stock
1	Sandwich	Poulet-curry	2.50	0
2	Burger	Royal-Cheese	5.10	1
3	Sandwich	Martino	NULL	0
4	Burger	King-Bacon	5.40	0
5	Pizza	4 fromages	8.50	1

SELECT id FROM plats WHERE stock;
peut ne pas fonctionner

id
2
5

Il faut utiliser :

SELECT id FROM plats WHERE stock=1;



ATTENTION aux valeurs **NULL**

NULL = NULL retourne false

Si vous voulez récupérer le plat qui n'a pas de prix :

SELECT * FROM plats WHERE prix IS NULL

id	type_plat	nom	prix	stock
3	Sandwich	Martino	NULL	0

39

39

Fonction utile pour la gestion des **NULL**

La fonction **COALESCE** renvoi la première valeur non **NULL** :

id	type_plat	nom	prix
1	Sandwich	Poulet-curry	2.50
2	Burger	Royal-Cheese	5.10
3	Sandwich	Martino	NULL
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50

id	nom	prix
1	Poulet-curry	2.50
2	Royal-Cheese	5.10
3	Martino	0
4	King-Bacon	5.40
5	4 fromages	8.50

SELECT id, nom, COALESCE(prix,0) AS 'prix' FROM plats

40

40

L'opérateur LIKE

Est utilisé pour les comparaisons partielles.

% remplace un nombre inconnu de caractères

_ remplace un seul caractère inconnu

id	type_plat	nom	prix
1	Sandwich	Poulet-curry	2.50
2	Burger	King-Cheese	5.10
3	Sandwich	Martino	3.00
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50

SELECT * FROM plats WHERE nom LIKE 'King%' OR nom LIKE '_ fromages';

id	type_plat	nom	prix
2	Burger	King-Cheese	5.10
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50

41

41

L'opérateur BETWEEN

Table plats

id	type_plat	nom	prix
1	Sandwich	Poulet-curry	2.50
2	Burger	Royal-Cheese	5.10
3	Sandwich	Martino	3.00
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50

SELECT * FROM plats WHERE (prix BETWEEN 3 AND 6)

id	type_plat	nom	prix
2	Burger	Royal-Cheese	5.10
3	Sandwich	Martino	3.00
4	Burger	King-Bacon	5.40

Les parenthèses
sont facultatives
mais conseillées

BETWEEN est assez
pratique pour les
recherches par date

Requête équivalente : SELECT * FROM plats WHERE prix >= 3 AND prix <= 6

42

42

L'opérateur IN

Table *plats*

id	type_plat	nom	prix
1	Sandwich	Poulet-curry	2.50
2	Burger	Royal-Cheese	5.10
3	Sandwich	Martino	3.00
4	Burger	King-Bacon	5.40
5	Pizza	4 fromages	8.50

```
SELECT * FROM plats
WHERE id NOT IN (1,4,2)
```

id	type_plat	nom	prix
3	Sandwich	Martino	3.00
5	Pizza	4 fromages	8.50

```
SELECT * FROM plats WHERE id IN (1,4,2)
```

id	type_plat	nom	prix
1	Sandwich	Poulet-curry	2.50
2	Burger	Royal-Cheese	5.10
4	Burger	King-Bacon	5.40

Requête équivalente : SELECT * FROM plats
WHERE id <> 1 AND id <> 4 AND id <> 2;

Requête équivalente : SELECT * FROM plats WHERE id=1 OR id=4 OR id=2;

43

43

L'opérateur IN et requête imbriquée

Le contenu des parenthèse du IN peut être remplacé par une requête SELECT ne retournant qu'une seule colonne.

Table *types_plat*

id_type	nom
1	Sandwich
2	Burger
3	Pizza

Table *plats*

id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
3	1	Martino	3.00
4	2	King-Bacon	5.40
5	3	4 fromages	8.50

```
SELECT DISTINCT nom FROM types_plat
WHERE id_type IN (
    SELECT id_type FROM plats
    WHERE (prix BETWEEN 3 AND 6)
);
```

nom
Burger
Sandwich

44

44

Les structures de contrôle

```
SELECT
    nom,
    IF(prix<3.5, 'pas cher', 'trop cher') AS 'tarif',
    CASE
        WHEN stock=0 THEN 'rupture'
        WHEN stock<10 THEN 'faible'
        ELSE 'ok'
    END AS stock
FROM plats;
```

id	type_plat	nom	prix	stock
1	Sandwich	Poulet-curry	2.50	35
2	Burger	Royal-Cheese	5.10	4
3	Sandwich	Martino	3.00	87
4	Burger	King-Bacon	5.40	0
5	Pizza	4 fromages	8.50	3

nom	tarif	stock
Poulet-curry	pas cher	ok
Royal-Cheese	trop cher	faible
Martino	pas cher	ok
King-Bacon	trop cher	rupture
4 fromages	trop cher	faible

45

45

Traitement des données temporelles

```
SELECT YEAR('2019-02-01 10:32:54');    -> 2019
SELECT MONTH('2019-02-01 10:32:54');   -> 2
SELECT DAY('2019-02-01 10:32:54');     -> 1
SELECT HOUR('2019-02-01 10:32:54');    -> 10
SELECT MINUTE('2019-02-01 10:32:54');  -> 32
SELECT SECOND('2019-02-01 10:32:54');  -> 54
SELECT MICROSECOND('2019-02-01 10:32:54.456456'); -> 456456
```

Connaître la date et/ou l'heure actuelle : `CURRENT_TIMESTAMP, NOW(),`
`CURRENT_DATE, CURRENT_TIME`

```
SELECT DATE_FORMAT('2018-11-03', '%a %d/%m/%Y'); -> Sat 03/11/2018
```

Il existe d'autres fonctions dédiées aux traitements des données temporelles :
`DATE_ADD, DATEDIFF, DAYOFWEEK, DAYOFYEAR, TIMEDIFF, UTC_TIME, WEEKOFYEAR,`
`SEC_TO_TIME, MONTHNAME, etc.`

46

46

Autres fonctions

Les classiques :

SELECT TRIM (' boulette ');	-> boulette
SELECT POSITION ('ca', 'Abracadabra');	-> 5 (0 s'il ne trouve pas)
SELECT SUBSTRING ('Abracadabra',5,2);	-> ca
SELECT UPPER ('Sarah');	-> SARAH
SELECT LOWER ('Sarah');	-> sarah
SELECT REPLACE ('Abracadabra', 'ra', 'R4');	-> AbR4cadabR4
SELECT ROUND (1.58);	-> 2
SELECT FLOOR (1.58);	-> 1
SELECT TRUNCATE (1.58,1);	-> 1.5

Il est possible d'effectuer des conversions de type avec CAST(col AS INTEGER)

Il existe des fonctions

- dédiées aux REGEX
- mathématiques comme COS, ACOS, SIN, ASIN, POW, LOG, PI, SQRT, etc.
- de chiffrement et de compression AES_ENCRYPT, SHA1, MD5, COMPRESS, etc.
- dédiées au JSON et à l'XML
- dédiées aux opérations binaires, aux données spatiales, etc.

47

47

Les fonctions d'agrégation

```
SELECT
  COUNT(id),
  MIN(prix),
  MAX(prix),
  AVG(prix),
  SUM(stock),
  SUM(stock*prix) AS valeur
FROM plats;
```

id	type_plat	nom	prix	stock
1	Sandwich	Poulet-curry	2.50	35
2	Burger	Royal-Cheese	5.10	4
3	Sandwich	Martino	3.00	87
4	Burger	King-Bacon	5.40	0
5	Pizza	4 fromages	8.50	3

COUNT(id)	MIN(prix)	MAX(prix)	AVG(prix)	SUM(stock)	valeur
5	2.50	8.50	4.900000	129	394.40

48

48

Les fonctions d'agrégation avec GROUP BY

```
SELECT
  type_plat,
  COUNT(id),
  MIN(prix),
  MAX(prix),
  AVG(prix),
  SUM(stock),
  SUM(stock*prix) AS valeur
FROM plats
GROUP BY type_plat;
```

id	type_plat	nom	prix	stock
1	Sandwich	Poulet-curry	2.50	35
2	Burger	Royal-Cheese	5.10	4
3	Sandwich	Martino	3.00	87
4	Burger	King-Bacon	5.40	0
5	Pizza	4 fromages	8.50	3

type_plat	COUNT(id)	MIN(prix)	MAX(prix)	AVG(prix)	SUM(stock)	valeur
Burger	2	5.10	5.40	5.250000	4	20.40
Pizza	1	8.50	8.50	8.500000	3	25.50
Sandwich	2	2.50	3.00	2.750000	122	348.50

Comme le ORDER BY, le GROUP BY peut se faire sur plusieurs colonnes

49

49

Filtrage du GROUP BY avec HAVING

```
SELECT
  type_plat,
  MIN(prix),
  MAX(prix),
  AVG(prix),
  SUM(stock),
  SUM(stock*prix) AS valeur
FROM plats
GROUP BY type_plat;
HAVING SUM(stock) < 10
```

id	type_plat	nom	prix	stock
1	Sandwich	Poulet-curry	2.50	35
2	Burger	Royal-Cheese	5.10	4
3	Sandwich	Martino	3.00	87
4	Burger	King-Bacon	5.40	0
5	Pizza	4 fromages	8.50	3

type_plat	MIN(prix)	MAX(prix)	AVG(prix)	SUM(stock)	valeur
Burger	5.10	5.40	5.250000	4	20.40
Pizza	8.50	8.50	8.500000	3	25.50

HAVING agit comme le WHERE mais sur les opérations résultant des regroupements.

50

50

Jointure par produit cartésien

Une jointure exploitant les données de plusieurs tables

C'est la clause **JOIN** qui est dédiée à cette tâche mais il est possible de s'en passer en effectuant un **produit cartésien** :

```
SELECT types_plat.nom_type, plats.nom, plats.prix
FROM (plats, types_plat)
WHERE types_plat.id_type=plats.id_type AND plat.id=5
```

id_type	nom_type
1	Sandwich
2	Burger
3	Pizza

Table *types_plat*

id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
3	1	Martino	3.00
4	2	King-Bacon	5.40
5	3	4 fromages	8.50

Table *plats*

nom_type	nom	prix
Pizza	4 fromages	8.50

Les parenthèses sont
conseillées avec
MySQL/MariaDB et
proscrites avec
d'autres SGBDR



51

51

Le produit cartésien n'est pas "la" solution

```
SELECT types_plat.nom_type, plats.nom, plats.prix
FROM (plats, types_plat)
WHERE types_plat.id_type=plats.id_type AND plat.id=5;
```

Il est grandement conseillé de réécrire la requête ci-dessus en utilisant la clause **JOIN**

```
SELECT types_plat.nom_type, plats.nom, plats.prix
FROM (plats)
JOIN (types_plat) ON (types_plat.id_type=plats.id_type )
WHERE plat.id=5;
```

Par rapport au produit cartésien, la clause **JOIN** :

- offre une meilleure lisibilité du code
- permet différents types de jointure
- obtient souvent de meilleures performances

Notez aussi que la virgule du produit cartésien peut être remplacée par la clause **CROSS JOIN**



52

52

Jointure et technique du "surnommage"

L'écriture des jointure peut être un processus assez long.
Afin de gagner un peu de temps je vous conseille d'utiliser
la technique de "surnommage"

```
SELECT types_plat.nom_type, plats.nom, plats.prix FROM (plats)
JOIN (types_plat) ON (types_plat.id_type=plats.id_type )
WHERE plat.id=5;
```



"surnommage" des tables

```
SELECT t.nom_type, p.nom, p.prix FROM (plats AS p)
JOIN (types_plat AS t) ON (t.id_type=p.id_type )
WHERE p.id=5;
```



Pour le "surnommage" des tables,
le mot **AS** est facultatif.

53

53

Jointure et filtrage

```
SELECT
    t.nom_type,
    p.nom,
    p.prix
FROM (plats AS p)
JOIN (types_plat AS t)
ON (t.id_type=p.id_type )
WHERE p.id>2 AND t.nom_type<>'Pizza';
```

Dans le **ON**, ce sont les conditions de jointure

Dans le **WHERE**, ce sont les conditions de filtrage

54

54

La jointure interne : [INNER] JOIN

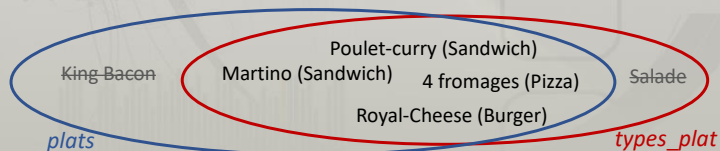
```
SELECT t.nom_type, p.nom, p.prix FROM (plats AS p)
INNER JOIN (types_plat AS t) ON (t.id_type=p.id_type );
```

Le mot **INNER** est facultatif, la jointure interne est le mode de jointure par défaut

id	id_type	nom	prix	id_type	nom_type	nom_type	nom	prix
1	1	Poulet-curry	2.50	1	Sandwich	Sandwich	Poulet-curry	2.50
2	2	Royal-Cheese	5.10	2	Burger	Burger	Royal-Cheese	5.10
3	1	Martino	3.00	3	Pizza	Sandwich	Martino	3.00
4	NULL	King-Bacon	5.40	4	Salade	Pizza	4 fromages	8.50
5	3	4 fromages	8.50					

Table plats

Table types_plat



55

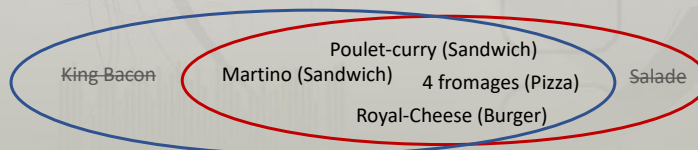
55

La jointure naturelle : NATURAL JOIN

```
SELECT t.nom_type, p.nom, p.prix FROM (plats AS p)
NATURAL JOIN (types_plat AS t)
```

Le SGBDR va effectuer la jointure en se basant sur les colonnes portant les mêmes noms et types

Table plats				Table types_plat				
id	id_type	nom	prix	id_type	nom_type	nom_type	nom	prix
1	1	Poulet-curry	2.50	1	Sandwich	Sandwich	Poulet-curry	2.50
2	2	Royal-Cheese	5.10	2	Burger	Burger	Royal-Cheese	5.10
3	1	Martino	3.00	3	Pizza	Sandwich	Martino	3.00
4	NULL	King-Bacon	5.40	4	Salade	Pizza	4 fromages	8.50
5	3	4 fromages	8.50					



56

56

Attention à la jointure naturelle

```
SELECT t.nom AS nom_type, p.nom, p.prix FROM (plats AS p)
NATURAL JOIN (types_plat AS t)
```

Table plats

id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
3	1	Martino	3.00
4	NULL	King-Bacon	5.40
5	3	4 fromages	8.50

Table types_plat

id_type	nom
1	Sandwich
2	Burger
3	Pizza
4	Salade

Résultat vide

```
SELECT t.nom AS nom_type, p.nom, p.prix FROM (plats AS p)
NATURAL JOIN (types_plat AS t) USING (id_type)
```



Perso, je vous déconseille d'utiliser les jointures naturelles.

Il est toujours préférable de décrire explicitement les conditions de jointure

nom_type	nom	prix
Sandwich	Poulet-curry	2.50
Burger	Royal-Cheese	5.10
Sandwich	Martino	3.00
Pizza	4 fromages	8.50

57

57

La jointure externe par la gauche

```
SELECT t.nom_type, p.nom, p.prix FROM (plats AS p)
LEFT OUTER JOIN (types_plat AS t) ON (t.id_type=p.id_type);
```

Le mot **OUTER** est facultatif

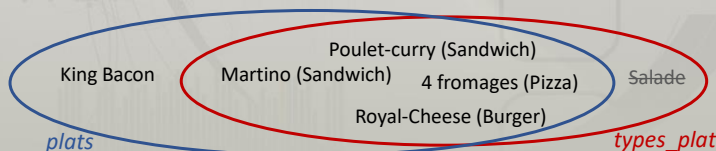
id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
3	1	Martino	3.00
4	NULL	King-Bacon	5.40
5	3	4 fromages	8.50

Table plats

id_type	nom_type
1	Sandwich
2	Burger
3	Pizza
4	Salade

Table types_plat

nom_type	nom	prix
Sandwich	Poulet-curry	2.50
Burger	Royal-Cheese	5.10
Sandwich	Martino	3.00
Pizza	4 fromages	8.50
NULL	King-Bacon	5.40



58

58

La jointure externe par la droite

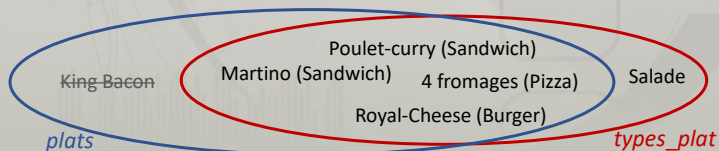
```
SELECT t.nom_type, p.nom, p.prix FROM (plats AS p)
RIGHT OUTER JOIN (types_plat AS t) ON (t.id_type=p.id_type );
```

Le mot **OUTER** est facultatif

id	id_type	nom	prix	id_type	nom_type	nom_type	nom	prix
1	1	Poulet-curry	2.50	1	Sandwich	Sandwich	Poulet-curry	2.50
2	2	Royal-Cheese	5.10	2	Burger	Burger	Royal-Cheese	5.10
3	1	Martino	3.00	3	Pizza	Sandwich	Martino	3.00
4	NULL	King-Bacon	5.40	4	Salade	Pizza	4 fromages	8.50
5	3	4 fromages	8.50			Salade	NULL	NULL

Table plats

Table types_plat



59

59

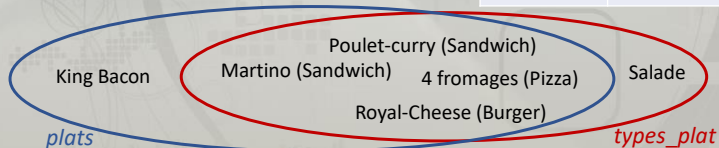
La jointure externe bilatérale

```
SELECT t.nom_type, p.nom, p.prix FROM (plats AS p)
FULL OUTER JOIN (types_plat AS t) ON (t.id_type=p.id_type );
```

id	id_type	nom	prix	id_type	nom_type	nom_type	nom	prix
1	1	Poulet-curry	2.50	1	Sandwich	Sandwich	Poulet-curry	2.50
2	2	Royal-Cheese	5.10	2	Burger	Burger	Royal-Cheese	5.10
3	1	Martino	3.00	3	Pizza	Sandwich	Martino	3.00
4	NULL	King-Bacon	5.40	4	Salade	Pizza	4 fromages	8.50
5	3	4 fromages	8.50			NULL	King-Bacon	5.40
						Salade	NULL	NULL

Table plats

Table types_plat



Le FULL JOIN n'est pas disponible avec MySQL/ MariaDB.

Il est toutefois possible d'obtenir le même effet avec des UNION

60

60

Les opérateurs ensemblistes : UNION

id	nom
1	Sandwich
2	Burger
3	Pizza
4	Salade
5	Lasagne

Table plats

id	menu	prix
1	Sandwich	4.00
2	Burger	7.30
3	Pizza	11.50
4	Mitraillette	5.50
5	Salade	6.00

Table menus

nom
Sandwich
Burger
Pizza
Salade
Lasagne
Mitraillette

SELECT nom **FROM** plats **UNION** **SELECT** menu **FROM** menus



Salade n'apparaît qu'une seule fois

61

61

Les opérateurs ensemblistes : UNION ALL

id	nom
1	Sandwich
2	Burger
3	Pizza
4	Salade
5	Lasagne

Table plats

id	menu	prix
1	Sandwich	4.00
2	Burger	7.30
3	Pizza	11.50
4	Mitraillette	5.50
5	Salade	6.00

Table menus

nom
Sandwich
Burger
Pizza
Salade
Lasagne
Sandwich
Burger
Pizza
Mitraillette
Salade

SELECT nom **FROM** plats **UNION ALL** **SELECT** menu **FROM** menus



Avec d'autres SGBDR que MySQL/MariaDB, le second **SELECT** peut être omis

62

62

Les opérateurs ensemblistes : EXCEPT

L'opérateur EXCEPT permet de récupérer la différence

id	nom
1	Sandwich
2	Burger
3	Pizza
4	Salade
5	Lasagne

Table plats

id	menu	prix
1	Sandwich	4.00
2	Burger	7.30
3	Pizza	11.50
4	Mitraillette	5.50
5	Salade	6.00

Table menus

nom
Lasagne

SELECT nom FROM plats EXCEPT menu FROM menus

Avec ORACLE
c'est MINUS



MySQL ne supporte pas cette fonction
et MariaDB uniquement à partir de la version 10.3

En l'absence d'EXCEPT, quelle serait
une requête équivalente ?

SELECT nom FROM plats WHERE nom NOT IN
(SELECT menu FROM menus)

63

63

Les opérateurs ensemblistes : INTERSECT

id	nom
1	Sandwich
2	Burger
3	Pizza
4	Salade
5	Lasagne

Table plats

id	menu	prix
1	Sandwich	4.00
2	Burger	7.30
3	Pizza	11.50
4	Mitraillette	5.50
5	Salade	6.00

Table menus

nom
Sandwich
Burger
Pizza
Salade

SELECT nom FROM plats INTERSECT menu FROM menus

A nouveau, MySQL ne supporte pas cette fonction
et MariaDB uniquement à partir de la version 10.3



En l'absence d'INTERSECT, quelle
serait une requête équivalente ?

SELECT nom FROM plats WHERE nom IN
(SELECT menu FROM menus)

64

64

Requêtes imbriquées

Il existe de nombreuses possibilités d'imbrication de requête.

Il est possible de classer les requêtes imbriquées en fonction du type de retour :

- 1 col – 1 tuple (remplace une seule valeur)
- N col - 1 tuple (remplace un enregistrement)
- 1 col – N tuples (remplace une liste de valeurs)
- N col – N tuples (remplace une table)

Toutes les combinaisons d'imbrication ne sont pas possibles.

Exemple : il n'est pas possible d'imbriquer une requête autre que 1-1 ou 1-N dans une clause IN

65

65

Requêtes imbriquées : exemple 1-1

id_type	nom_type
1	Sandwich
2	Burger
3	Pizza
4	Salade

Table *types_plat*

id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
3	1	Martino	3.00
4	2	King-Bacon	5.40
5	3	4 fromages	8.50

Table *plats*

```
SELECT
    t.nom_type,
    (SELECT MIN(p1.prix) FROM plats AS p1 WHERE p1.id_type=t.id_type) AS 'min',
    (SELECT MAX(p2.prix) FROM plats AS p2 WHERE p2.id_type=t.id_type) AS 'max'
FROM types_plat AS t
ORDER BY t.nom_type
```

type_plat	min	max
Burger	5.10	5.40
Pizza	8.50	8.50
Salade	NULL	NULL
Sandwich	2.50	3.00



Avec ce type d'utilisation, une requête imbriquée ne peut renvoyer qu'une seule valeur.

Les requêtes ne renvoyant qu'une seule valeur peuvent être imbriquées presque partout

66

66

Requêtes imbriquées : exemple N - 1

id	nom
1	Sandwich
2	Burger
3	Pizza
4	Salade
5	Lasagne

Table plats

id	menu	prix
1	Sandwich	4.00
2	Burger	7.30
3	Pizza	11.50
4	Mitraillette	5.50
5	Salade	6.00

Table menus

id	nom
1	Sandwich
2	Burger
3	Pizza

```
SELECT p.id, p.nom FROM plat AS p
WHERE (p.id, p.nom) MATCH (SELECT m.id, m.menu FROM menus AS m WHERE m.id=p.id)
ORDER BY p.id
```



Actuellement, l'opérateur MATCH n'est pas fonctionnel avec MySQL/MariaDB

67

67

Requêtes imbriquées : exemple 1 - N

Table types_plat

id_type	nom
1	Sandwich
2	Burger
3	Pizza

Table plats

id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
3	1	Martino	3.00
4	2	King-Bacon	5.40
5	3	4 fromages	8.50

```
SELECT DISTINCT nom FROM types_plat
WHERE id_type IN (
    SELECT id_type FROM plats
    WHERE (prix BETWEEN 3 AND 6)
);
```

nom
Burger
Sandwich

68

68

Requêtes imbriquées : exemple N - N

id	nom
1	Sandwich
2	Burger
3	Pizza
4	Salade
5	Lasagne

Table *plats*

id	nom
1	Sandwich
2	Burger
3	Pizza

```
SELECT p.id, p.nom FROM
  (SELECT id, nom FROM plats WHERE id<4) AS p
ORDER BY p.id
```

69

69

L'opérateur GROUP_CONCAT

id_type	nom_type
1	Sandwich
2	Burger
3	Pizza
4	Salade

Table *types_plat*

id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
3	1	Martino	3.00
4	2	King-Bacon	5.40
5	3	4 fromages	8.50

Table *plats*

```
SELECT
  t.nom_type,
  GROUP_CONCAT(p.nom SEPARATOR ',') AS 'choix'
FROM (types_plat AS t)
LEFT JOIN (plats AS p) ON (p.id_type=t.id_type)
GROUP BY t.id_type
ORDER BY t.nom_type
```



GROUP_CONCAT = MySQL/MariaDB
STRING_AGG = SQL Srv et PostgreSQL
LISTAGG = Oracle



nom_type	choix
Burger	Royal-Cheese, King-Bacon
Pizza	4 fromages
Salade	NULL
Sandwich	Poulet-curry, Martino

70

70

L'opérateur GROUP_CONCAT

```
SELECT t.nom_type,
       (SELECT GROUP_CONCAT(p.nom SEPARATOR ',')
        FROM plats AS p WHERE p.id_type=t.id_type) AS 'choix'
FROM (types_plat AS t)
ORDER BY t.nom_type
```

id_type	nom_type
1	Sandwich
2	Burger
3	Pizza
4	Salade

Table *types_plat*

id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
3	1	Martino	3.00
4	2	King-Bacon	5.40
5	3	4 fromages	8.50

Table *plats*

Cet opérateur
n'est pas très
"algèbre relationnelle"



Cependant, l'utilisation
de **GROUP_CONCAT**
peut procurer des gains
en performances en
réduisant le nombre de
requêtes et/ou la
quantité de données
transférée

nom_type	choix
Burger	Royal-Cheese, King-Bacon
Pizza	4 fromages
Salade	NULL
Sandwich	Poulet-curry, Martino



1

L'algèbre relationnelle

Théorie mathématique proche de la théorie des ensembles

inventée en 1970 par Edgar Frank Codd,

Les opérations relationnelles permettent de créer **une nouvelle relation (table)** à partir d'opérations élémentaires sur d'autres relations (tables)

N'est pas vraiment un langage à destination des développeurs
(n'est pas mathématiquement complet)
mais est **au cœur du langage de requête de SQL**

Les SGBDR ne fonctionnent pas à 100% selon les règles ensemblistes :

	SGBDR	SQL	A.R.
plusieurs lignes identiques dans une relation/table ->	✓	✓	✗

2

2

Notions

En algèbre relationnelle, le terme « relation » est constitué de 2 choses :

- **Un schéma** : structure d'une relation
{id: INT, nom: VARCHAR(60), prenom: VARCHAR(60)}
3 attributs -> **arité = 3**
- **Une extension** : le contenu de la relation (n-uplets)
{id: 1, nom: "Dutruc", prenom: "Kevin"}
{id: 2, nom: "Dumachin", prenom: "Marc"}
{id: 3, nom: "Vandenbrol", prenom: "Sarah"}



En A.R., les notations ne sont pas standardisées...

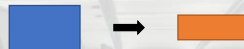
3

3

Familles d'opérateurs relationnels

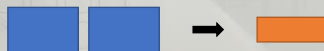
1. Les opérateurs unaires

produisent une nouvelle relation à partir d'une autre relation.



2. Les opérateurs ensemblistes

produisent une nouvelle relation à partir de deux relations de même degré et de même domaine.



3. Les opérateurs n-aires

permettent de produire une nouvelle relation à partir de deux ou plusieurs autres relations.



4

4

4.1 Les opérateurs unaires

5

5

La sélection (ou restriction)

$$\sigma_{(E)} R$$

Relation (R) x expression logique (E) $\rightarrow$ Relation ($\sigma_{(E)} R$)

Relation regroupant exclusivement toutes les occurrences de la relation R qui satisfont l'expression logique E

Le résultat est donc une nouvelle relation qui a les mêmes attributs que R .

Également appelée "restriction"

Equivalent SQL : **WHERE**

6

6

La sélection

VOITURES		
id	voiture	couleur
1	VW Polo	rouge
2	Opel Corsa	rose
3	Mercedes C220	vert
4	Fiat Panda	jaune
5	Porsche 911	transparente

$\sigma_{(id < 3)}$ VOITURES		
id	voiture	couleur
1	VW Polo	rouge
2	Opel Corsa	rose

7

7

La projection

$$\pi_{(A1, A2, \dots, An)} R$$

Relation (R) x liste d'attributs (Ax) $\rightarrow$ Relation ($\pi_{(Ax)} R$)

La projection permet de choisir les attributs d'une relation

En d'autres termes, la projection consiste à supprimer les attributs autres que $A1, A2, \dots, An$ d'une relation

Equivalent SQL : **SELECT**

Ne confondez pas
Sélection (WHERE)
et
Projection (SELECT)



8

8

La projection

VOITURES		
id	voiture	couleur
1	VW Polo	rouge
2	Opel Corsa	rose
3	Mercedes C220	vert
4	Fiat Panda	jaune
5	Porsche 911	transparente

$\pi_{(couleur)} VOITURES$

couleur
rouge
rose
vert
jaune
transparente

9

9

La projection : suppression des doublons

VOITURES		
id	voiture	couleur
1	VW Polo	rouge
2	Opel Corsa	noir
3	Mercedes C220	rouge
4	Fiat Panda	jaune
5	Porsche 911	jaune

$\pi_{(couleur)} VOITURES$

couleur
rouge
noir
jaune

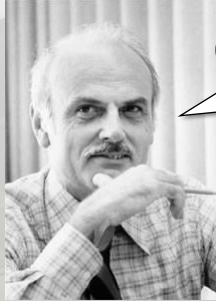
N'oubliez pas qu'en A.R. il ne peut y avoir de doublons dans les relations.
(DISTINCT implicite)



10

10

Les doublons



en.wikipedia.org/wiki/Edgar_F._Codd

Si quelque chose est "vrai",
le dire deux fois ne le rendra
pas plus "vrai"

*Les objets réels n'ont qu'une seule caractéristique commune :
ils sont tous différents*

(anonymous, A. Taivalsaari, JOOP, Nov. 1997)

A l'exception des opérations d'agrégation, il est assez délicat de manipuler des tuples identiques

Utilité de la clé primaire

11

11

Rebaptiser

$\rho_{a/b}(R)$

Consiste à renommer les attributs b en a

Equivalent SQL : **AS**

GENS	
id	qui
1	Le Gros Charles
2	Monsieur Patrick

$\rho_{\text{identifiant} / \text{id}}(\text{GENS})$	
identifiant	qui
1	Le Gros Charles
2	Monsieur Patrick

Cette notation n'est
pas toujours respectée



12

12

Combinaison des opérations

- **Forme imbriquée :**

- $\pi_{\text{couleur}} (\sigma_{\text{id}<3} \text{VOITURES})$

- **Forme séquentielle :**

- $X \leftarrow \sigma_{\text{id}<3} \text{VOITURES}$

- $R \leftarrow \pi_{\text{couleur}} X$

Principe de **compositionnalité**

*Le résultat d'une expression peut être utilisé
comme argument d'une autre expression*

13

13

Algèbre relationnelle

4.2 Les opérateurs ensemblistes

14

14

L'union

$$R_1 \cup R_2$$

Relation (R_1) x Relation (R_2) $\rightarrow$ Relation ($R_1 \cup R_2$)

Construit une relation constituée des n-uplets appartenant à l'une ou l'autre des deux relations

Opération portant sur **deux relations ayant le même schéma**

Equivalent SQL : **UNION**

ATTENTION que
contrairement au SQL, avec
l'A.R. une relation ne
contient pas de doublon



15

15

L'union

VOITURES		
id	nom	couleur
1	VW Polo	rouge
2	Opel Corsa	rose
3	Mercedes C220	vert
4	Fiat Panda	jaune
5	Porsche 911	transparente

MOTOS		
id	nom	couleur
1	Suzuki Bandit 1250	rouge
2	Honda CBF 650	rose
3	BMW R1200GS	vert
4	Ducati Monster	jaune
5	Yamaha MT07	transparente

MOTOS $\cup$ VOITURES		
id	nom	couleur
1	Suzuki Bandit 1250	rouge
2	Honda CBF 650	rose
3	BMW R1200GS	vert
4	Ducati Monster	jaune
5	Yamaha MT07	transparente
1	VW Polo	rouge
2	Opel Corsa	rose
3	Mercedes C220	vert
4	Fiat Panda	jaune
5	Porsche 911	transparente

16

16

L'intersection

$$R_1 \cap R_2$$

Relation (R_1) x Relation (R_2) $\rightarrow$ Relation ($R_1 \cap R_2$)

Construit une relation constituée des n-uplets communs aux deux relations

Opération portant sur deux relations ayant le même schéma

Equivalent SQL : **INTERSECT**

Pas d'INTERSECT
en MySQL...

```
SELECT DISTINCT value FROM table1  
WHERE value IN  
( SELECT value FROM `table2` );
```



17

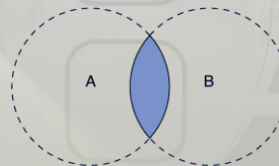
17

L'intersection

VEHICULES1		
id	nom	couleur
1	VW Polo	rouge
2	Opel Corsa	rose
3	Mercedes C220	vert
4	Fiat Panda	jaune
6	Porsche 911	transparente

VEHICULES2		
id	nom	couleur
1	Suzuki Bandit 1250	rouge
2	Honda CBF 650	rose
3	BMW R1200GS	vert
4	Fiat Panda	jaune
5	Porsche 911	transparente

VEHICULES1 $\cap$ VEHICULES2		
id	nom	couleur
4	Fiat Panda	jaune



18

18

La différence

$$R_1 - R_2$$

Relation (R_1) x Relation (R_2) -> Relation ($R_1 - R_2$)

Construit une relation constituée des n-uplets de R_1 ne se trouvant pas dans R_2

Opération portant sur deux relations ayant le même schéma

Equivalent SQL : **EXCEPT** (MINUS avec ORACLE)

Pas d'EXCEPT en MySQL...

```
SELECT DISTINCT value FROM table1
WHERE value NOT IN
( SELECT value FROM `table2` );
```

19

19

La différence

VEHICULES1		
id	nom	couleur
1	VW Polo	rouge
2	Opel Corsa	rose
3	Mercedes C220	vert
4	Fiat Panda	jaune
6	Porsche 911	transparente

VEHICULES2		
id	nom	couleur
1	Suzuki Bandit 1250	rouge
2	Honda CBF 650	rose
3	BMW R1200GS	vert
4	Fiat Panda	jaune
5	Porsche 911	transparente

VEHICULES1 - VEHICULES2		
id	nom	couleur
1	VW Polo	rouge
2	Opel Corsa	rose
3	Mercedes C220	vert
6	Porsche 911	transparente

20

20

4.3 Les opérateurs n-aires

21

21

Le produit cartésien

$$R_1 \times R_2$$

Relation (R_1) $\times$ Relation (R_2) $\rightarrow$ Relation ($R_1 \times R_2$)

Construit une relation constituée de tous les attributs de R_1 et de R_2

Equivalent SQL : **SELECT * FROM table1, table2**

En fonction des auteurs, cet opérateur est des fois classé dans la famille des opérateurs ensemblistes



22

22

Le produit cartésien

VEHICULES		
id	nom	couleur
1	VW Polo	rouge
2	Opel Corsa	rose
3	Mercedes C220	vert

GENS	
id	qui
1	Le Gros Charles
2	Monsieur Patrick

VEHICULES x GENS				
id	nom	couleur	id_1	qui
1	VW Polo	rouge	1	Le Gros Charles
1	VW Polo	rouge	2	Monsieur Patrick
2	Opel Corsa	rose	1	Le Gros Charles
2	Opel Corsa	rose	2	Monsieur Patrick
3	Mercedes C220	vert	1	Le Gros Charles
3	Mercedes C220	vert	2	Monsieur Patrick

23

23

La jointure

$$R_1 \bowtie_E R_2$$

Relation (R_1) x Relation (R_2) x Expression (E) -> Relation ($R_1 \bowtie_E R_2$)

Relation regroupant toutes les possibilités de combinaison des occurrences des relations R_1 et R_2 qui satisfont l'expression logique E

Equivalent SQL : **JOIN**

`SELECT R1.*, R2.* FROM (R1) JOIN (R2) ON (E)`

Similaire à un produit cartésien suivi d'une sélection :

`SELECT R1.*, R2.* FROM (R1,R2) WHERE E`

Si les relations R1 et R2 ont des attributs en commun, il est nécessaire de les renommer



24

24

La jointure

VEHICULES		
id	nom	proprietaire
1	VW Polo	1
2	Opel Corsa	NULL
3	Mercedes C220	2

GENS	
id_mec	qui
1	Le Gros Charles
2	Monsieur Patrick

VEHICULES ⋈ _(id_mec = proprietaire) GENS					
id	nom	proprietaire	id_mec	qui	
1	VW Polo	1	1	Le Gros Charles	
3	Mercedes C220	2	2	Monsieur Patrick	

Test d'égalité = **équi-jointure**

25

25

La jointure

VEHICULES		
id	nom	Prix
1	VW Polo	4999
2	Opel Corsa	4500
3	Mercedes C220	25000

VALEURS		
lemin	quoi	lemax
0	Pas cher	5000
5000	Normal	20000
20000	Cher	9999999999

VEHICULES ⋈ _(prix >= lemin AND prix < lemax) VALEURS					
id	nom	Prix	lemin	quoi	lemax
1	VW Polo	4999	0	Pas cher	5000
2	Opel Corsa	4500	0	Pas cher	5000
3	Mercedes C220	25000	20000	Cher	9999999999

Test de comparaison = **thêta-jointure**

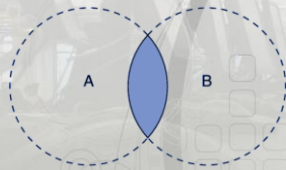
26

26

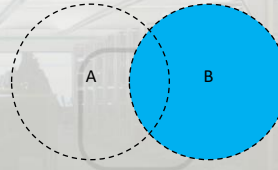
Les différentes jointures

Comme en SQL, il existe différentes jointures :

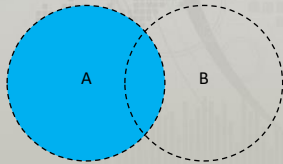
INNER JOIN : $R_1 \bowtie_E R_2$



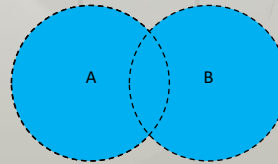
RIGHT JOIN : $R_1 \bowtie_{\rightarrow E} R_2$



LEFT JOIN : $R_1 \bowtie_{\leftarrow E} R_2$



FULL JOIN : $R_1 \bowtie_E R_2$



27

27

La jointure naturelle

$R1 \bowtie R2$

Relation regroupant toutes les possibilités de combinaison des occurrences des relations $R1$ et $R2$ qui satisfont une **expression logique implicite** :

test d'égalité entre les attributs qui portent le même nom dans les relations R_1 et R_2

Les attributs communs sont fusionnés au sein de la relation résultante

Il est possible de limiter le test d'égalité à un sous-ensemble des attributs communs :

$R1 \bowtie_{a1,a2,\dots,an} R2$

28

28

La jointure naturelle

VEHICULES		
id	nom	id_cat
1	VW Polo	1
2	Opel Corsa	1
3	Mercedes C220	2

CATEGORIES	
id_cat	quoi
1	Cacahuète
2	Berline

VEHICULES $\bowtie$ CATEGORIES			
id	nom	id_cat	quoi
1	VW Polo	1	Cacahuète
2	Opel Corsa	1	Cacahuète
3	Mercedes C220	2	Berline

29

29

L'ordre des sélections

La sélection peut se faire avant la jointure :

$$V \leftarrow \sigma_{(marque='Opel')} VOITURES$$

$$R \leftarrow V \bowtie_{(idfk_couleur = idpk_couleur)} COULEURS$$

La sélection peut se faire après la jointure :

$$V \leftarrow VOITURES \bowtie_{(idfk_couleur = idpk_couleur)} COULEURS$$

$$R \leftarrow \sigma_{(marque='Opel')} V$$

Dans la pratique, il est probable que la première approche soit la plus performante



30

30

La semi-jointure ⋈

VEHICULES		
id	nom	id_proprio
1	VW Polo	1
2	Opel Corsa	NULL
3	Mercedes C220	2

GENS	
id_proprio	qui
1	Le Gros Charles
2	Monsieur Patrick

VEHICULES ⋈ GENS		
id	nom	id_proprio
1	VW Polo	1
3	Mercedes C220	2

La semi-jointure est une jointure naturelle où la relation résultante ne contient aucun attribut provenant de la relation "jointe"

31

31

L'anti-jointure ▷

VEHICULES		
id	nom	id_proprio
1	VW Polo	1
2	Opel Corsa	NULL
3	Mercedes C220	2

GENS	
id_proprio	qui
1	Le Gros Charles
2	Monsieur Patrick

VEHICULES ▷ GENS		
id	nom	id_proprio
2	Opel Corsa	NULL

L'anti-jointure peut être vue comme l'inverse d'une jointure naturelle

32

32

La division

$$R_1 \div R_2 \quad \text{ou} \quad R_1 / R_2$$

Relation (R_1) x Relation (R_2) -> Relation (R_1 / R_2)

Opération portant sur deux relations R_1 et R_2 , telles que le schéma de R_2 est strictement inclus dans celui de R_1 , qui génère une troisième relation regroupant toutes les parties d'occurrences de la relation R_1 qui, associées à toutes les occurrences de la relation R_2 , se retrouvent dans R_1 .

...

33

33

La division

VEHICULES	
nom	quoi
VW Polo	Cacahuète
Opel Corsa	Cacahuète
Opel Corsa	Poubelle
Mercedes C220	Berline
Mercedes C220	Cacahuète

CATEGORIES
quoi
Cacahuète
Berline

VEHICULES ÷ CATEGORIES	
nom	
Mercedes C220	

Il n'y a pas d'opérateur pour la division relationnelle en SQL.

34

34

Bases de données et division

Si la division relationnelle n'existe pas en SQL, c'est que ça sert à rien !

Ne soit pas si catégorique !

Certaines recherches peuvent nécessiter une division relationnelle

35

35

Le SQL vs la division relationnelle

Snacks		Articles
snack	article	article
Pro Burger	bière	bière
Pro Burger	frites	frites
Pro Burger	cheeseburger	cheeseburger
Le Crasse Croute	bière	dürüm
Le Crasse Croute	cheeseburger	
Le Crasse Croute	dürüm	
Monsieur Frite	bière	
Monsieur Frite	frites	
Monsieur Frite	cheeseburger	
Monsieur Frite	dürüm	
Amour de crasse	bière	
Amour de crasse	frites	
Amour de crasse	dürüm	
The Fritkot	bière	
The Fritkot	frites	
The Fritkot	cheeseburger	
The Fritkot	dürüm	

Quels sont les snacks qui proposent de la bière, des frites, des cheeseburgers et des dürüms ?

Facile !
Il suffit d'opérer une division relationnelle,

Snacks ÷ Articles	
snack	
Monsieur Frite	
The Fritkot	

Mais comment faire en SQL ?

36

36

Le SQL vs la division relationnelle (1)

SNACKS		ARTICLES
snack	article	article
Pro Burger	bière	bière
Pro Burger	frites	frites
Pro Burger	cheeseburger	cheeseburger
Le Crasse Croute	bière	dürüm
Le Crasse Croute	cheeseburger	
Le Crasse Croute	dürüm	
Monsieur Frite	bière	
Monsieur Frite	frites	
Monsieur Frite	cheeseburger	
Monsieur Frite	dürüm	
Amour de crasse	bière	
Amour de crasse	frites	
Amour de crasse	dürüm	
The Fritkot	bière	
The Fritkot	frites	
The Fritkot	cheeseburger	
The Fritkot	dürüm	

SELECT snack FROM SNACKS
GROUP BY snack
HAVING COUNT(*) = (SELECT COUNT(*) FROM ARTICLES)

+ frites
 cervelas de cheval

Ne fonctionne pas s'il y a une redondance dans la table articles
 Ne fonctionne pas si des articles non référencés par la table articles sont présents dans la table snacks



37

37

Le SQL vs la division relationnelle (2)

SNACKS		ARTICLES
snack	article	article
Pro Burger	bière	bière
Pro Burger	frites	frites
Pro Burger	cheeseburger	cheeseburger
Le Crasse Croute	bière	dürüm
Le Crasse Croute	cheeseburger	frites
Le Crasse Croute	dürüm	
Monsieur Frite	bière	
Monsieur Frite	frites	
Monsieur Frite	cheeseburger	
Monsieur Frite	cervelas	
Amour de crasse	bière	
Amour de crasse	frites	
Amour de crasse	dürüm	
The Fritkot	bière	
The Fritkot	frites	
The Fritkot	cheeseburger	
The Fritkot	dürüm	

SELECT snack FROM SNACKS
GROUP BY snack
HAVING COUNT(*) = (SELECT COUNT(DISTINCT article) FROM ARTICLES)

Résout le problème de redondance

SELECT DISTINCT article FROM SNACKS
WHERE article NOT IN (SELECT article FROM ARTICLES)

Requête permettant de trouver le ou les articles présents dans la table SNACKS et non référencés dans la table ARTICLES

38

38

Le SQL vs la division relationnelle (3)

SNACKS		ARTICLES
snack	article	article
Pro Burger	bière	bière
Pro Burger	frites	frites
Pro Burger	cheeseburger	cheeseburger
Le Crasse Croute	bière	dürüm
Le Crasse Croute	cheeseburger	frites
Le Crasse Croute	dürüm	
Monsieur Frite	bière	
Monsieur Frite	frites	
Monsieur Frite	cheeseburger	
Monsieur Frite	cervelas	
Amour de crasse	bière	
Amour de crasse	frites	
Amour de crasse	dürüm	
The Fritkot	bière	
The Fritkot	frites	
The Fritkot	cheeseburger	
The Fritkot	dürüm	

SELECT DISTINCT snack **FROM** SNACKS
WHERE article **NOT IN** (**SELECT** article **FROM** SNACKS
WHERE article **NOT IN**
(**SELECT** article **FROM** ARTICLES))

GROUP BY snack
HAVING COUNT(article) =
(**SELECT** COUNT(**DISTINCT** article) **FROM** ARTICLES)

Snacks ÷ Articles	
snack	
The Fritkot	

39

39

Algèbre relationnelle

4.4 Suppléments

40

40

Les fonctions d'agrégation

PLATS				
id	type_plat	nom	prix	stock
1	Sandwich	Poulet-curry	2.50	35
2	Burger	Royal-Cheese	5.10	4
3	Sandwich	Martino	3.00	87
4	Burger	King-Bacon	5.40	0
5	Pizza	4 fromages	8.50	3

Normalement, c'est la lettre Σ (script F) à la place de **g**

Cette notation n'est pas toujours respectée

Σ COUNT(id), MIN(prix), MAX(prix), AVG(prix), SUM(stock) PLATS				
COUNT(id)	MIN(prix)	MAX(prix)	AVG(prix)	SUM(stock)
5	2.50	8.50	4.900000	129

A. Silberschatz, H. Korth, S. Sudarshan, Database System Concepts, 5th Edition

41

41

Agrégation avec GROUP BY

id	type_plat	nom	prix	stock
1	Sandwich	Poulet-curry	2.50	35
2	Burger	Royal-Cheese	5.10	4
3	Sandwich	Martino	3.00	87
4	Burger	King-Bacon	5.40	0
5	Pizza	4 fromages	8.50	3

GROUP BY

Affichage de la colonne

type_plat g type_plat, COUNT(id), MIN(prix), MAX(prix), AVG(prix), SUM(stock) PLATS					
type_plat	COUNT(id)	MIN(prix)	MAX(prix)	AVG(prix)	SUM(stock)
Burger	2	5.10	5.40	5.250000	4
Pizza	1	8.50	8.50	8.500000	3
Sandwich	2	2.50	3.00	2.750000	122

42

42

Le tri

PLATS				
id	type_plat	nom	prix	stock
1	Sandwich	Poulet-curry	2.50	35
2	Burger	Royal-Cheese	5.10	4
3	Sandwich	Martino	3.00	87
4	Burger	King-Bacon	5.40	0
5	Pizza	4 fromages	8.50	3

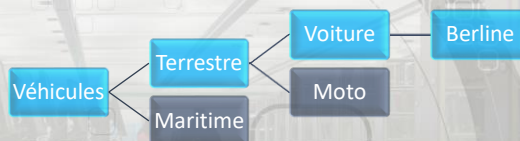
$\tau_{\text{type_plat, nom}} (\pi_{\text{type_plat, nom, prix}} \text{ PLATS})$		
type_plat	nom	prix
Burger	King-Bacon	5.40
Burger	Royal-Cheese	5.10
Pizza	4 fromages	8.50
Sandwich	Martino	3.00
Sandwich	Poulet-curry	2.50

43

43

La fermeture transitive

id	nom	parent
1	Véhicules	NULL
2	Terrestre	1
3	Maritime	1
4	Voiture	2
5	Moto	2
6	Berline	4



Les ancêtres de "Berline" ?

La fermeture transitive implique l'utilisation d'un algorithme récursif (si le nombre de niveau est inconnu) et l'algèbre relationnelle ne dispose pas d'un tel opérateur

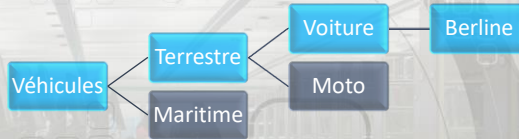
Et en SQL ?

44

44

La fermeture transitive

id	nom	parent
1	Véhicules	NULL
2	Terrestre	1
3	Maritime	1
4	Voiture	2
5	Moto	2
6	Berline	4



WITH RECURSIVE ancetres AS (
 SELECT * FROM vehicules
 WHERE nom='Berline'
 UNION
 SELECT v.* FROM (vehicules AS v)
 JOIN (ancetres AS a) ON (v.id=a.parent)
)
 SELECT * FROM ancetres;

WITH RECURSIVE n'est
disponible qu'à partir des
versions 8 de MySQL et
10.2.2 de MariaDB

Et en SQL ?



45

45

... il m'a aussi fallu un
peu de temps pour y
voir un intérêt

Il n'y a aucun intérêt
à étudier l'algèbre
relationnelle !
On a le SQL.

D'après moi, l'A.R. trouve toute son utilité, justement quand on
n'a pas la possibilité d'utiliser du SQL (complet).
Il faut alors implémenter "manuellement" un traitement sur les
données et c'est là que l'A.R. intervient en tant que langage de
conception.
Il est bien plus facile de traduire en algorithme une expression
A.R. que SQL. Ce cas de figure est fréquent avec les systèmes de
type NoSQL.

46

46

Résumé

Algèbre relationnelle	SQL
$\sigma_{\text{condition}}$	WHERE condition
$\pi_{\{A_1, A_2, \dots, A_n\}}$	SELECT col ₁ , col ₂ , ... col _n
$\rho_{\text{new/old}}$ OU $\rho(\text{new, old})$	AS
$\cup$	UNION
$\cap$	INTERSECT
$-$	EXCEPT
$R_1 \times R_2$	FROM R1, R2
$R_1 \bowtie_{\text{condition}} R_2$	R1 JOIN R2 ON condition
$R_1 \Join_{\text{condition}} R_2$	R1 LEFT JOIN R2 ON condition
$R_1 \Join_{\text{condition}} R_2$	R1 RIGHT JOIN R2 ON condition
$R_1 \Join_{\text{condition}} R_2$	R1 FULL JOIN R2 ON condition
$R_1 \bowtie R_2$	R1 NATURAL JOIN R2
$\% \text{ OU } /$	
$\Sigma_{\text{AVG}(\text{prix})} R \text{ OU } g_{\text{AVG}(\text{prix})} R$	AVG(prix) FROM R
$\text{col1, col2 } \Sigma \text{ OU } \text{col1, col2 } g$	GROUP BY col1, col2
$\tau_{\text{col1, col2}}$	ORDER BY col1, col2
	WITH RECURSIVE

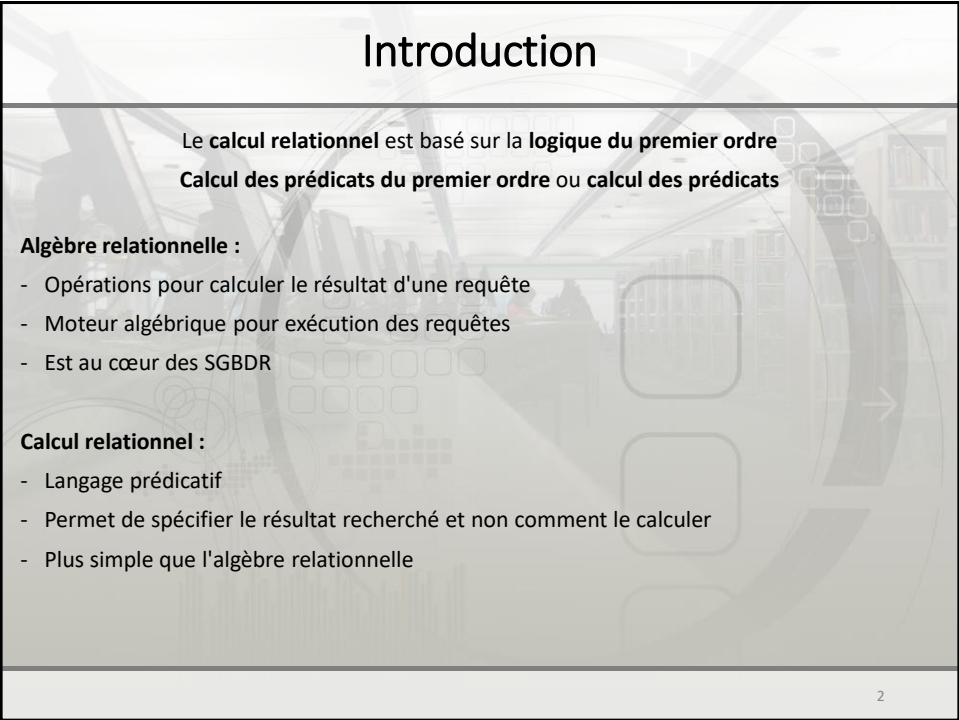


Chapitre 5

Calcul Relationnel

1

1



Introduction

Le **calcul relationnel** est basé sur la **logique du premier ordre**
Calcul des prédicats du premier ordre ou **calcul des prédicats**

Algèbre relationnelle :

- Opérations pour calculer le résultat d'une requête
- Moteur algébrique pour exécution des requêtes
- Est au cœur des SGBDR

Calcul relationnel :

- Langage prédicatif
- Permet de spécifier le résultat recherché et non comment le calculer
- Plus simple que l'algèbre relationnelle

2

2

Les langages prédicatifs

Calcul (relationnel) de tuples :

- Tuple Relational Calculus (TRC)
- Les variables dans les expressions sont des relations (tables)

$x \in \text{ARTICLES}$

Calcul relationnel de domaines :

- Domain Relational Calculus (DRC)
- Les variables dans les expressions sont des attributs

$x \in \text{ARTICLES.prix}$

3

3

Logique du premier ordre

Variables		Non A	A ET B	A OU B	A équivaut à B	A implique B
Table de vérité :		Connecteurs logiques :				
A	B	$\neg A$	$A \wedge B$	$A \vee B$	$A \leftrightarrow B$	$A \rightarrow B$
faux	faux	vrai	faux	faux	vrai	vrai
faux	vrai	vrai	faux	vrai	faux	vrai
vrai	faux	faux	faux	vrai	faux	faux
vrai	vrai	faux	vrai	vrai	vrai	vrai

$A \leftrightarrow B = \text{IF } A = B \text{ THEN vrai ELSE faux}$

$A \rightarrow B = \text{IF } A \text{ est vrai THEN } B \text{ ELSE vrai}$ 

Opérateurs de comparaison : $=, \neq, <, >, \leq, \geq$

Quantificateurs :

- Quantificateur universel : $\forall$ (pour tout)
- Quantificateur existentiel : $\exists$ (il existe au moins un)

4

4

Chapitre 5.1

Calcul relationnel de tuples

5

5

Structure d'une requête TRC

$\{ a.id_article, a.nom, a.prix \mid ARTICLES(a) \wedge a.type=Burger \wedge a.prix < 6€ \}$

Attributs retournés

Est vrai
si a est un tuple
de la relation ARTICLES

Prédicats à satisfaire

π
SELECT

FROM

σ
WHERE

Il est possible de rencontrer des requêtes TRC écrites sous cette forme :

$a \in ARTICLES$

$\{ a.id_article, a.nom, a.prix \mid a.type=Burger \wedge a.prix < 6€ \}$

6

6

Jointure

ARTICLES

id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
...	...	...	...

TYPES

id_type	nom
1	Sandwich
2	Burger
3	Pizza

$\{a.nom, a.prix \mid \text{ARTICLES}(a) \wedge \exists t (\text{TYPES}(t) \wedge t.nom = \text{Burger} \wedge t.id_type = a.id_type) \}$

7

7

Double jointure

VOITURES

id	nom	annee	prix
1	Polo	2010	8k
2	Corsa	2018	18k
...	...	...	...

VOITURES_COULEURS

id_voiture	id_couleur
1	3
2	1
2	2

COULEURS

id_couleur	couleur
1	Gris
2	Bleu
3	Pourpre

Afficher le nom et le prix des voitures grises :

$\{v.nom, v.prix \mid \text{VOITURES}(v) \wedge \exists vc \exists c (\text{VOITURES_COULEURS}(vc) \wedge \text{COULEURS}(c) \wedge vc.id_voiture = v.id_voiture \wedge c.couleur = \text{Gris} \wedge c.id_couleur = vc.id_couleur) \}$

Nous utilisons l'opérateur existentiel pour les relations dont les attributs ne sont pas désirés dans le résultat

8

8

Lien entre quantificateur et résultat

ARTICLES

id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
...	...	...	...

TYPES

id_type	nom
1	Sandwich
2	Burger
3	Pizza

$\{a.\text{nom}, a.\text{prix} \mid \text{ARTICLES}(a) \wedge \exists t (\text{TYPES}(t) \wedge t.\text{nom}=\text{Burger} \wedge t.\text{id_type}=a.\text{id_type})\}$

$\{a.\text{nom}, a.\text{prix}, t.\text{nom} \mid \text{ARTICLES}(a) \wedge \text{TYPES}(t) \wedge t.\text{nom}=\text{Burger} \wedge t.\text{id_type}=a.\text{id_type}\}$

9

9

Quantificateur universel

VOITURES

id	nom	annee	prix
1	Polo	2010	8k
2	Corsa	2018	18k
...	...	...	...

VOITURES_COULEURS

id_voiture	id_couleur
1	3
2	1
2	2

COULEURS

id_couleur	couleur
1	Gris
2	Bleu
3	Pourpre

Les noms et prix des voitures qui sont de toutes les couleurs :

$\{v.\text{nom}, v.\text{prix} \mid \text{VOITURES}(v) \wedge \forall c \text{ COULEURS}(c) \rightarrow \exists vc (\text{VOITURES_COULEURS}(vc) \wedge vc.\text{id_voiture}=v.\text{id_voiture} \wedge c.\text{id_couleur}=vc.\text{id_couleur})\}$

Pour toutes les couleurs, il faut que la condition d'existence $\exists vc$ soit validée

10

10

Chapitre 5.2

Calcul relationnel de domaines

11

11

Structure d'une requête DRC

Recherche des noms des clients de plus de 30 ans:

$$\{ cn \mid \exists ca \text{ CLIENTS}(\text{nom} : cn, \text{age} : ca) \wedge ca > 30 \}$$

Version simplifiée :

$$\{ cn \mid \text{CLIENTS}(\text{nom} : cn, \text{age} > 30) \}$$

12

12

Structure d'une requête DRC

Recherche d'e-mail et de numéro de téléphone de Sofia Green :

$$\{ cm, ct \mid \exists cn, cp \text{ CLIENTS}(\text{nom} : cn, \text{prenom} : cp, \text{email} : cm, \text{tel} : ct) \wedge cn = \text{'Green'} \wedge cp = \text{'Sofia'} \}$$

Version simplifiée :

$$\{ cm, ct \mid \text{CLIENTS}(\text{nom} = \text{'Green'}, \text{prenom} = \text{'Sofia'}, \text{email} : cm, \text{tel} : ct) \}$$

13

13

Jointure

ARTICLES

id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
...	...	...	...

TYPES

id_type	nom
1	Sandwich
2	Burger
3	Pizza

Recherche des noms et prix des articles de type Burger :

$$\{ an, pn \mid \exists tid (\text{ARTICLES}(\text{nom} : an, \text{prix} : pn, \text{id_type} : tid) \wedge \text{TYPES}(\text{nom} = \text{'Burger'} \wedge \text{id_type} : tid)) \}$$

En DRC, la jointure est implicite. Avec TRC il y a une condition de jointure explicite :

$$\{ a.\text{nom}, a.\text{prix} \mid \text{ARTICLES}(a) \wedge \exists t (\text{TYPES}(t) \wedge t.\text{nom} = \text{'Burger'} \wedge t.\text{id_type} = a.\text{id_type}) \}$$

14

14

Double jointure

VOITURES

id	nom	annee	prix
1	Polo	2010	8k
2	Corsa	2018	18k
...	...	...	...

VOITURES_COULEURS

id_voiture	id_couleur
1	3
2	1
2	2

COULEURS

id_couleur	couleur
1	Gris
2	Bleu
3	Pourpre

Afficher le nom et le prix des voitures grises :

$$\{vn, vp \mid \exists vi$$

$$(COULEURS(id_couleur : cid, couleur = 'Gris') \wedge$$

$$VOITURES_COULEURS(id_voiture : vi, id_couleur : cid) \wedge$$

$$VOITURES(id : vi, nom : vn, prix : vp))\}$$

15

15

Quantificateur universel

VOITURES

id	nom	annee	prix
1	Polo	2010	8k
2	Corsa	2018	18k
...	...	...	...

VOITURES_COULEURS

id_voiture	id_couleur
1	3
2	1
2	2

COULEURS

id_couleur	couleur
1	Gris
2	Bleu
3	Pourpre

Les noms et prix des voitures qui sont de toutes les couleurs :

$$\{vn, vp \mid \exists vi \text{ VOITURES}(id : vi, nom : vn, prix : vp) \wedge$$

$$\forall ci (COULEURS(id_couleur : ci) \rightarrow$$

$$VOITURES_COULEURS(id_voiture = vi, id_couleur = ci))\}$$

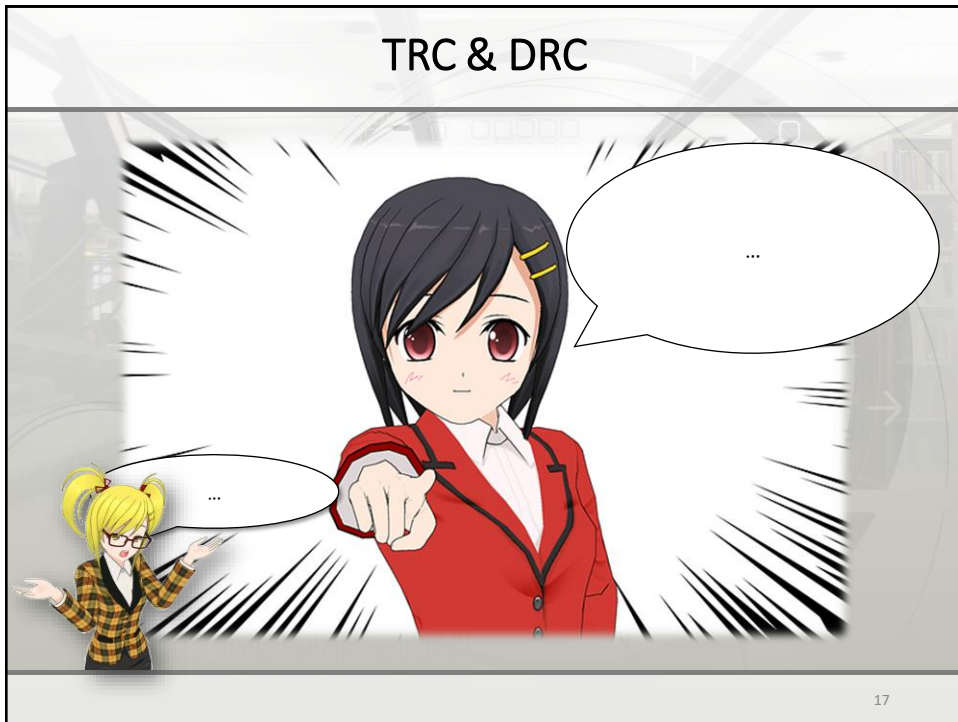
Les noms et prix des voitures qui sont sans couleur :

$$\{vn, vp \mid \exists vi \text{ VOITURES}(id : vi, nom : vn, prix : vp) \wedge$$

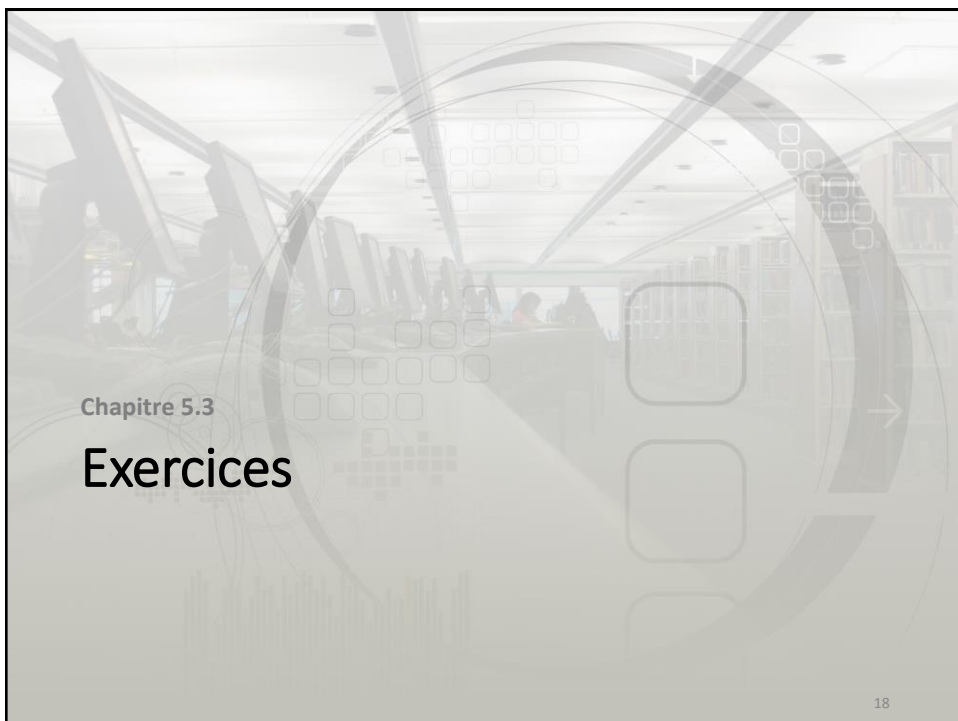
$$\forall ci (VOITURES_COULEURS(id_voiture \neq vi, id_couleur = ci))\}$$

16

16



17



18



1



2

Recherche sans index

id	nom	prénom	tel
1	Green	Roby	0471 123 123
2	Blue	Will	0486 123 123
3	Pink	Jen	0470 123 123
4	Red	Sarah	0477 123 123
5	Pink	Elise	0473 123 123
6	White	John	0499 123 123
7	Black	Jack	0494 123 123
...	...	...	...

rep array()

```
foreach (CLIENTS as CLIENT) {
    if (CLIENT[tel]==0494 123 123)
        rep.add (CLIENT[id])
}
```

Complexité $O(n)$

Qui à le numéro
0499 123 123



3

3

Recherche avec index

TELindex

tel	id
0470 123 123	3
0471 123 123	1
0473 123 123	5
0477 123 123	4
0486 123 123	2
0494 123 123	7
0499 123 123	6
...	...

CLIENTS

id	nom	prénom	tel
1	Green	Roby	0471 123 123
2	Blue	Will	0486 123 123
3	Pink	Jen	0470 123 123
4	Red	Sarah	0477 123 123
5	Pink	Elise	0473 123 123
6	White	John	0499 123 123
7	Black	Jack	0494 123 123
...	...	...	...

Avec un index unique de
type liste triée :

Pire cas : $O(\log_2 n)$

Non-unique :

Pire cas : $\sim O(n)$

En pratique : $\sim O(\log_2 n)$

```
recherche_dichotomique(clé, liste) {
    x= liste.taille() / 2;
    if (liste[x].tel == clé) return CLIENTS[liste[x].id];
    if (liste[x].tel > clé) return recherche_dichotomique(clé, liste[1:x]);
    else return recherche_dichotomique(clé, liste[x:n]);
}
recherche_dichotomique(0499 123 123, TELindex);
```

4

4

Indexation



Avantages des index :

- Accélèrent (considérablement) les recherches, jointures et agrégations



Inconvénients :

- Occupent de l'espace
- Ralentissent les mises à jour (INSERT, UPDATE et DELETE)

Différentes structures d'indexation (B-Arbre, Hash, bitmap)

Plus d'infos sur l'indexation
au chapitre suivant !



5

5

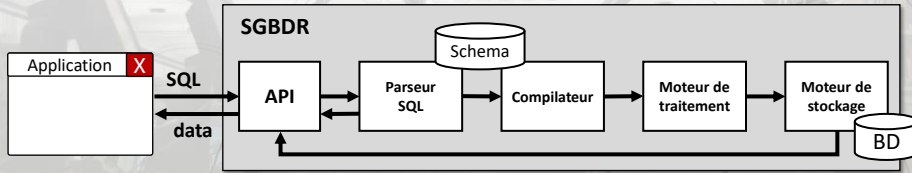
Chapitre 6.2

Les requêtes préparées

6

6

Architecture interne d'un SGBDR



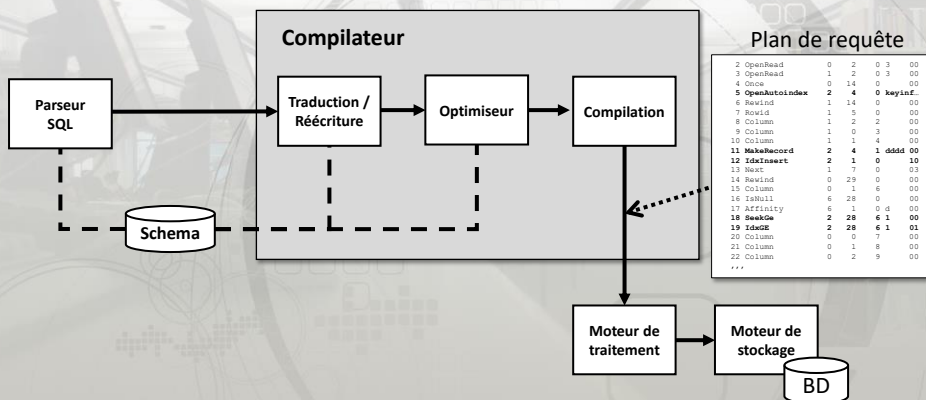
Processus de traitement d'une requête :

1. L'application envoie la requête au travers de l'API
2. La requête va être parsée et vérifiée Si peu de données ← Prend un peu de temps
3. Un plan d'exécution va être généré par le compilateur ← Prend beaucoup de temps
4. Le moteur de traitement va exécuter le plan...
5. ... en accédant aux données via le moteur de stockage
6. Le résultat est renvoyé au client

7

7

Le compilateur SQL



L'idée première des requêtes préparées :

- Court-circuiter la phase d'analyse du parseur ainsi que toutes les étapes du compilateur

8

8

Architecture interne d'un SGBDR

Processus de traitement d'une requête préparée :

Phase 1 :

1. L'application envoie la requête au travers de l'API
2. La requête va être parsée et vérifiée
3. Un plan d'exécution va être généré par le compilateur
4. Sauvegarde du plan et envoi d'un identifiant de requête au client

Phase 2 :

1. Le client envoie l'identifiant de plan accompagné d'éventuels paramètres
2. Le moteur de traitement va exécuter le plan...
3. ... en accédant aux données via le moteur de stockage
4. Le résultat est renvoyé au client

Si une même requête doit être exécutée plusieurs fois,

après la 1^{re} exécution, seule la phase 2 est nécessaire

9

9

Requête préparée en SQL

Préparation d'une requête :

```
PREPARE test FROM 'SELECT * FROM matable WHERE ID=? AND truc>?';
```

Les '?' présents dans la requête représentent des paramètres.

Déclaration des variables :

```
SET @x=104 ;  
SET @y=18 ;
```

Exécution de la requête :

```
EXECUTE test USING @x, @y ;
```

Gros avantage des requêtes préparées : **seul le code SQL préparé est compilé.**

Si les paramètres envoyés après compilation contiennent du code SQL malicieux, il ne sera pas exécuté !

Bien utilisées, elles permettent donc d'éviter les injections SQL



10

10

Exemple avec PDO et PHP

```
...  
$connexion = new PDO("...", "...", "...");  
  
$query = "INSERT INTO clients (nom, prenom, blabla, quand)  
VALUES (:nom, :prenom, :blabla, NOW() )";  
  
$result = $connexion->prepare($query);  
  
$result->bindValue("nom", $x);  
$result->bindValue("prenom", $y);  
$result->bindValue("blabla", $z);  
  
$result->execute();  
  
...
```

11

11

Exemple 2 avec PDO et PHP

```
...  
$connexion = new PDO("...", "...", "...");  
$tab = array(1,4,45,3,17,62);  
  
$query = "SELECT * FROM unetable WHERE id=:id";  
$result = $connexion->prepare($query);  
  
foreach($tab as $t)  
{  
    $result->bindValue("id", $t);  
    $result->execute();  
    ...  
}  
...
```

Ok ce code n'est pas optimal... autant faire une seule requête avec un WHERE id IN ().

C'était juste pour l'exemple ☺



12

12

Exemple 3 de code avec PDO et PHP

```
...  
$connexion = new PDO("...", "...", "...");  
  
$query = "INSERT INTO clients (nom, prenom, blabla, quand)  
VALUES ('$x', '$y', '$z' , NOW() )";  
$result = $connexion->prepare($query);  
  
$result->execute();  
...
```

Ici nous avons
un code bien **stupide** !

Ce code ne permet pas d'éviter les
injections SQL étant donné que la requête
est compilée **AVEC** ses paramètres



13

13

Requête préparée et session

ATTENTION : une requête préparée est stockée en session utilisateur

Pour supprimer une requête préparée :

DROP PREPARE test ;

ou

DEALLOCATE PREPARE test ;

14

14

Requête préparée



Avantages des requêtes préparées

1. Protection contre les injections SQL
2. Accélèrent les scripts qui exécutent une même requête en boucle
3. Peuvent réduire sensiblement la charge réseau pour les scripts du point ci-dessus

Ce genre de script est peu courant dans un système bien conçu.



Inconvénients des requêtes préparées

1. Légère baisse de réactivité
2. Légère augmentation de la charge réseau

} Virtuellement 2 interactions
client/serveur
pour une seule requête

15

15

Chapitre 6.3

VUES

16

16

Exemple

VENDEURS
<u>id_vendeur</u>
nom
prenom

RECETTES
<u>id_vendeur</u> (FK)
<u>date_recette</u>
montant

Chaque vendeur encode chaque jour le total de ses recettes.

Nous voulons récupérer le total des recettes par vendeur et par mois. Nous voulons obtenir les colonnes 'année', 'mois', 'nom', 'prénom' et 'recettes' :

```
SELECT YEAR( r.date_recette ) AS 'annee', MONTH( r.date_recette ) AS 'mois',  
       v.id_vendeur, v.nom, v.prenom, SUM( r.montant ) AS 'recettes'  
FROM (recettes AS r ) JOIN (vendeurs AS v) ON (v.id_vendeur=r.id_vendeur)  
GROUP BY annee, mois, r.id_vendeur
```

17

17

Exemple avec une VUE

VENDEURS
<u>id_vendeur</u>
nom
prenom

RECETTES
<u>id_vendeur</u> (FK)
<u>date_recette</u>
montant

Nous voulons **créer une VUE** pour récupérer le total des recettes par vendeur et par mois. Nous voulons obtenir les colonnes 'année', 'mois', 'nom', 'prénom' et 'recettes' :

```
CREATE VIEW recettes_mensuelles_vendeurs AS  
SELECT YEAR( r.date_recette ) AS 'annee', MONTH( r.date_recette ) AS 'mois',  
       v.id_vendeur, v.nom, v.prenom, SUM( r.montant ) AS 'recettes'  
FROM (recettes AS r ) JOIN (vendeurs AS v) ON (v.id_vendeur=r.id_vendeur)  
GROUP BY annee, mois, r.id_vendeur
```

La vue s'utilise ensuite comme une table :

```
SELECT * FROM recettes_mensuelles_vendeur WHERE annee=2019
```

18

18

Utilité des vues

Utilité principale :

- Méthode d'abstraction qui permet de rendre l'application agnostique du schéma

Utilité secondaire :

- Offre un mécanisme de confidentialité supplémentaire en permettant de masquer des colonnes à un compte qui n'aurait accès qu'aux vues.

Fonctionnalités souvent méconnue :

- Il est possible de mettre à jour les tables si la requête originelle n'est pas ambiguë
- S'il est possible d'effectuer des mises à jour et que la requête originelle n'est "vraiment" pas ambiguë, il peut également être possible d'effectuer des insertions.

(exemple : toutes les colonnes non-nulles et sans valeur par défaut doivent être présentes dans la requête)

19

19

Abstraction du schéma par les Vues



20

20

Test de performance

Comparons les durées d'exécution de la requête et de sa vue correspondante avec un filtre :

```
SELECT YEAR( r.date_recette ) AS 'annee', MONTH( r.date_recette ) AS 'mois',  
       v.id_vendeur, v.nom, v.prenom, SUM( r.montant ) AS 'recettes'  
FROM (recettes AS r ) JOIN (vendeurs AS v) ON (v.id_vendeur=r.id_vendeur)  
WHERE v.id_vendeur IN (1,2)  
GROUP BY annee, mois, v.id_vendeur
```

```
SELECT * FROM recettes_mensuelles_vendeur WHERE id_vendeur IN (1,2)
```

VENDEURS
<u>Id_vendeur</u>
nom
prenom

200 vendeurs

RECETTES
<u>id_vendeur</u> (FK)
<u>date_recette</u>
montant

658 414 recettes

Config de test :

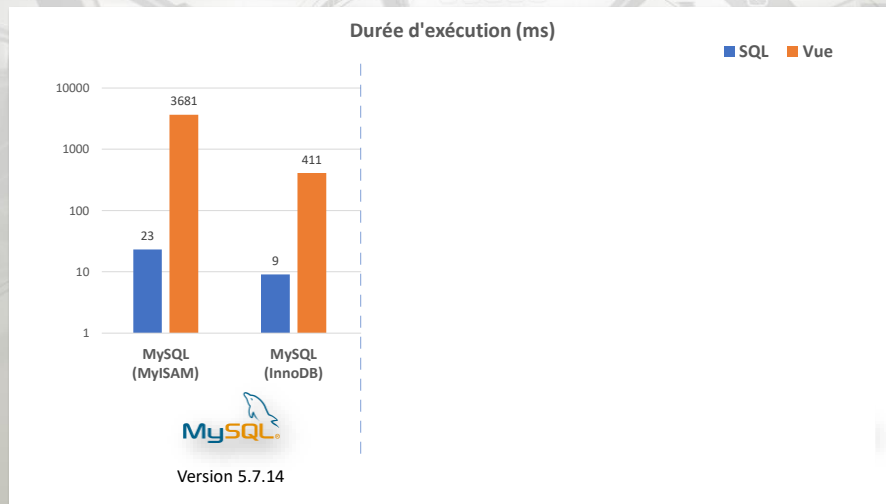
Core i7 6700K (4 core / 8 threads @ 4GHz)
64 Go DDR4 2400MHz Dual Channel
RAID 1 : 2 x Seagate Constellation 1 To SATA III
Windows 10 Pro

Tests effectués à chaud

21

21

Test de performance



22

22

Performances des vues

Pour l'exemple précédent, MySQL et MariaDB traitent la vue en 2 temps :

1. la requête source de la vue est exécutée;
2. le filtre est appliqué sur le résultat.

PostgreSQL, quant à lui, va procéder autrement :

1. le filtre va être ajouté à la requête initiale;
2. cette "nouvelle" requête va être exécutée.

Avec MySQL et MariaDB, aucun index ne peut être utilisé par le filtre, contrairement à PostgreSQL qui va générer un nouveau plan de requête optimisé.

23

23

Performances des vues

Attention, les vues ne sont pas systématiquement plus lentes :

```
SELECT YEAR( r.date_recette ) AS 'annee', MONTH( r.date_recette ) AS 'mois',  
        v.id_vendeur, v.nom, v.prenom, SUM( r.montant ) AS 'recettes'  
FROM (recettes AS r ) JOIN (vendeurs AS v) ON (v.id_vendeur=r.id_vendeur)  
GROUP BY annee, mois, v.id_vendeur
```

```
SELECT * FROM recettes_mensuelles_vendeur
```

Même durée peu importe le SGBDR

24

24

Vues indexées

```
SELECT * FROM recettes_mensuelles_vendeur WHERE annee=2008
```

Il pourrait être intéressant d'ajouter un index sur la colonne "annee" de la vue mais, à l'exception de SQL server et Oracle qui gèrent les "indexed views", il n'est pas possible d'ajouter des index aux vues :



Alors... il n'y a pas moyen d'accélérer les vues ?

Non... mais on peut utiliser des vues matérialisées



25

Vue matérialisée

Une vue matérialisée : remplacer la vue par une vraie table

Comme c'est une table on peut donc indexer des colonnes.

Mais alors tu introduis une **##*#!#** de redondance dans la BD... Tu fais un gros doigt aux formes normales quoi !

À toi de voir sur tu préfères un système poussif respectant les F.N. à un système réactif qui s'écarte des notions théoriques



OK mais alors... pour chaque MAJ des données faut également penser à faire des requêtes pour MAJ tes vues matérialisées... c'est pas un peu lourd ?

T'inquiète, pour ça il y a les "trigger"



26

Déclencheurs

27

27

Définition

Un déclencheur (ou Trigger) est un code que le SGBD doit exécuter dès qu'un événement particulier se produit.

Les déclencheurs peuvent être déclenchés (...) à partir des commandes DML suivantes :

- **INSERT**
- **UPDATE**
- **DELETE**

ATTENTION : ils ne peuvent être déclenchés suite aux opérations DDL comme DROP ou TRUNCATE

Le code exécuté par le trigger peut se faire :

- Soit **avant ou après** un INSERT, UPDATE ou DELETE
- Soit une fois par ligne modifiée, soit une fois par requête SQL

28

28

Exemple (1)

Nous désirons une vue matérialisée pour cette requête :

```
SELECT YEAR( r.date_recette ) AS 'annee', MONTH( r.date_recette ) AS 'mois',  
       v.id_vendeur, SUM( r.montant ) AS 'recettes'  
FROM (recettes AS r ) JOIN (vendeurs AS v) ON (v.id_vendeur=r.id_vendeur)  
GROUP BY annee, mois, r.id_vendeur
```

1. On crée la table :

```
CREATE TABLE vuemat_recettes (  
  annee INTEGER NOT NULL,  
  mois  INTEGER NOT NULL,  
  id_vendeur INTEGER NOT NULL,  
  recettes DECIMAL(10,2) NOT NULL,  
  PRIMARY KEY(annee, mois, id_vendeur),  
  INDEX(id_vendeur)  
);
```

29

29

Exemple (2)

2. On crée le trigger qui devra se déclencher pour chaque insertion dans la table recettes :

DELIMITER |

```
CREATE TRIGGER trigger_insert_recettes  
BEFORE INSERT ON recettes  
FOR EACH ROW  
BEGIN  
  INSERT INTO vuemat_recettes (annee, mois, id_vendeur, recettes)  
  VALUES (YEAR(NEW.date_recette), MONTH(NEW.date_recette),  
          NEW.id_vendeur, NEW.montant)  
  ON DUPLICATE KEY  
  UPDATE recettes= recettes + NEW.montant;  
  
END |  
  
DELIMITER ;
```

RECETTES

<u>id_vendeur</u> (FK)
<u>date_recette</u>
montant

VUEMAT_RECETTES

<u>annee</u>
<u>mois</u>
<u>id_vendeur</u> (FK)
recettes

30

30

Exemple (3)

3. On crée le trigger qui devra se déclencher pour chaque UPDATE pour la table recettes :

DELIMITER |

```
CREATE TRIGGER trigger_update_recettes
BEFORE UPDATE ON recettes
FOR EACH ROW
BEGIN
    UPDATE vuemat_recettes
        SET recettes= recettes + (NEW.montant – OLD.montant)
        WHERE annee=YEAR(NEW.date_recette)
            AND mois=MONTH(NEW.date_recette)
            AND id_vendeur=NEW.id_vendeur;
END |
DELIMITER ;
```

RECETTES
<u>id_vendeur</u> (FK)
<u>date_recette</u>
montant

VUEMAT_RECETTES
<u>annee</u>
<u>mois</u>
<u>id_vendeur</u> (FK)
recettes

31

31

Exemple (4)

4. On crée le trigger qui devra se déclencher pour chaque DELETE pour la table recettes :

DELIMITER |

```
CREATE TRIGGER trigger_delete_recettes
BEFORE DELETE ON recettes
FOR EACH ROW
BEGIN
    UPDATE vuemat_recettes
        SET recettes= recettes – OLD.montant
        WHERE annee=YEAR(OLD.date_recette)
            AND mois=MONTH(OLD.date_recette)
            AND id_vendeur=OLD.id_vendeur;
END |
DELIMITER ;
```

RECETTES
<u>id_vendeur</u> (FK)
<u>date_recette</u>
montant

VUEMAT_RECETTES
<u>annee</u>
<u>mois</u>
<u>id_vendeur</u> (FK)
recettes

32

32

Trigger et vues matérialisées



- **Les vues matérialisées sont des vraies tables et permettent d'accélérer les recherches :**

- possibilité de créer des index ;
- données directement accessibles (pas besoin de les calculer dynamiquement) ;
- contiennent souvent moins de données que les tables sources (pas toujours nécessaire d'indexer).

- **Les déclencheurs :**

- offrent un mécanisme transparent pour les "codeurs" ;
- permettent de maintenir la BD dans un état cohérent.



- **Les vues matérialisées :**

- occupent de la place ;
- introduisent de la redondance.

- **L'utilisation des déclencheurs à un coût :**

Toute opération d'écriture dans une table pourvue de déclencheurs provoquera un **overhead** lié aux écritures additionnelles et aux éventuels maintiens des index.

33

33

Cas d'utilisation

C'est génial ton truc mais ce n'est adapté qu'aux cas où il y a bien plus de lectures que d'écritures en BD

Exact !

Donc pour ce type de cas, avec les vues matérialisées et les déclencheurs, je vais avoir des perfs de ouf au prix de juste un peu d'espace de stockage.

Dans ces conditions, autant utiliser systématiquement cette technique !

OUI... mais en fait... il y a quand même un petit soucis...

34

Gestion des contraintes d'intégrité référentielle



FOREIGN KEY (`id\_vendeur`) **REFERENCES** `vendeurs`(`id\_vendeur`)
ON DELETE **CASCADE** **ON UPDATE** **CASCADE**

Si on efface un vendeur, alors toutes les entrées de ce vendeur dans la table recettes seront automatiquement supprimées (possible avec InnoDB).

On pourrait s'attendre à ce que la suppression des enregistrements de la table recettes provoque le trigger associé à cette table mais avec MySQL/MariaDB... il n'en est rien.

ATTENTION : avec MySQL/MariaDB, les déclencheurs sont liés aux commandes SQL, pas aux opérations internes du SGBD...

35

Cascade de trigger

Si MySQL et MariaDB implémentent les déclencheurs de cette façon, c'est pour éviter la problématique de la **CASCADE de trigger** (ce qui permet de ne l'éviter que partiellement...).

Une cascade de trigger c'est quand un trigger déclenche un autre trigger qui en déclenche un autre etc. jusqu'à redéclencher le trigger initial ce qui provoque **une récursion infinie...**

C'est une des raisons qui pousse les hébergeurs à désactiver les triggers et procédures stockées pour les **hébergements mutualisés**.

Pour prévenir ce genre de situation des SGBDR comme PostgreSQL permettent de checker le nombre d'appels avec la fonction **pg_trigger_depth()** (à condition de s'en servir).



OK mais et pour MySQL on fait comment ?

Il y a 4 solutions !

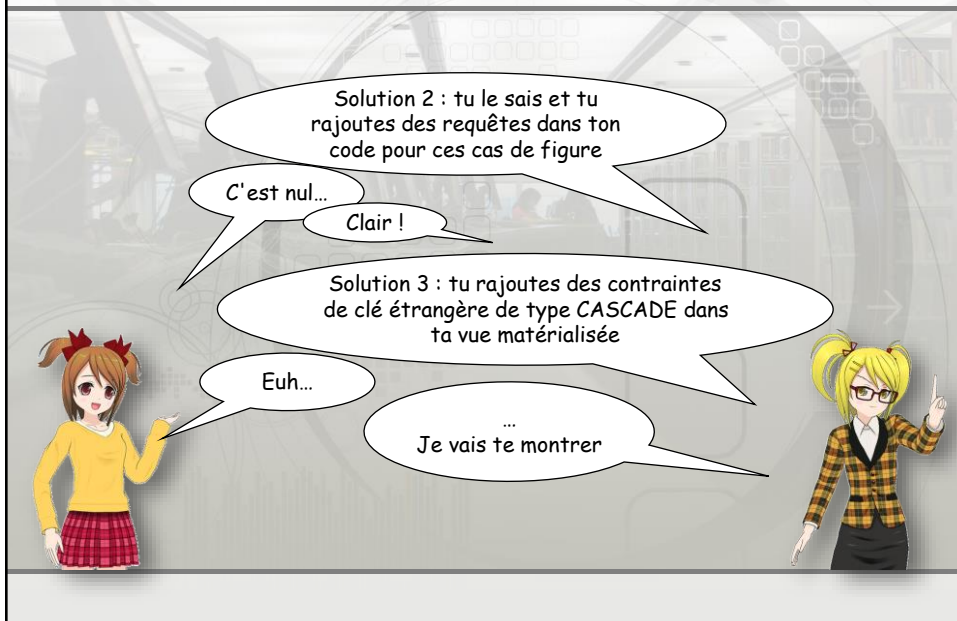
Solution 1 : tu n'utilises pas MySQL/MariaDB hihhi

...



36

Solutions pour MySQL/MariaDB



37

Gestion des contraintes d'intégrité référentielle

**FOREIGN KEY (`id\_vendeur`) REFERENCES `vendeurs`(`id\_vendeur`)
ON DELETE CASCADE ON UPDATE CASCADE**

VENDEURS
<u>id_vendeur</u>
nom
prenom

RECETTES
id_vendeur (FK)
<u>date_recette</u>
montant

VUEMAT_RECETTES
<u>annee</u>
<u>mois</u>
id_vendeur (FK)
recettes

Trigger

**FOREIGN KEY (`id\_vendeur`)
REFERENCES `vendeurs`(`id\_vendeur`)
ON DELETE CASCADE ON UPDATE CASCADE**

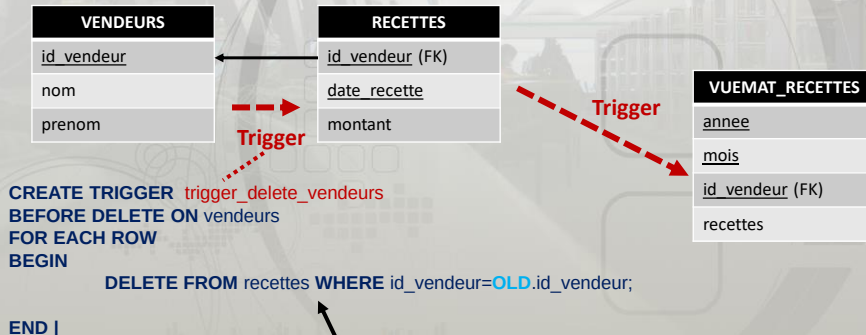
C'est cool ça ! Mais je suppose que tu vas me dire que la meilleure solution c'est la 4...

Pas forcément.
C'est juste ma préférée

38

Gestion des contraintes d'intégrité référentielle

Solution 4 : Avec MySQL et MariaDB, on se passe des effets de CASCADE pour les clés étrangères et on utilise des triggers un peu partout :



Comme le trigger contient du SQL, en cas de suppression d'un vendeur, le second trigger sera automatiquement exécuté.

39

Chapitre 6.5

Fonctions et Procédures stockées

40

40

Fonction

Il est possible de créer ses propres fonctions dans le SGBD :

```
CREATE FUNCTION cm2inch(cm FLOAT)  
RETURNS FLOAT  
BEGIN  
    RETURN (cm / 2.54);  
END
```

```
mysql> select cm2inch(25.4);  
+-----+  
| cm2inch(25.4) |  
+-----+  
|          10 |  
+-----+  
1 row in set (0.00 sec)
```

Contrairement à une procédure, une fonction doit fournir une sortie (une valeur ou une relation)

Une fonction ne peut pas :

- Gérer des transactions
- Faire des mise à jour (INSERT, UPDATE, DELETE)
- Pas de DDL (CREATE, ALTER, TRUNCATE, etc.)
- Pas d'EXEC/CALL (appel à des procédures stockées)

41

Procédure stockée

C'est comme une fonction, mais sans les limitations inerrantes.

```
CREATE PROCEDURE nom_de_la_procedure([parametres])  
BEGIN  
    ...  
END
```

Avec les procédures stockées, les paramètres peuvent être :

- **IN**
- **OUT**
- **INOUT**

Une procédure est appelée avec la commande : **CALL maprocedure('machin');**

Contrairement aux requêtes préparées, les fonctions et procédures stockées ne sont pas stockées en session utilisateur. Elles sont sauvegardées durablement sur le serveur.

Elles peuvent même être déclenchées par un trigger

42

SQL/PSM

SQL / Persistent Stored Modules

Fait partie de la norme SQL

Un **langage procédural** utilisé pour les procédures stockées

MySQL et MariaDB* supportent un dialecte SQL/PSM

PostgreSQL aussi : le **PL/pgSQL**

Oracle également : **PL/SQL**

Tout comme SQL Server : **Transact-SQL**

* Les dernières version de MariaDB sont également compatibles PL/SQL (pas totalement)

Subdivision du SQL

DDL Data Definition Language	Création et modification de la BD CREATE, ALTER, DROP, TRUNCATE
DML Data Manipulation Language	Traitement des données INSERT, UPDATE, DELETE, SELECT
DCL Data Control Language	Gestion des droits d'accès GRANT, REVOKE
TCL Transaction Control Language	Contrôle la bonne exécution des transactions SET TRANSACTION, COMMIT, ROLLBACK
SQL/PSM Persistent Stored Module	Langage procédural pour les procédures stockées BEGIN, END, IF THEN, ELSE, etc.

43

Procédure stockée : exemple 1

DELIMITER |

```
CREATE PROCEDURE volume_cube(IN taille INT, OUT volume INT)
BEGIN
    SELECT POW(taille,3) INTO volume;
END|
```

DELIMITER ;

Appel de la procédure :

```
mysql> CALL volume_cube(10, @x);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT @x;
+-----+
| @x    |
+-----+
| 1000  |
+-----+
```

44

Procédure stockée : exemple 2 (1)

Comme pour les vues, les procédures stockées peuvent être utilisées comme mécanisme d'abstraction du schéma

Procédure permettant d'insérer un plat :

```
CREATE PROCEDURE insert_plat (
  IN vnom VARCHAR(255),
  IN vprix FLOAT,
  IN vplat VARCHAR(255))
BEGIN
  INSERT INTO plats (nom,prix,type_plat) VALUES (vnom, vprix, vplat);
  SET p_lastid= LAST_INSERT_ID();
  SELECT p_lastid AS 'id';
END |
```

id	nom	prix	type_plat
1	Poulet-curry	2.50	Sandwich
2	Royal-Cheese	5.10	Burger
...	...	...	...

PLATS

Cette procédure permet même de récupérer l'id (auto incrémenté) du plat nouvellement inséré.

45

Procédure stockée : exemple 2 (2)

Et puis on décide de diviser la table en 2. Il suffit alors de modifier la procédure sans toucher à un seul octet de l'application :

```
CREATE PROCEDURE insert_plat (
  IN vnom VARCHAR(255),
  IN vprix FLOAT,
  IN vplat VARCHAR(255))
BEGIN
  DECLARE p_lastid INT;
  DECLARE t_lastid INT;

  SELECT id_type INTO t_lastid FROM types_plats WHERE nom=vplat;
  IF t_lastid IS NULL THEN
    INSERT INTO types_plats (nom) VALUES (vplat);
    SET t_lastid = LAST_INSERT_ID();
  END IF;

  INSERT INTO plats (id_type,nom,prix) VALUES (t_lastid, vnom, vprix);
  SET p_lastid= LAST_INSERT_ID();
  SELECT p_lastid AS 'id', t_lastid AS 'id_type';
END |
```

id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
...	...	...	...

PLATS

id_type	nom
1	Sandwich
2	Burger
3	Pizza

TYPES_PLATS

46

Les procédures stockées



- Fournissent un mécanisme d'abstraction du schéma pour les utilisateurs :
 - Code de l'application plus simple
 - Pas d'accès direct aux tables (meilleure sécurité)
- Meilleure réactivité car les procédures stockées sont précompilées
- Gains substantiels au niveau de la charge réseau :
 - Requêtes plus courtes
 - Moins d'interactions
- Possible d'appeler des scripts externes



- Syntaxe plutôt "has been" et fonctionnalités limitées du langage
- Elles sont précompilées, ce qui ne protège pas totalement contre les injections SQL si l'implémentation est maladroite.
- Nécessite un peu de "skill"

47

47

Bases de données épaisses

La BD est accédée uniquement par des :

- **Vues (matérialisées)** : pour les lectures
- **Procédures Stockées** : pour les écritures

Rend le code de l'application agnostique du schéma de la BD
(bien plus qu'avec un ORM qui lui... **ne l'est pas**)

Cette approche est souvent la
meilleure façon de procéder.

Par contre, elle est plus complexe à mettre
en œuvre et requiert donc plus de "skill"



48

48

Contraintes et Assertions

49

49

Contraintes

Les contraintes permettent de se prémunir contre l'enregistrement de données incohérentes

Types de contraintes en SQL :

- 1. Les contraintes de domaine :** pour la validation d'une valeur
- 2. Les contraintes de table :** pour la validation d'une ligne
- 3. Les assertions :** pour valider des ensembles de lignes

50

50

Les contraintes de domaine

Pour la validation d'une valeur.

1. Le DBA crée un nouveau type de données décrivant une plage de valeurs possibles
2. Les colonnes qui sont de ce nouveau type devront obligatoirement stocker des données conformes à la plage de valeurs définie.

```
CREATE DOMAIN type_age int CONSTRAINT contrainte_sur_age CHECK( VALUE>17 AND VALUE<130);
```

```
CREATE TABLE clients (  
    id_client INT PRIMARY KEY,  
    nom VARCHAR(255) NOT NULL,  
    prenom VARCHAR(255) NOT NULL,  
    age type_age NOT NULL  
);
```

51

51

Les contraintes de domaine

Autre exemple :

```
CREATE DOMAIN nom_jour VARCHAR(10) CONSTRAINT contrainte_sur_jour  
CHECK(VALUE IN ('lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi', 'samedi', 'dimanche'));
```

Ce procédé n'est pas supporté par
MySQL et MariaDB

Cependant, on peut s'en passer grâce
aux contraintes de table.



52

52

Les contraintes de table

Pour valider un enregistrement dans une table.

Les contraintes de table :

- **NOT NULL** : oblige à fournir une valeur non nulle
- **UNIQUE** : il ne peut y avoir plusieurs fois la même valeur (sera indexé) mais peut éventuellement contenir des valeurs nulles
- **PRIMARY KEY** : comme UNIQUE mais il ne peut pas y avoir de valeurs nulles (sera indexé)
- **FOREIGN KEY** : la valeur devra correspondre à une PK d'une autre table et il peut éventuellement contenir des valeurs nulles
- **DEFAULT** : permet de fournir une valeur par défaut (pas réellement une contrainte)
- **CHECK(condition)** : permet définir les conditions à respecter (plage de valeur)

53

53

Les contraintes de table

```
CREATE TABLE articles (  
  id_article INT NOT NULL,  
  reference VARCHAR(255) NOT NULL,  
  nom_article VARCHAR(255) NOT NULL,  
  stock INT DEFAULT 0,  
  prix DECIMAL(10,2) NOT NULL,  
  id_categorie INT,  
  
  PRIMARY KEY(id_article),  
  UNIQUE(nom_article) ,  
  CHECK(prix>0),  
  FOREIGN KEY(id_categorie) REFERENCES categories(id_categorie),  
  INDEX index_fk_categorie (id_categorie)  
);
```



Quand vous définissez une clé étrangère, le SGBD ne va pas forcément automatiquement y associer un index. Pensez donc à bien vérifier vos index.

54

54

Les assertions

Permettent de valider des ensembles de lignes

PERSONNEL
<u>id_personne</u>
nom
...

PERSONNEL_SERVICE
<u>id_personne</u> (FK)
<u>id_service</u> (FK)
<u>id_role</u> (FK)

SERVICES
<u>id_service</u>
service

ROLES
<u>id_role</u>
role

Ce procédé n'est pas supporté par MySQL et MariaDB

Cependant, on peut s'en passer grâce à triggers et/ou aux procédures stockées

```
CREATE ASSERTION assertion_truc_machin CHECK (  
  (SELECT COALESCE(MAX(t.x), 0) FROM  
    (SELECT COUNT(*) AS 'x' FROM services AS s  
      JOIN personnel_service AS ps ON (ps.id_service=s.id_service  
        AND ps.id_role= (SELECT id_role FROM roles WHERE role='Chef'))  
      GROUP BY s.id_service) AS t) <= 1  
)
```

Cette assertion va empêcher qu'il y ait plus d'une personne qui endossent le rôle de "Chef" pour un même service



55

55

Chapitre 6.7

Les transactions

56

56

Mise en contexte (1)



Vegeta commande sur Amazon le dernier Call of Duty

57

57

Mise en contexte (2-ok)

UPDATE compte SET argent = argent - 39.99 WHERE client = 'Vegeta'

UPDATE compte SET argent = argent + 39.99 WHERE client = 'Amazon'



58

58

Mise en contexte (2-nok)

UPDATE compte SET argent = argent - 39.99 WHERE cli

UPDATE compte SET arge....

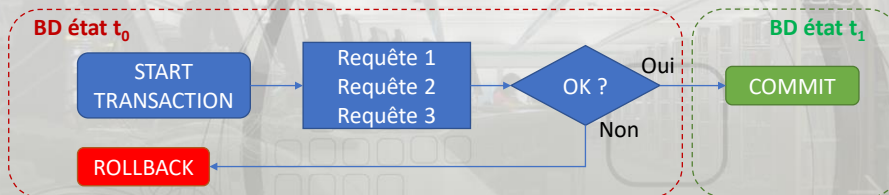
AAAAAAAAAAAAHHHHH !!!

59

59

Les transactions

Les transactions permettent de s'assurer qu'un ensemble de requêtes puisse être correctement traité avant de l'exécuter.



Mécanisme de protection qui :

- permet de maintenir la DB dans un **état cohérent** ;
- permet de garantir l'**ACIDité** des transactions ;
- est indispensable aux utilisations de type OLTP.

60

60

ACIDité d'une transaction

Propriété ACID

Ensemble de propriétés qui garantissent qu'une transaction est exécutée de façon fiable

- Atomicité :** une transaction se fait au complet ou pas du tout
- Cohérence :** assure que chaque transaction amènera le système d'un état valide à un autre état valide
- Isolation :** Toute transaction doit s'exécuter comme si elle était la seule sur le système (pas de dépendance)
- Durabilité :** lorsqu'une transaction a été confirmée, elle demeure enregistrée même à la suite d'une panne

61

61

Charge OLTP

OLTP (Online Transaction Processing) :
(traitement transactionnel en ligne) :

*Contexte d'utilisation typique des activités opérationnelles
(e-commerce, production, système de réservation, etc.)*

Un système OLTP doit :

- Pouvoir répondre à un nombre élevé de transactions (ensemble solidaire de requêtes) par minute.
- Doit être performant tant en lecture qu'en écriture.
- Garantir l'ACIDité des transactions.

62

62

Exemple

SET autocommit = 0;

Certains moteurs comme InnoDB, considèrent chaque requête comme une transaction.
Cette commande permet de désactiver ce mécanisme

try {

START TRANSACTION;

UPDATE compte SET argent = argent - 39.99 WHERE client = 'Vegeta'

UPDATE compte SET argent = argent + 39.99 WHERE client = 'Amazon'

COMMIT;

}

catch(...) {

ROLLBACK;

}

63

63

Exemple en PHP

```
try
{
    $pdo->beginTransaction();

    $pdo->query('SELECT ...');
    $pdo->query('INSERT ...');
    $pdo->query('UPDATE ...');

    $pdo->commit();
}
catch(Exception $e) {

    $pdo->rollback();

    echo 'Erreur : ' . $e->getMessage() . '<br />';
    echo 'N° : ' . $e->getCode();

    exit();
}
```

64

64

Exemple dans une procédure stockée

```
CREATE PROCEDURE insert_plat (IN vnom CHAR(64), IN vprix FLOAT, IN vplat CHAR(64))
BEGIN
    DECLARE p_lastid INT;
    DECLARE t_lastid INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION, SQLWARNING ROLLBACK;

    START TRANSACTION;
    SELECT id_type INTO t_lastid FROM types_plats WHERE nom=vplat;
    IF t_lastid IS NULL THEN
        INSERT INTO types_plats (nom) VALUES (vplat);
        SET t_lastid = LAST_INSERT_ID();
    END IF;

    INSERT INTO plats (id_type,nom,prix) VALUES (t_lastid, vnom, vprix);
    SET p_lastid= LAST_INSERT_ID();
    COMMIT;

    SELECT p_lastid AS 'id', t_lastid AS 'id_type';
END |
```

65

Gestion des accès concurrents

Si 2 transactions concurrentes veulent modifier une même ressource :

- Verrouillage pessimiste :

La ressource est accessible à une seule transaction et les autres sont en attente

Exemple : InnoDB (MySQL/MariaDB) verrouille le tuple
MyISAM (MySQL/MariaDB) verrouille la table
SQLite verrouille la BD complète...

- Verrouillage optimiste :

Les accès peuvent se faire concurremment

Le SGBD conserve différentes versions d'une ressource associées à éventuellement un Timestamp (Timestamp Ordering)

Exemple : PostgreSQL (mécanisme MVCC)

66

66

MVCC en verrouillage optimiste

Multiversion concurrency control (des fois MCC)

Chaque version d'une ressource dispose d'un tag (numéro de transaction ou timestamp)

En verrouillage optimiste, assure qu'une transaction n'aura jamais à attendre la disponibilité d'une ressource en gérant plusieurs versions de chaque objet

Exemple : **PostgreSQL** garde toutes les version d'un même tuple dans la table (même les versions issues de transactions abordées)

Nécessité d'implémenter un mécanisme de "Purge"

Purge automatique (purge lag configurable) et possibilité de forcer manuellement (commande VACUUM en PostgreSQL)

67

67

MVCC en verrouillage optimiste

MVCC n'est pas forcément synonyme de verrouillage optimiste

Exemple avec le moteur **InnoDB** (MySQL/MariaDB) :

Ce moteur est à verrouillage pessimiste au niveau du tuple.

La table ne contient que la dernière version du tuple, les autres sont dans un "undo-log" en utilisant le MVCC

Des threads s'occupent de purger les "undo-log" régulièrement (périodicité configurable)

Pour choisir un SGBD(R) adapté à son contexte, il peut être opportun de prendre en considération ces différents mécanismes de verrouillage



68

68

Transactions -> les logs

Concomitamment aux données, les SGBD(R) sauvegardent toutes les modifications de la BD (DDL et DML) dans des fichiers logs.

Utilité : - Revenir en arrière en cas de problème
- Souvent utilisé pour la réplication

Les mécanismes de log varient en fonction du SGBD

Peut impacter les performances

Peut se paramétrer relativement finement (optimisations)

Avec certains SGBD, possibilité de "loguer" les requêtes de lecture qui génèrent des "timeout"

69

69

Journal transactionnels (exemples)

Avec SQL Server :

- le journal des logs actifs (pas encore *commit*)
- le journal des logs inactif (ceux qui sont *commit*)

MySQL/MariaDB :

- Un bin log configurable en
 - Statement-based : que les requêtes (DDL+DML)
 - Row-based : DDL + modifications pour chaque tuple (séparément)
 - Mixed
- Plus un WAL (Write-Ahead Log) si le moteur est InnoDB

En plus des journaux transactionnels (ci-dessus), les SGBD peuvent ajouter d'autres logs comme les *Undo Log*, *Rollback Log*, etc.

70

70

WAL (Write-Ahead Log)

Les écritures sont d'abord écrites dans le log sur support physique

Les tables sont mises à jour en RAM

Tous les X, une opération de checkpoint met à jour la BD sur le support physique.

En fonction du X (configurable), permet de diminuer le nombre d'opérations d'écriture.

De plus, il est moins coûteux d'écrire une entrée dans le WAL que de modifier une donnée en BD (pas d'index, ni de structure de données complexe)

71

71

Séparer logs et données

Il peut être envisagé d'utiliser des supports de stockage différents pour logs et BD.

Permet d'écrire simultanément sur les 2 supports (perfs)

En cas de perte d'un des 2 supports, on conserve une version (soit la BD, soit les logs)

Permet de gérer plus facilement l'espace libre (les 2 systèmes ne s'impactent pas)

Permet de configurer spécifiquement les supports (filesystem, hardware, ...) :

- Pour les logs, il faut un support rapide pour les écritures séquentielles
- Pour la BD, le support doit être rapide pour des lectures/écritures aléatoires

72

72

Le partitionnement des données

73

73

Le partitionnement

Le fait de **diviser une table (ou BD) en plusieurs morceaux** pour les répartir, le plus souvent, entre différents serveurs. Dans ce cas on parlera de ***sharding***.

Peut aussi se faire sur une seule et même machine. Par exemple : dénormaliser le schéma pour séparer les données fréquemment accédées des données accessoires.

Quand une seule machine n'est plus capable de traiter efficacement les données et/ou pour fournir plus de **scalabilité** -> partitionnement

Les règles de partitionnement peuvent être définies manuellement ou être automatique.

Il est possible de partitionner la BD en **différentes sous-BD isolées** les unes des autres ou déployer **une seule instance répartie entre différents serveurs**.

74

74

Le partitionnement horizontal

Table			
Col1	Col2	Col3	Col4
1	Xxx	Aaa	Fff
2	Yyy	Bbb	Ggg
3	Zzz	Ccc	Hhh
4	www	ddd	iii

Table(P1)			
Col1	Col2	Col3	Col4
1	Xxx	Aaa	Fff
2	Yyy	Bbb	Ggg

Table(P2)			
Col1	Col2	Col3	Col4
3	Zzz	Ccc	Hhh
4	www	ddd	iii

Le partitionnement peut se faire :

- de manière automatique et transparente :

```
if( id%2==1) -> Node 1
else -> Node 2
```

- ou dynamiquement sur base de règles établies au préalable :

```
if( sexe=='m') -> Node 1
else -> Node 2
```

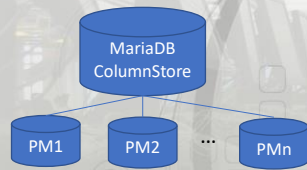
La logique de *partitionnement* peut être implémentée dans le SGDB ou dans une couche logicielle en amont

75

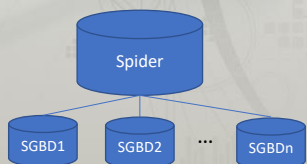
75

Partitionnement horizontal : architectures

Il existe **moult**es façons de concevoir un système de partitionnement horizontal. Voici quelques exemples :



Une seule instance de la BD



Un nœud principal qui distribue les données entre plusieurs SGBDR :

- plusieurs instances ;
- est tout de même capable de joindre les données des différents SGBDR.

76

76

Sharding horizontal



- Le partitionnement horizontal offre la possibilité au système d'interroger les fragments de table simultanément par les différents serveurs :

- gain en rapidité
- les index sont également fragmentés (gain en rapidité d'écriture)

- Load balancing pour les écritures



- Infrastructure plus complexe

- Augmentation de la latence

- Les recherches croisées (jointures) peuvent saturer le canal de communication utilisé entre les serveurs (bottleneck).

Contrairement au "croyances populaires", ces problématiques sont également présentes avec les systèmes NoSQL



77

77

Le partitionnement vertical

Table			
Col1	Col2	Col3	Col4
1	Xxx	Aaa	Fff
2	Yyy	Bbb	Ggg
3	Zzz	Ccc	Hhh
4	www	ddd	iii



Table(P1)	
Col1	Col2
1	Xxx
2	Yyy
3	Zzz
4	www

Table(P2)	
Col3	Col4
Aaa	Fff
Bbb	Ggg
Ccc	Hhh
ddd	iii

Une des idées principales du partitionnement vertical est de **séparer les colonnes souvent utilisées des colonnes rarement accédées** afin de diminuer la taille des tables. Des vues servent alors à reconstituer les tables initiales.

Le partitionnement vertical est plus *"domain specific"*

Également appelé *row splitting*

A ne pas confondre avec les (wide) Column Stores

78

78

Ne pas confondre

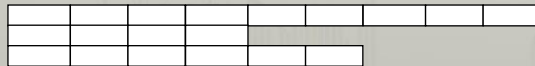
- Partitionnement vertical



- Column Store



- Wide Column Store



79

79

Chapitre 6.9

Historisation des données

80

80

Historisation des données

Garder la trace de l'évolution des données au fil du temps

Pour, par exemple :

- Faire de l'analyse de données
- Rechercher des causes d'incident
- Rejouer des scénarios

Ce procédé peut :

- être **codé manuellement** avec des tables dédiées et des trigger
- profiter du **mécanisme MVCC** qui peut être détourné (et désactiver la purge)
- être basé sur l'analyse des **fichiers logs**
- être **activé nativement avec certains SGBDR/moteurs de stockage**

81

81

System Versioned Tables

Temporal tables

Fonctionnalité proposée avec les dernières versions du moteur InnoDB

```
CREATE TABLE capteurs (  
  id INT PRIMARY KEY,  
  valeur DOUBLE NOT NULL,  
) ENGINE=InnoDB WITH SYSTEM VERSIONING;
```

```
INSERT INTO capteurs VALUES (1, 2.5);  
INSERT INTO capteurs VALUES (2, 7.5);  
UPDATE capteurs SET valeur=2.8 WHERE id=1;  
UPDATE capteurs SET valeur=2.9 WHERE id=1;
```

id_capteur	valeur
1	2.9
2	7.5

```
SELECT * FROM capteurs  
FOR SYSTEM_TIME ALL  
WHERE id=1;
```

id_capteur	valeur
1	2.5
1	2.8
1	2.9

82

82

System Versioned Tables

Avec ce mécanisme, 2 colonnes (cachées) supplémentaires sont accessibles :

- ROW_START
- ROW_END

```
SELECT *, ROW_START, ROW_END FROM capteurs FOR SYSTEM_TIME ALL WHERE id=1;
```

id_capteur	valeur	ROW_START	ROW_END
1	2.5	2018-05-03 07:21:12.386980	2018-05-03 07:22:15.456452
1	2.8	2018-05-03 07:22:15.456452	2018-05-03 07:23:52.002455
1	2.9	2018-05-03 07:23:52.002455	2038-01-09 03:14:07.999999

```
SELECT * FROM capteurs FOR SYSTEM_TIME AS OF TIMESTAMP '2018-05-03 07:23:00'  
WHERE id=1;
```

id_capteur	valeur
1	2.8

83

83

System Versioned Tables

```
SELECT *, ROW_START, ROW_END  
FROM capteurs  
FOR SYSTEM_TIME FROM '2018-05-03 07:23:00' TO NOW()  
WHERE id=1;
```

id_capteur	valeur	ROW_START	ROW_END
1	2.8	2018-05-03 07:22:15.456452	2018-05-03 07:23:52.002455
1	2.9	2018-05-03 07:23:52.002455	2038-01-09 03:14:07.999999

84

84

Réplication et proxy

85

85

Réplication

Répliquer les données sur plusieurs serveurs

Nécessite au minimum 2 serveurs : 1 Master et 1 Slave

Utilité première : avoir une copie de la BD. En cas de crash du serveur principal, le Slave peut alors prendre le relais

Autre utilité : soutenir de fortes charges. Les requêtes d'interrogation peuvent être distribuées sur tous les serveurs plutôt que de solliciter uniquement le Master

Il existe différents types de réplication :

- Réplication asynchrone
- Réplication semi-synchrone
- Réplication synchrone

86

86

Réplication asynchrone

Le type de synchronisation **le plus simple** (et le plus répandu)

Le Master n'attend pas le Slave

Le maître inscrit tous ses événements dans un **journal de log**

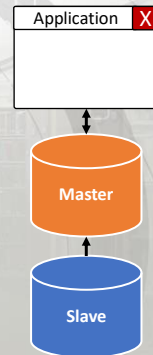
L'esclave consulte ce journal à intervalle régulier

lag

Si le maître crash, il y a une **perte des X dernières transactions** (le lag)

Le ou les **esclaves peuvent être sollicités pour les lectures** (ils sont en lecture seule)

Les réponses des esclaves peuvent être temporellement fausses à cause du lag

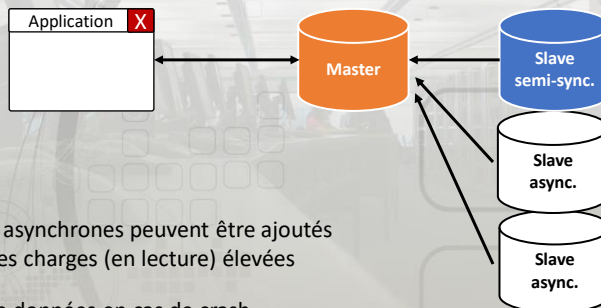


87

87

Réplication semi-synchrone

Le maître ne valide pas la transaction tant qu'au moins un esclave ne l'a pas répliqué



D'autres esclaves asynchrones peuvent être ajoutés pour supporter des charges (en lecture) élevées

Évite les pertes de données en cas de crash

En cas de crash, c'est l'esclave semi-synchrone qui est promulgué maître

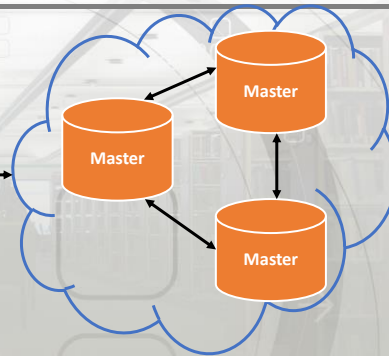
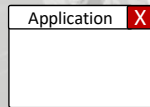
Ajoute une latence aux requêtes

88

88

Réplication synchrone

Une réplication synchrone (multi-master)
nécessite au moins 3 masters (consensus)



L'application peut lire et écrire sur n'importe quel maître

Les transactions sont validées de manière synchrone
sur tous les masters

Pas de temps mort en cas de crash d'un serveur

Pas de perte de données car pas de lag

Génère plus de latence pour les écritures que les autres solutions (croît avec le nombre de nœuds)

L'ensemble du cluster va au rythme du nœud le plus lent (importance d'avoir un bon *load balancing*)

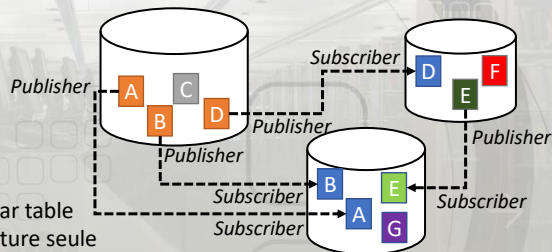
89

89

Réplication logique (transactionnelle)

Avec la réplication logique (PostgreSQL) ou transactionnelle (SQL Server)
ce n'est pas la BD qui est répliquée mais la (ou les) table(s).

Principe de "publish – subscribe"



Par défaut, c'est assez proche
d'une réplication semi-synchrone par table
avec les tables "subscibées" en lecture seule

Possibilité de passer en synchrone où le "subscriber" peut remonter les maj au "producer"

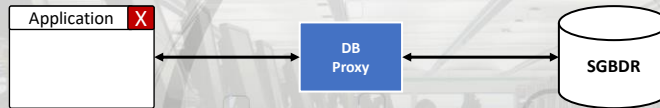
Permet de faire des architectures plus flexibles qu'avec les réplifications conventionnelles

90

90

Proxy de base de données

Un proxy est un logiciel qui gère la connectivité entre 2 partis



L'application cliente est connectée au proxy en pensant que c'est le SGBDR

Le fait que l'application ne se connecte pas directement au SGBDR offre de nombreuses possibilités si on utilise un proxy qui est de type **SQL-aware** :

- Routage du trafic SQL (pour les infrastructures multi-serveurs) et *load balancing*
- Firewall SQL
- Réécriture des requêtes
- Switching Master/Slave automatique
- Meilleure sécurité car il n'y a que le proxy qui peut interroger les SGBDR

91

91

Chapitre 6.11

Quelques définitions

92

92

Quelques définitions

Technologie sémantique :

- Permet d'effectuer des recherches sur base du sens des valeurs et non sur la valeur des valeurs ...
- Ajoute des nouveaux index et fonctionne en complément de l'indexation FULL-Text

Fuzzy Search (recherche approximative) :

- Trouver des chaînes de caractères qui correspondent à un motif approximatif
- Exemple : entrer "algoritme" et le système trouve "algorithme"

Multi-tenant / multi-base :

- Permet d'exécuter des requêtes qui impliquent plusieurs bases de données
- Natif dans SQL Server et avec des performances optimisées
- Non natif mais possible avec Oracle(option \$\$\$) et PostgreSQL
- Natif avec MySQL/MariaDB

La taille des "pages"

Les SGBD découpent les données en fragments : pages

Ce sont les pages qui sont la plus petite unité de la BD qui est lue/écrite sur le disque (et non le tuple)

Même en RAM, les données sont découpées en pages

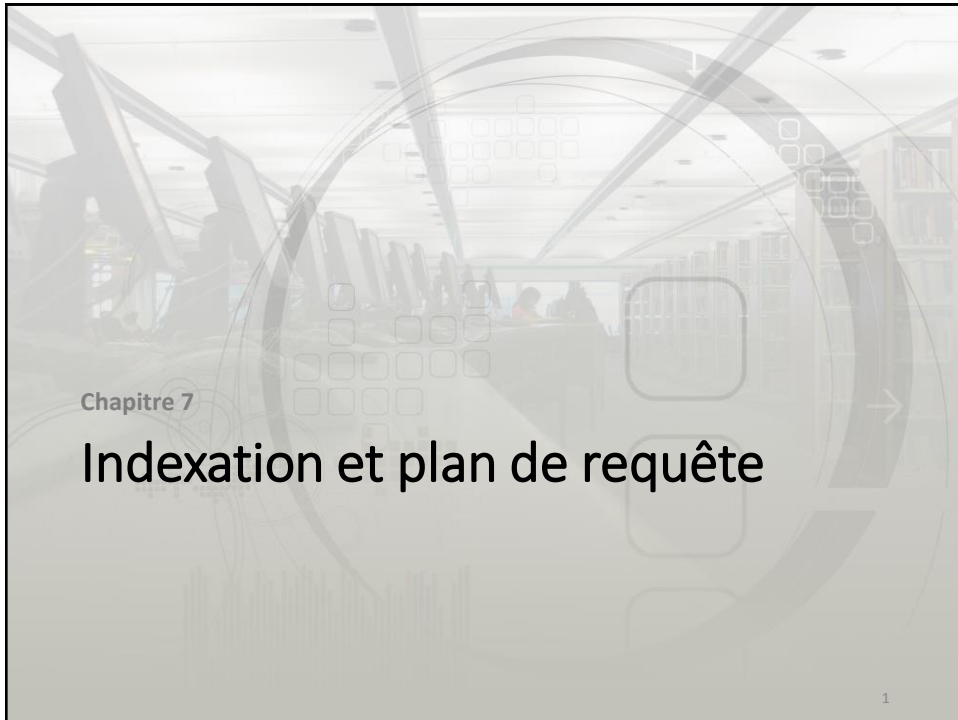
La taille des pages est paramétrable (à la création de la BD)

Conseils :

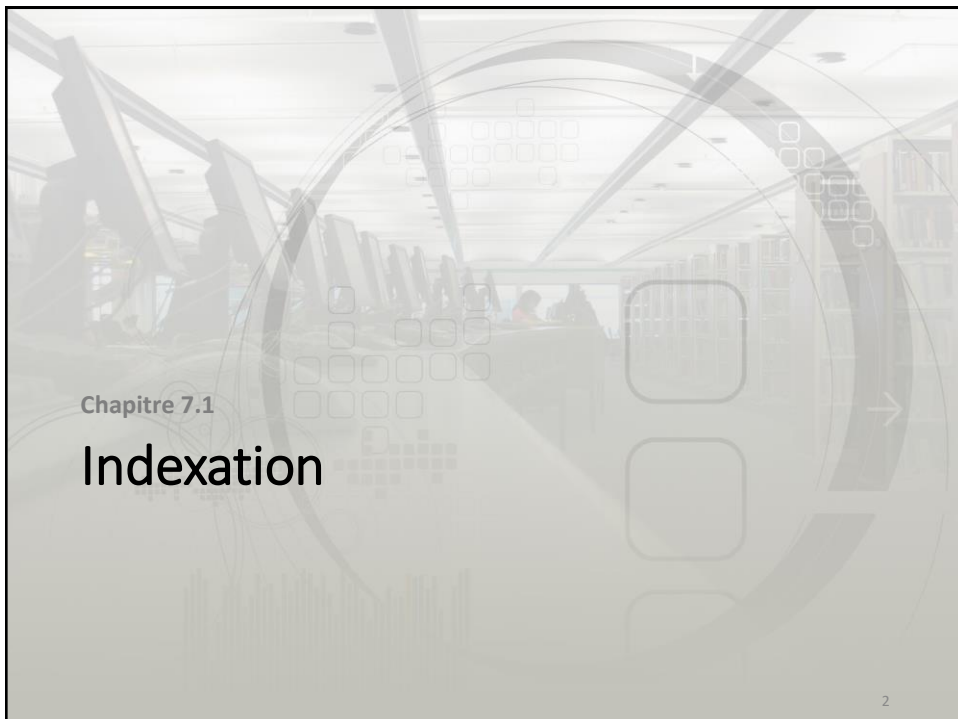
Une bonne pratique est de configurer la **même taille que le système de fichiers** du disque (évite au système des écritures inutiles)

Si la taille des tuples est telle qu'une page ne peut même pas en contenir 4, alors une augmentation de la taille peut améliorer sensiblement les performances (idéalement, doit rester un multiple de *cluster-octet*)

Le calcul de la taille d'un tuple peut varier fortement d'un SGBD à l'autre. Sur certains systèmes, les colonnes TEXT, BLOB, ... sont stockées ailleurs que dans la table (dans des *overflow-pages*)



1



2

Recherche sans index

id	nom	prénom	tel
1	Green	Roby	0471 123 123
2	Blue	Will	0486 123 123
3	Pink	Jen	0470 123 123
4	Red	Sarah	0477 123 123
5	Pink	Elise	0473 123 123
6	White	John	0499 123 123
7	Black	Jack	0494 123 123
...	...	...	...

rep array()

```
foreach (CLIENTS as CLIENT) {
    if (CLIENT[tel]==0494 123 123)
        rep.add (CLIENT[id])
}
```

Complexité $O(n)$

Qui à le numéro
0499 123 123



3

3

Recherche avec index

TELIndex

tel
0470 123 123
0471 123 123
0473 123 123
0477 123 123
0486 123 123
0494 123 123
0499 123 123
...

CLIENTS

id	nom	prénom	tel
1	Green	Roby	0471 123 123
2	Blue	Will	0486 123 123
3	Pink	Jen	0470 123 123
4	Red	Sarah	0477 123 123
5	Pink	Elise	0473 123 123
6	White	John	0499 123 123
7	Black	Jack	0494 123 123
...	...	...	...

Avec un index unique de
type liste triée :

Pire cas : $O(\log_2 n)$

Non-unique :

Pire cas : $\sim O(n)$

En pratique : $\sim O(\log_2 n)$

```
recherche_dichotomique(clé, liste) {
    x= liste.taille() / 2;
    if (liste[x].tel == clé) return CLIENTS[liste[x].id];
    if (liste[x].tel > clé) return recherche_dichotomique(clé, liste[1:x]);
    else return recherche_dichotomique(clé, liste[x:n]);
}
recherche_dichotomique(0499 123 123, TELIndex);
```

4

4

Types d'index

CREATE INDEX nom_index ON la_table (col_a_indexer)

Types :

- **INDEX** : pas unique et peut être NULL
- **PRIMARY KEY** : est un index unique et non NULL
- **UNIQUE** : est un index unique mais les valeurs NULL sont admises
- **SPATIAL** : pour les données géospatiales
- **FULLTEXT** : pour les recherches textuelles avancées (voir plus loin)

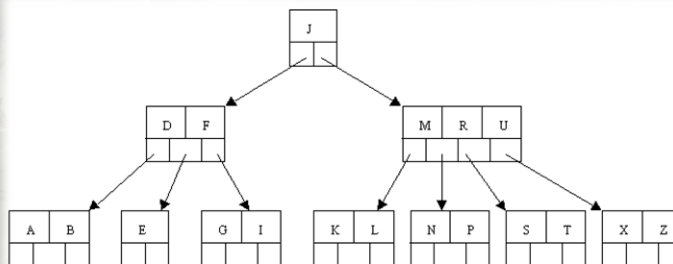
Notez que certains SGBD associent automatiquement un index (non unique et pouvant être NULL) aux FOREIGN KEYS mais ce n'est pas systématique (pensez à vérifier vos index)

5

5

Structure de donnée

La forme générale utilisée pour un index est le B-arbre (B-tree)



Source : <https://cis.stvincent.edu/html/tutorials/swd/btree/btree.html>

Arbre binaire de recherche équilibré avec, dans la pratique,
un **nombre élevé de fils par nœud** (quelques dizaines à plusieurs milliers)

Voir cours de structure de données pour plus d'info.

6

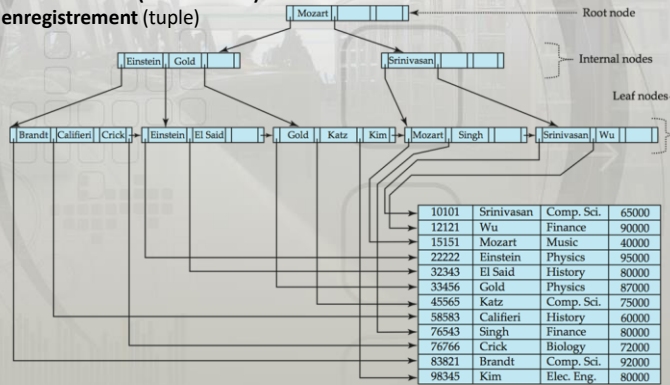
6

Dans la pratique : Arbre B+ (B+ Tree)

Avec les SGBD c'est la variante B+ qui est utilisée :

- Ce sont **des clés** qui sont stockées et non les données
- **Les clés sont répétées** dans les niveaux inférieurs
- Toutes les **clés sont présentes dans les feuilles**
- Toutes les feuilles font partie d'une liste (**doublement**) chaînées
- A une **clé est associé un enregistrement (tuple)**


Heap-Table
(tas, pile, amas)



Source : <https://3.treffpunkt-oc.de/wire/kium-classic-engine-diagram.html>

7

Pourquoi un Arbre B+

- Cette structure a été choisie pour obtenir de "bonnes" performances avec les HDD (disques durs magnétiques -> pas SSD) en essayant de restreindre le nombre d'accès au support

- Un des problèmes majeurs des HDD est la latence (~8ms).

Pour lire 1000 fichiers de 100ko avec un HDD offrant un débit de 100 Mo/s et une latence de 10ms :

$$(1000 * 0,1 \text{ Mo}) / 100 \text{ Mo/s} + \underbrace{1000 * 10 \text{ ms}}_{10 \text{ secondes}} = 11 \text{ secondes}$$

- Ce sont des arbres B+ avec un nombre élevé de clés par nœud qui sont utilisés. Les clés d'un même nœud sont stockées séquentiellement sur le support physique ce qui ne génère qu'une seule latence par nœud.
- Cependant, avoir un seul nœud reviendrait à stocker une liste triée ce qui est peu optimal pour les opération d'écriture. Il est donc nécessaire de trouver un compromis.

8

Pour ou contre l'arbre B+ ?



- De l'ordre de "hauteur" accès disque
- Le nœud racine est souvent dupliqué en RAM (+ éventuellement d'autres nœuds)
- Recherches en $O(\log_2 t \cdot \log_i n) \approx O(\log n)$
- **Une faible hauteur d'arbre suffit dans la plupart des cas**
Exemple : avec $t=1000$ et seulement 3 niveaux, on peut indexer 10^9 tuples
- Il n'est pas toujours nécessaire de stocker des valeurs entières pour les clés intermédiaires (des débuts de chaînes peuvent suffire)



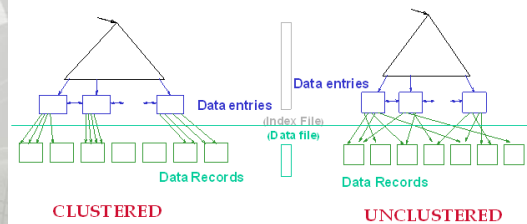
- **Occupent de l'espace...**
- ... surtout si les clés sont de grande dimension (texte)...
- ... dans ce cas -> t diminue et l'arbre est plus haut
- **Maintenir un index a un cout non négligeable** (phase de rééquilibrage)

9

9

Clustered et Unclustered Index

Clustered vs. Unclustered Index



Source : <https://courses.cs.washington.edu/courses/cse544/99sp/lectures/storage/sld025.htm>

- Il ne peut y avoir qu'un seul *clustered index* par table
- À une clé primaire est généralement associé un **index primaire** sous forme de *clustered index*. Tous les autres index (**index secondaires**) de la table sont *unclustered*
- Les *clustered index* :
 - sont performants pour les recherches de type BETWEEN ;
 - permettent de sortir des données déjà triées ;
 - ont un cout de maintenance plus élevé (**préférez des PK artificielles auto-incrémentées**).

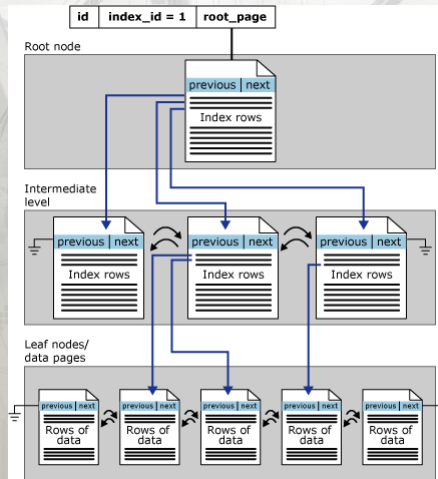


10

10

Exemple

SQL Server utilise par défaut des index clusterisés



Source : <https://docs.microsoft.com/en-us/previous-versions/sql/sql-server-2008-r2/ms177443%28v%3dsql.105%29>

11

11

Index creux et dense

TELindex_dense

tel
0470 123 123
0471 123 123
0473 123 123
0477 123 123
0486 123 123
0494 123 123
0499 123 123
...

CLIENTS

id	nom	prénom	tel
1	Green	Roby	0471 123 123
2	Blue	Will	0486 123 123
3	Pink	Jen	0470 123 123
4	Red	Sarah	0477 123 123
5	Pink	Elise	0473 123 123
6	White	John	0499 123 123
7	Black	Jack	0494 123 123
...	...	...	...

TELindex_creux

tel
0470 123 123
0486 123 123
0499 123 123
...

Un index dense contient toutes les occurrences de clé de la table

Un index creux n'en contient qu'une fraction

Lo!... ça sert à rien ton index creux !

Si, j'ai identifié 2 cas d'utilisation

12

12

Index creux et dense

TELindex_dense

tel
0470 123 123
0471 123 123
0473 123 123
0477 123 123
0486 123 123
0494 123 123
0499 123 123
...

CLIENTS

id	nom	prénom	tel
1	Green	Roby	0471 123 123
2	Blue	Will	0486 123 123
3	Pink	Jen	0470 123 123
4	Red	Sarah	0477 123 123
5	Pink	Elise	0473 123 123
6	White	John	0499 123 123
7	Black	Jack	0494 123 123
...	...	...	...

TELindex_creux

tel
0470 123 123
0486 123 123
0499 123 123
...

Cependant, il est vrai que dans l'exemple ci-dessus, l'index creux...

... même en l'absence de l'index dense... n'a que peu d'intérêt.

Ah ok, c'était juste pour l'exemple alors ?

Yep

13

Utilité des index creux (cas 1)

- **Cas 1 : être utilisé pour une clé primaire**
 - permet d'en réduire la taille
 - diminue (très) sensiblement le cout des mises à jour
 - les recherches sont souvent moins efficaces

PKindex

id
1
4
7
...

CLIENTS

id	nom	prénom	tel
1	Green	Roby	0471 123 123
2	Blue	Will	0486 123 123
3	Pink	Jen	0470 123 123
4	Red	Sarah	0477 123 123
5	Pink	Elise	0473 123 123
6	White	John	0499 123 123
7	Black	Jack	0494 123 123
...	...	...	...

Il ne peut pas y avoir de croisements avec un index creux

Ce qui signifie que dans ce cas, un index creux doit toujours être un *clusterised index* !

14

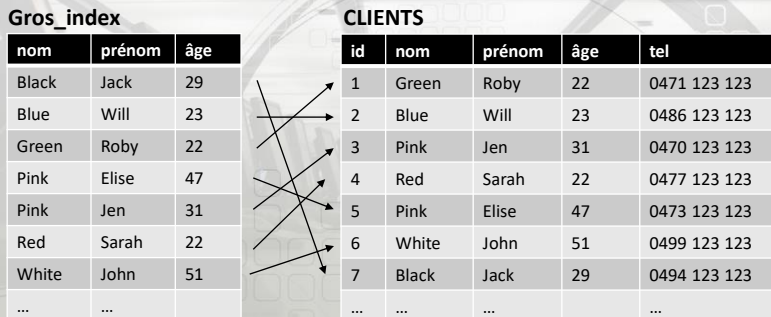
Utilité des index creux (cas 2)

- **Cas 2 : être utilisé comme index d'index**
 - Intéressant quand un index dense devient trop volumineux que pour être stocké en RAM. Dans ce cas, l'ajout d'un index creux pour indexer l'index dense permet d'améliorer la réactivité du système



15

Indexer plusieurs colonnes



- **Occupe plus de place**
- **Dans certain cas, peut permettre de s'abstenir de charger la table.** Si nous ne cherchons que les noms et prénoms des clients, la structure d'indexation est suffisante
- **N'accélèrent que les recherches incluant au moins la première colonne de l'index :**
 - Par [nom], par [nom, prénom], par [nom, prénom, âge] et éventuellement par [nom, âge] (s'il y a beaucoup de noms différents)
 - Si nous cherchons les clients de 22 ans, l'index ci-dessus sera inutile

16

Indexer plusieurs colonnes

Gros_index

nom	prénom	âge
Black	Jack	29
Blue	Will	23
Green	Roby	22
Pink	Elise	47
Pink	Jen	31
Red	Sarah	22
White	John	51
...	...	

Pour accélérer les recherches par âge (sans nom, prénom) il faut alors rajouter un autre index uni-colonne sur âge.

Dans cette situation, de nombreux DBA ont l'automatisme d'ajouter un index "prénom, âge" et un index « âge »

Cela peut être une bonne idée tout comme cela peut être contre-productif. Tout dépend des types de recherches souvent effectuées.

Dans la pratique, il est préférable d'utiliser ce type d'index avec parcimonie.

Il arrive également qu'on n'ait pas le choix -> clés primaires composées de plusieurs colonnes. Dans ce cas, **il est EXTRÊMEMENT important de bien choisir l'ordre des colonnes indexées** (et ne pas oublier de rajouter des index si nécessaire)

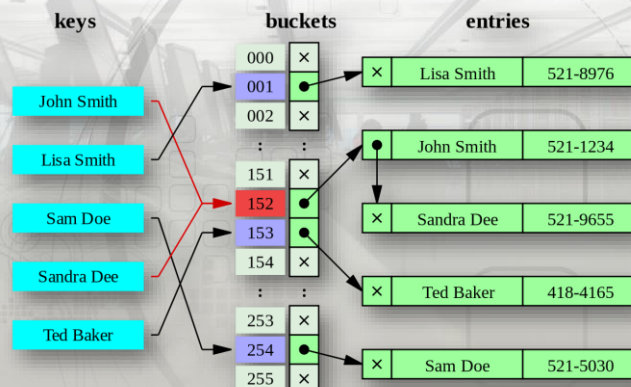
Notez qu'afin de soulager le système, on peut n'indexer que les X premiers caractères d'une chaîne

17

17

Indexation par table de hachage

CREATE HASH INDEX nom_index ON la_table (col_a_indexer)



Source : https://en.wikipedia.org/wiki/Hash_table

Voir cours de structure de données pour plus d'info sur cette structure

18

18

Pour ou contre la table de hachage ?



- Pas toujours disponible
- Ne permet que des équijointures (test d'égalité uniquement)
- Ne produit pas de résultat trié
- Pas adapté pour les jeux de données de petite taille (mais faut-il les indexer ?)
- Gestion des collisions
- Structure de taille fixe (doit se redimensionner/partitionner aux fil du temps, sinon les performances chutent suite aux collisions)



- La puissance de calcul et les capacités mémoire ont bien plus augmenté ces dernières années que les latences n'ont diminué, ce qui rend l'utilisation de ce type de structure **de plus en plus judicieux**
- Maintenance moins couteuse qu'une structure en arbre
- **En pratique, la complexité peut être en $O(1)$** (dépend des données, de la taille de la table et de la fonction utilisée)

19

19

Réflexion sur les tables de hachage

Ne pas utiliser cette structure :

- pour l'indexation multicolonne si toutes les colonnes ne font pas partie de la recherche
- pour indexer des colonnes pour lesquelles vous êtes susceptible d'effectuer des recherches par plage de valeur (typiquement les dates)
- pour indexer des colonnes où vous ne testez pas l'égalité
- Si vous triezy souvent vos données sur base des clés recherchées

Structure d'indexation de prédilection des KV-Store car meilleure structure pour rechercher une occurrence d'une clé.

Si vous avez souvent ce type de requête : `SELECT * FROM table WHERE col='xxx'`
un index de type HASH pourra être plus rapide qu'un index B+Tree
-> pratique pour les clés primaires artificielles

20

20

Index bitmap

CREATE BITMAP INDEX nom_index ON la_table (col_a_indexer)

Index particulier principalement utilisé pour indexer les colonnes qui ont peu de valeurs différentes (énumérations)

CLIENTS

id	nom	prénom	sexe	tel
1	Green	Roby	femme	0471 123 123
2	Blue	Will	homme	0486 123 123
3	Pink	Jen	femme	0470 123 123
4	Red	Sarah	femme	0477 123 123
5	Pink	Elise	femme	0473 123 123
6	White	John	homme	0499 123 123
7	Black	Jack	homme	0494 123 123
...	...	...		...

Index_bitmap

homme	femme
0	1
1	0
0	1
0	1
0	1
1	0
1	0
...	...

L'index bitmap se passe de pointeur, c'est un fichier trié sur la clé contenant autant de bits par records qu'il y a de valeur différentes pour la colonne indexée.

21

21

Pour ou contre l'index BITMAP



- **Occupent peu d'espace** (et peuvent être compressés pour en occuper encore moins)
- **Rapide pour trouver les enregistrements faisant partie d'un sous-ensemble** : `SELECT * FROM clients WHERE sexe='homme'`
- **Efficace avec les opérateurs binaires (OR, AND, XOR, etc.)** : `SELECT * FROM table WHERE couleur='bleu' OR couleur='rouge'`
- Contrairement aux arbres, avec le bitmap **les valeurs NULL sont indexées** (NULL = tous les bits à 0 dans l'index)



- **L'index doit être reconstruit en cas d'ajout d'une nouvelle valeur**
- **Si l'index est compressé, les opérations d'écritures seront ralenties** (convient mieux à l'OLAP qu'à l'OLTP)

22

22

Chapitre 7.2

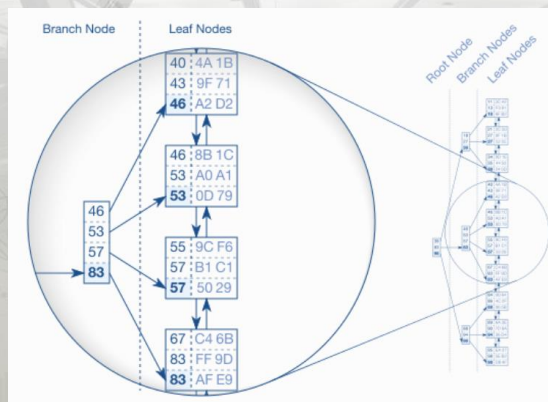
Utilisation de l'index

23

23

Les feuilles de l'index

Les feuilles sont des listes doublement chaînées



<https://use-the-index-luke.com/sql/anatomy/the-tree>

24

24

Principe de la recherche

`SELECT * FROM table WHERE id=42;`

Si l'index est UNIQUE :

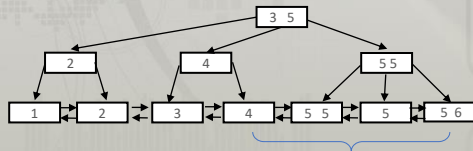
INDEX UNIQUE SCAN*

une traversée de l'arbre

Sinon :

INDEX RANGE SCAN*

une traversée de l'arbre
+
parcourir x feuilles



**Terminologie ORACLE*

25

25

Principe de la recherche (Heap Table)

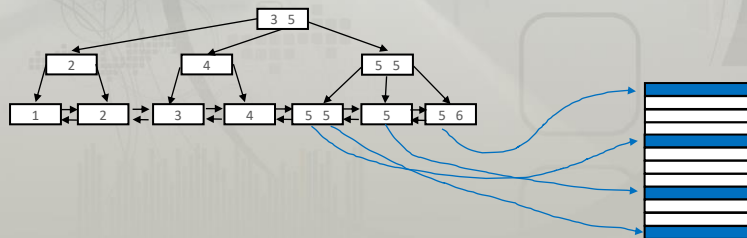
`SELECT * FROM table WHERE id=5;`

Avec une "Heap-Table"

INDEX UNIQUE ou RANGE SCAN*

+

TABLE ACCESS BY INDEX ROWID*



**Terminologie ORACLE*

26

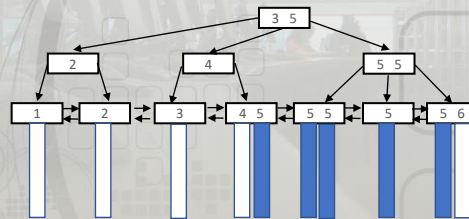
26

Principe de la recherche (Clustered Index)

```
SELECT * FROM table WHERE id=5;
```

Avec un "clustered index"

INDEX UNIQUE ou RANGE SCAN



Uniquement si c'est une recherche sur l'index clusterisé

Dans l'exemple, il y a des doublons dans l'index, ce qui est rarement le cas, car il est plus fréquent d'utiliser la clé primaire comme index clusterisé.

Ce type de structure est efficace pour les recherche de type BETWEEN



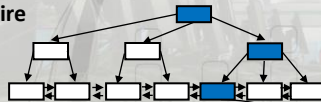
27

27

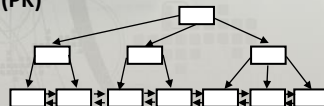
Index secondaire (Heap Table)

```
SELECT * FROM table WHERE email='pink@red.blue';
```

Index secondaire



Index primaire (PK)



**Terminologie ORACLE*

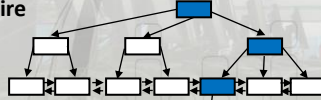
28

28

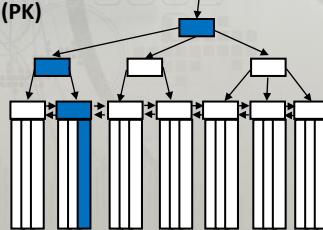
Index secondaire (Clustered Index)

`SELECT * FROM table WHERE email='pink@red.blue';`

Index secondaire



Index primaire (PK)



Avec ce type de structure, les recherches via un index secondaire sont généralement plus lentes, car il y a 2 traversées d'index

Cette structure est par contre à favoriser si :

- Vous modifiez rarement une PK
- ET vous effacez rarement
- ET vous ne faites quasi que des recherches par PK
- ET que les clés s'insèrent naturellement de façon triée

**Terminologie ORACLE*

29

29

Qui fait quoi ?

	Heap-Table	Clustered Index
	MyISAM / Aria	InnoDB
	Par défaut	La clause CLUSTER permet d'agencer la Heap-Table en fonction de l'index lors d'opérations de maintenance
	Avec la clause NONCLUSTERED	Par défaut et par défaut sur la PK mais on peut le faire sur une autre colonne
	Par défaut	La clause ORGANIZATION INDEX et se fera toujours sur la PK
	Par défaut	La clause CLUSTERED INDEX essaye si possible d'insérer les tuples au bon endroit dans la Heap-Table

**Terminologie ORACLE*

30

30

Gros piège (1)

```
SELECT * FROM table WHERE LOWER(email) = LOWER('truc@brol.com');
```

Avec cette requête, l'index email n'est pas utilisé !

Si besoin, il faut donc créer un index dédié :

```
CREATE INDEX lowermail ON table (LOWER(email));
```

Cela peut fonctionner avec des USER DEFINED FUNCTIONS à condition qu'elles soient DETERMINISTIC



Il arrive que les ORM rajoutent implicitement des LOWER() dans les clauses WHERE



SQL Server, ne permet pas ce type d'index, mais la parade est d'indexer une "computed-column"

31

31

Gros piège (2)

ORACLE n'aime pas les NULL :

- Un string vide = NULL ...
- CONCAT('xyz',NULL) = NULL ...
- CREATE INDEX indexname ON table (col1, col2, col3)

Si un tuple est NULL pour col1, col2 et col3, alors il ne sera pas présent dans l'index...

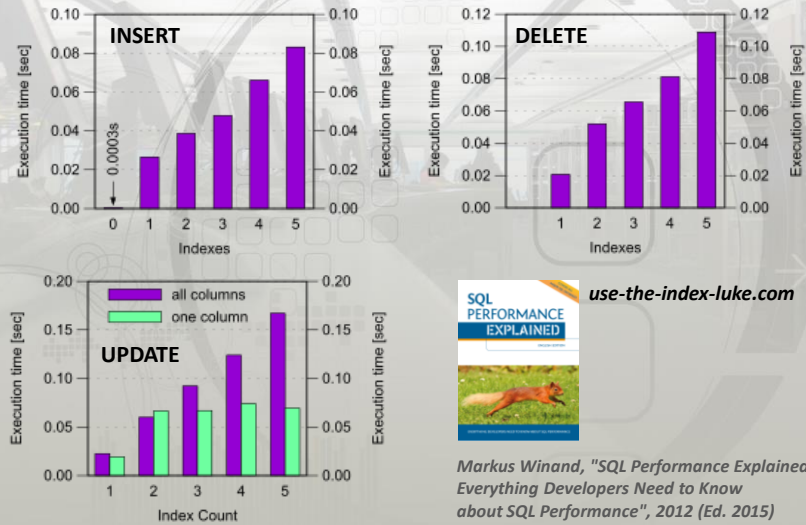
C'est valable aussi s'il n'y a qu'une seule colonne dans l'index

Il faut au moins qu'une colonne de l'index soit de type NOT NULL pour que l'index soit utilisé avec des recherche WHERE col IS NULL

32

32

Attention aux perfs. d'écriture



33

33

Chapitre 7.3

Indexation FULLTEXT

34

34

Limites d'un index "classique"

Ces requêtes seront accélérées si la colonne "col" est indexée :

```
SELECT * FROM table WHERE col='xxx'
```

```
SELECT * FROM table WHERE col LIKE 'xxx%'
```

Par contre, l'index sera **inutile** pour ce type de recherche :

```
SELECT * FROM table WHERE col LIKE '%xxx%'
```

Ce type de recherche s'apparente à une **recherche plein texte (Full-text Search)**

C'est pour accélérer ce type de recherche qu'il existe l'**indexation FULLTEXT**

35

35

Exemple d'index FULLTEXT

Table

id	titre	texte
1	Super ninja	Le super ninja bleu aime la bière
2	Super bière	Les amateurs de bière aiment la bière
3	Les tortues ninja	Entraînées par un rat ninja, les petites tortues sont devenues de grands ninjas
4	Petit deviendra grand	Sois patient petit, un jour tu deviendras grand... ou pas
...	...	...

Le mot "bière" est présent:

- 1x dans le texte de l'id 1
- 1x dans le titre de l'id 2
- 2x dans le texte de l'id 2

```
CREATE FULLTEXT INDEX nom_index  
ON nom_table(titre, texte);
```

Mot	Références
amateurs	2(col:3, occ:1)
aime	1(col:3, occ:1)
aiment	2(col:3, occ:1)
bière	1(col:3, occ:1), 2(col:2, occ:1), 2(col:3, occ:2)
bleu	1(col:3, occ:1)
devenues	3(col:3, occ:1)
deviendra	4(col:2, occ:1)
deviendras	4(col:3, occ:1)
entraînées	3(col:3, occ:1)
grand	4(col:2, occ:1), 4(col:3, occ:1)
grands	3(col:3, occ:1)
jour	4(col:3, occ:1)
ninja	1(col:2, occ:1), 1(col:3, occ:1), 3(col:2, occ:1), 3(col:3, occ:1)
ninjas	3(col:3, occ:1)
...	

36

36

Exemple d'index FULLTEXT

Table

id	titre	texte
1	Super ninja	Le super ninja bleu aime la bière
2	Super bière	Les amateurs de bière aiment la bière
3	Les tortues ninja	Entraînées par un rat ninja, les petites tortues sont devenues de grands ninjas
4	Petit deviendra grand	Sois patient petit, un jour tu deviendras grand... ou pas
...	...	...

Configuration d'un index FULLTEXT :

- Exclusion des mots rencontrés dans plus de X% des tuples (50% par défaut)
- Exclusion des mots courts de moins de XXX caractères
- Exclusion des mots trop communs (requiert un fichier linguistique)

Les mots exclus : Stopwords

Index FULLTEXT(titre, texte)

Mot	Références
amateurs	2(col:3, occ:1)
aime	1(col:3, occ:1)
aiment	2(col:3, occ:1)
bière	1(col:3, occ:1), 2(col:2, occ:1), 2(col:3, occ:2)
bleu	1(col:3, occ:1)
devenues	3(col:3, occ:1)
deviendra	4(col:2, occ:1)
deviendras	4(col:3, occ:1)
entraînées	3(col:3, occ:1)
grand	4(col:2, occ:1), 4(col:3, occ:1)
grands	3(col:3, occ:1)
jour	4(col:3, occ:1)
ninja	1(col:2, occ:1), 1(col:3, occ:1), 3(col:2, occ:1), 3(col:3, occ:1)
ninjas	3(col:3, occ:1)
...	

37

37

La recherche naturelle

Table

id	titre	texte
1	Super ninja	Le super ninja bleu aime la bière
2	Super bière	Les amateurs de bière aiment la bière
3	Les tortues ninja	Entraînées par un rat ninja, les petites tortues sont devenues de grands ninjas
4	Petit deviendra grand	Sois patient petit, un jour tu deviendras grand... ou pas
...	...	...

```
SELECT id FROM nom_table
WHERE MATCH(titre, texte) AGAINST ('bière');
```

Le résultat est trié par score de pertinence :

```
SELECT id, MATCH(titre, texte) AGAINST ('bière')
FROM nom_table
WHERE MATCH(titre, texte) AGAINST ('bière');
```

id	MATCH(titre, texte) AGAINST ('bière')
2	0.2718571722507477
1	0.0906190574169159

Trouve tous les résultats contenant au moins un des mots recherchés :

```
SELECT id, MATCH(titre, texte) AGAINST ('bière ninja super')
FROM nom_table
WHERE MATCH(titre, texte) AGAINST ('bière ninja super');
```

id	MATCH(titre, texte) AGAINST ('bière ninja super')
1	0.45309528708457947
2	0.3624762296676636
3	0.1812381148338318

38

38

La recherche naturelle

Table

id	titre	texte
1	Super ninja	Le super ninja bleu aime la bière
2	Super bière	Les amateurs de bière aiment la bière
3	Les tortues ninja	Entraînées par un rat ninja, les petites tortues sont devenues de grands ninjas
4	Petit deviendra grand	Sois patient petit, un jour tu deviendras grand... ou pas
...	...	...

```
SELECT id, MATCH(titre, texte) AGAINST ('"super ninja" ')
FROM nom_table
WHERE MATCH(titre, texte) AGAINST ('"super ninja" ');
```

id	MATCH(titre, texte) AGAINST (...)
1	0.3624762296676636

```
SELECT id, MATCH(titre, texte) AGAINST ('"ninja super" ')
FROM nom_table
WHERE MATCH(titre, texte) AGAINST ('"ninja super" ');
```

Résultat vide



Attention que dans cet exemple, seul l'**index(titre, texte)** a été créé !
L'utilisation de **MATCH(texte) AGAINST (...)** peut conduire à une erreur en l'absence d'index portant uniquement sur la colonne **texte**.

Idem pour la colonne **titre**

39

39

Calcul de la pertinence

Table

id	titre	texte
1	Super ninja	Le super ninja bleu aime la bière
2	Super bière	Les amateurs de bière aiment la bière
3	Les tortues ninja	Entraînées par un rat ninja, les petites tortues sont devenues de grands ninjas
4	Petit deviendra grand	Soit patient petit, un jour tu deviendras grand... ou pas

```
SELECT id, MATCH(titre, texte) AGAINST ('bière')
FROM nom_table
WHERE MATCH(titre, texte) AGAINST ('bière');
```

Avec le moteur MySQL et le moteur de stockage InnoDB, l'enregistrement 2 obtient le score :

$$3 \times \log (4/2) \times \log (4/2) = 0.2718571$$

3 : nombre d'occurrences du mot dans l'enregistrement

4 : nombre total d'apparitions du mot dans la table

2 : nombre d'enregistrements qui contiennent le mot recherché

40

40

La recherche en mode booléen

Une recherche FULLTEXT en mode booléen :

- ignore la règle des mots fréquents (règle des XX%) ;
- Ne requiert pas d'index avec MyISAM (faut pas être pressé s'il n'y en a pas) ;
- En fonction des versions, il peut ne pas y avoir de score de pertinence (MyISAM)
- peut exiger que des mots soient présents (+) ou absents (-) ;
- peut utiliser * pour remplacer les derniers caractères d'un mot recherché (intéressant pour les pluriels, conjugaisons, etc.).
- En fonction des versions... les spécificités des interclassements peuvent être omises

41

41

La recherche en mode booléen

Table

id	titre	texte
1	Super ninja	Le super ninja bleu aime la bière
2	Super bière	Les amateurs de bière aiment la bière
3	Les tortues ninja	Entraînées par un rat ninja, les petites tortues sont devenues de grands ninjas
4	Petit deviendra grand	Soit patient petit, un jour tu deviendras grand... ou pas
...	...	...

```
SELECT id, MATCH(titre, texte)
  AGAINST ('+bière +ninja' IN BOOLEAN MODE)
FROM nom_table WHERE MATCH(titre, texte)
  AGAINST ('+bière +ninja' IN BOOLEAN MODE);
```

id	MATCH(titre, texte) AGAINST (...)
1	0.2718571722507477

```
SELECT id FROM nom_table
WHERE MATCH(titre, texte)
  AGAINST ('bière ninja' IN BOOLEAN MODE);
```

id
1
2
3

```
SELECT id FROM nom_table
WHERE MATCH(titre, texte)
  AGAINST ('aime*' IN BOOLEAN MODE);
```

id
1
2

42

42

La recherche avec extension de requête

Une recherche FULLTEXT qui se déroule en 2 étapes :

1. Une recherche FULLTEXT naturelle est effectuée
2. Le résultat de cette première recherche est réinjecté pour effectuer une seconde recherche FULLTEXT naturelle

id	titre	texte
1	Super ninja	Le super ninja bleu aime la bière
2	Super bière	Les amateurs de bière aiment la bière
3	Les tortues ninja	Entraînées par un rat ninja, les petites tortues sont devenues de grands ninjas
4	Petit deviendra grand	Soit patient petit, un jour tu deviendras grand... ou pas

```
SELECT id, MATCH(titre, texte)
  AGAINST ('bière' WITH QUERY EXPANSION)
FROM nom_table WHERE MATCH(titre, texte)
  AGAINST ('bière' WITH QUERY EXPANSION);
```

id	MATCH(titre, texte) AGAINST (...)
1	1.178047776222229
2	1.178047776222229
3	0.3624762296676636

43

43

Dialecte FULLTEXT

Les exemples précédents utilisent le dialecte de MySQL/MariaDB

Par exemple, avec MS SQL Server ce sont les instructions CONTAINS, CONTAINSTABLE qui sont utilisées

Il ne faut pas confondre la recherche FULLTEXT et l'indexation FULLTEXT.

Une recherche LIKE '%mot%' peut être considérée comme une recherche FULLTEXT mais aucun index ne sera utilisé (même un FULLTEXT)

L'indexation est un outil puissant, mais **pas suffisamment** que pour constituer à lui seul un **moteur de recherche** digne de ce nom.

Il manque principalement la fonctionnalité de **racinisation (stemming)**

44

44

Racinisation (stemming)

MySQL/MariaDB requiert des outils externes pour apporter cette fonctionnalité (Sphinx, Elasticsearch ou Solr)

Par exemple, avec MS SQL Server, la clause FREETEXT permet la racinisation

Pour la racinisation, des dictionnaires linguistiques sont souvent requis (faut-il encore les trouver... surtout pour nos amis du nord du pays)

Ces dictionnaires ajoutent également la recherche de synonymes

Certains outils ajoutent aussi :

- la correction orthographique
- fuzzy searches
- l'autocomplétion

Actuellement, un SGBDR en configuration "par défaut" ne permet pas de réaliser un "vrai" moteur de recherche



45

45

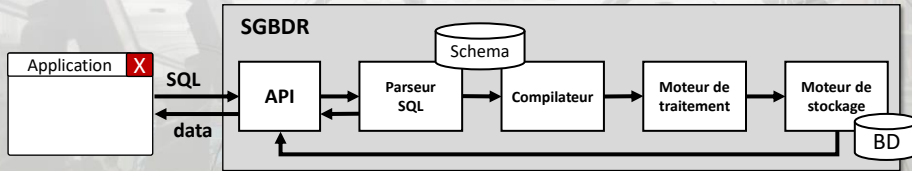
Chapitre 7.4

Plan d'exécution

46

46

Architecture interne d'un SGBDR



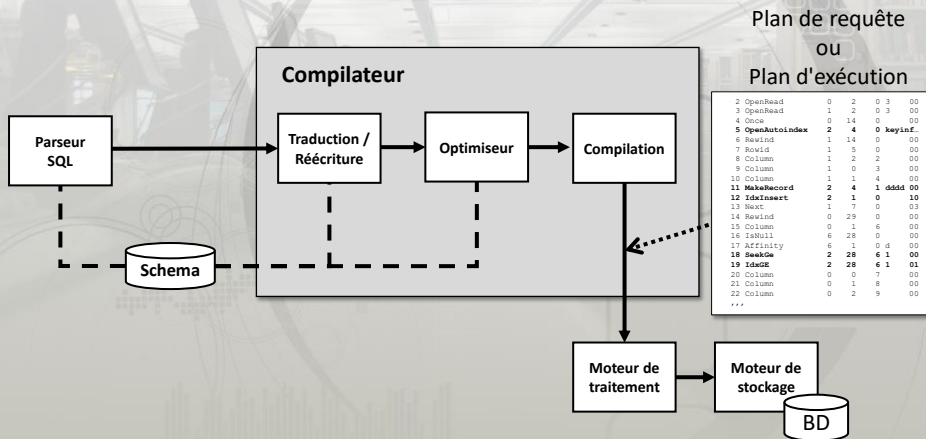
Processus de traitement d'une requête :

1. L'application envoie la requête au travers de l'API
2. La requête va être parsée et vérifiée ← Prend un peu de temps
3. Un plan d'exécution va être généré par le compilateur ← Prend beaucoup de temps
4. Le moteur de traitement va exécuter le plan...
5. ... en accédant aux données via le moteur de stockage
6. Le résultat est renvoyé au client

47

47

Le compilateur SQL



Plan de requête ou Plan d'exécution

2	OpenRead	0	2	0	3	00
3	OpenRead	1	2	0	3	00
4	Once	0	14	0	0	00
5	OpenIndex	2	4	0	keyInL	
6	Rowid	1	14	0	0	00
7	Rowid	1	5	0	0	00
8	Column	1	2	2	0	00
9	Column	1	0	3	0	00
10	Column	1	1	4	0	00
11	MakeRecord	2	4	1	0000	00
12	IdInsert	2	1	0	0	10
13	Next	1	7	0	0	03
14	Rowid	0	29	0	0	00
15	Column	0	1	6	0	00
16	IsNull	6	28	0	0	00
17	Affinity	6	1	0	0	00
18	Scan	2	28	6	1	00
19	IdIn	2	28	6	1	01
20	Column	0	0	7	0	00
21	Column	0	1	8	0	00
22	Column	0	2	9	0	00
...						

48

48

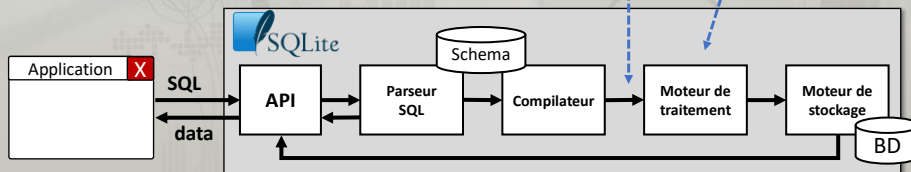
Exemple avec SQLite

Plan d'exécution

2	OpenRead	0	2	0	3	00
3	OpenRead	1	2	0	3	00
4	Once	0	14	0	0	05
5	OpenAutoIndex	2	4	0	keyId	00
6	Rewind	1	14	0	00	
7	RowId	1	9	0	00	
8	Column	1	2	2	00	
9	Column	1	0	3	00	
10	Column	1	1	4	00	
11	MakeRecord	2	4	1	dddd	00
12	IdelNext	2	1	0	10	
13	Next	1	7	0	03	
14	Rewind	0	29	0	00	
15	Column	0	1	6	00	
16	IsNull	6	28	0	00	
17	Affinity	6	1	10	00	
18	SeekOn	2	28	6	1	00
19	IdelNext	2	28	6	1	01
20	Column	0	0	7	00	
21	Column	0	1	8	00	
22	Column	0	2	9	00	
...						

VDBE
Virtual DataBase Engine

Liste d'opcodes exécutés par le VDBE



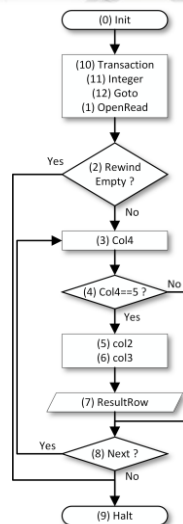
49

49

Exemple avec SQLite (sans index)

SELECT col2, col3 FROM table1 WHERE col4=5

Num.	Opcode	Paramètres				
		P1	P2	P3	P4	P5
0	Init	0	10	0		
1	OpenRead	0	3	0	4	
2	Rewind	0	9	0		
3	Column	0	4	1		
4	Ne	2	8	1	(BINARY)	54
5	Column	0	2	3		
6	Column	0	3	4		
7	ResultRow	3	2	0		
8	Next	0	3	0		1
9	Halt	0	0	0		
10	Transaction	0	0	2	0	1
11	Integer	5	2	0		
12	Goto	5	1	0		



Col4
n'est pas
indexée :

$O(n)$

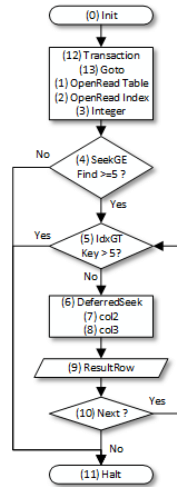
50

50

Exemple avec SQLite (avec index)

SELECT col2, col3 FROM table1 WHERE col4=5

Num.	Opcode	Paramètres				
		P1	P2	P3	P4	P5
0	Init	0	12	0		
1	OpenRead	0	3	0	4	
2	OpenRead	1	4	0	k(2,,)	
3	Integer	5	1	0		
4	SeekGE	1	11	1	1	
5	IdxGT	1	11	1		
6	DeferredSeek	1	0	0		
7	Column	0	2	2		
8	Column	0	3	3		
9	ResultRow	2	2	0		
10	Next	1	5	0		
11	Halt	0	0	0		
12	Transaction	0	0	3	0	1
13	Goto	0	1	0		



Col4 est indexée

Pire cas :
 $O(n + \log n)$
 $O(n)$

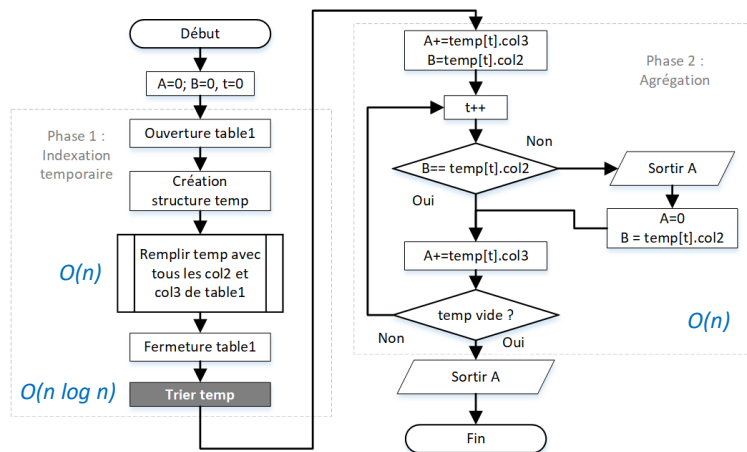
Cas moyen :
 $O(\log n)$

51

51

GROUP BY (avec index temporaire)

SELECT SUM(col3) FROM table1 GROUP BY col2



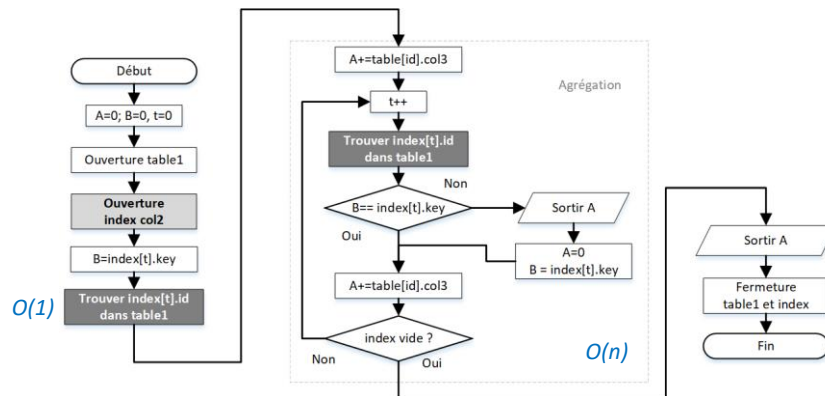
Opération de type SORT GROUP, une alternative serait un HASH GROUP

52

52

GROUP BY (avec index)

SELECT SUM(col3) FROM table1 GROUP BY col2



Plan de requête de type SORT GROUP -> structure d'indexation triée (B+ Tree)

53

53

Jointure (sans index)

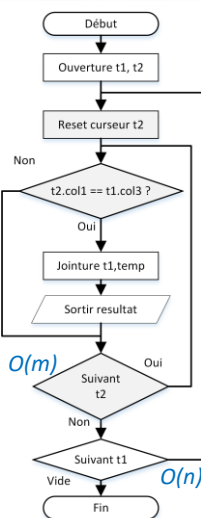
SELECT * FROM t1 JOIN t2 ON t1.col3=t2.col1;

Sans utilisation d'index, une jointure est implémentée sous forme de produit cartésien

$O(n \cdot m)$
 $O(n^2)$

Dans la pratique, cette stratégie d'exécution sera rarement appliquée par le SGBD.

En l'absence d'index le SGBD prendra souvent la décision de créer une structure d'indexation temporaire afin de réduire le nombre d'étapes

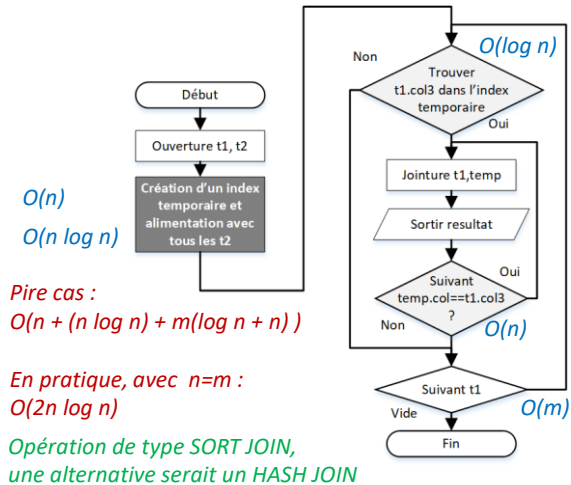


54

54

Jointure (avec index temporaire)

SELECT * FROM t1 JOIN t2 ON t1.col3=t2.col1;



55

55

Jointure (avec index)

SELECT * FROM t1 JOIN t2 ON t1.col3=t2.col1;

Implémentation de type SORT JOIN

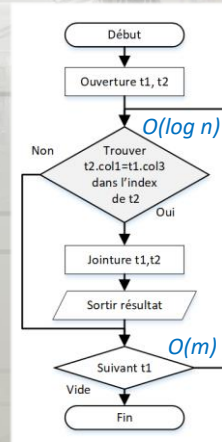
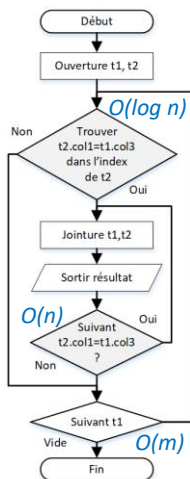
Structure d'indexation triée
 Index **non unique**

Pire cas :
 $O(m(\log n + n))$

En pratique, avec $n=m$:
 $O(n \log n)$

Structure d'indexation triée
 Index **unique**

Toujours : $O(m \log n)$



56

56

La clause EXPLAIN

Pas besoin d'analyser le plan de requête en détail pour faire de l'optimisation de requête.

Il y a la clause

- **EXPLAIN** (MySQL, MariaDB, PostgreSQL)
- **EXPLAIN PLAN** (Oracle)
- **SET SHOWPLAN ALL** (SQL Server)

57

57

EXPLAIN (exemple)

COMPOSANTS

id_composant

nom
description
photo
annee_sortie
id_marque

MARQUES

id_marque
nom_marque

```
EXPLAIN SELECT c.nom,c.description,c.photo FROM composants AS c
WHERE c.annee_sortie>2010 AND c.id_marque =
(SELECT m.id_marque FROM marques AS m WHERE m.nom_marque='Intel')
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	c	NULL	ref	id_marque	id_marque	4	const	42	33.33	Using where
2	SUBQUERY	m	NULL	ALL	NULL	NULL	NULL	NULL	106	10.00	Using where

Sans index sur nom_marque

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	c	NULL	ref	id_marque	id_marque	4	const	42	33.33	Using where
2	SUBQUERY	m	NULL	const	nom_marque	nom_marque	402	const	1	100.00	Using index

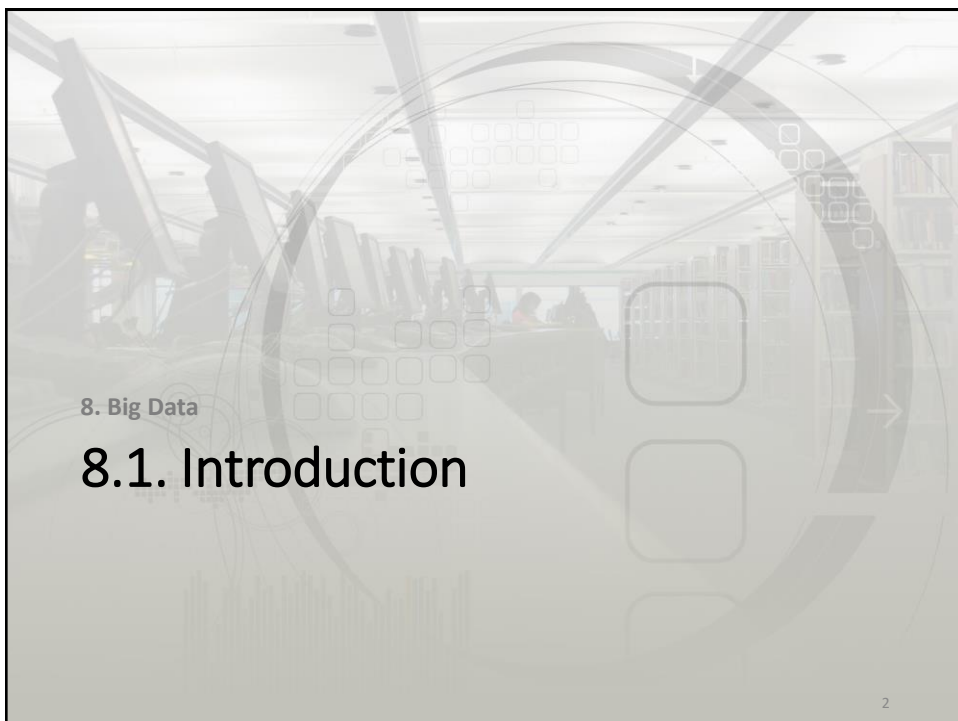
Avec index sur nom_marque

58

58

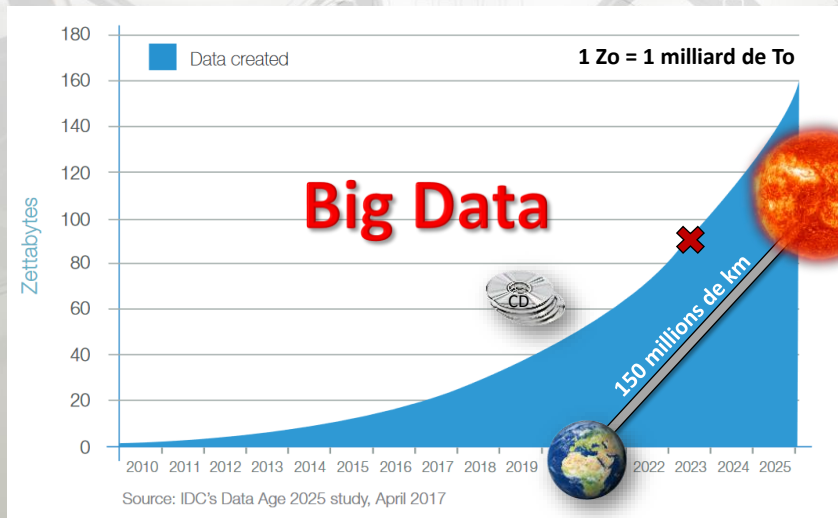


1



2

Croissance de la quantité de données créées par année



3

3

Le Big Data



4

4

La règle des 3 V

- **Volume**

Le critère principal du Big Data : la quantité de données à traiter. Avec les moyens actuels, de nombreux "experts" estiment que les volumes "type" **Big Data sont ≥ 1 To**

- **Vélocité**

Ici 2 axes :

- le rythme élevé auquel les données sont générées
- le besoin d'analyser mes données en "temps réel"

- **Variété**

Les sources sont diverses et variées et les données brutes sont souvent déstructurées

5

5

Big Data : définition

"Big Data refers to datasets whose size is beyond the ability of typical database software tools to capture, store, manage and analyse."

McKinsey Global Institute

Le Big Data, c'est quand les SGBDR ne sont plus capables d'effectuer les traitements

De nombreux "experts" font du "Big Data" avec des jeux de données de moins d'un To. Pourtant, les SGBDR modernes sont capables de traiter plusieurs To

Pour certains "experts", il est plus facile de succomber à la tentation des outils Big Data plutôt qu'à la remise en question de ses propres compétences en matière de SGBDR

6

6

Data Mining et Big DATA

Data Mining (exploration des données) :

- Existait bien avant le Big Data
- Extraction de connaissance à partir d'une grande quantité de **données structurées** :
 - algorithmique
 - statistiques
 - intelligence artificielle

Big Data :

- **Tenter** de résoudre les problèmes de stockage des données tout en fournissant un accès rapide.
- Offre la possibilité aux "développeurs" de **tenter** de structurer les données extraites pour éventuellement faire du Data Mining

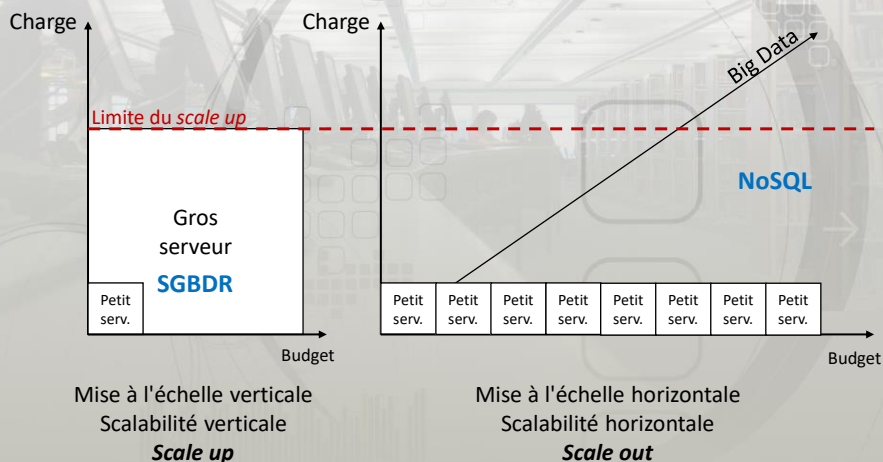
Certaines personnes pensent faire du Big Data mais ils font en fait du "simple" Data Mining

7

7

Scalabilité

Pouvoir adapter un système à la montée en charge en lui ajoutant des ressources



8

8

Bases de données NoSQL

Not only SQL

À l'origine, destinées à manipuler des bases de données géantes (Google, Amazon.com, Facebook ou eBay)

Renoncent aux fonctionnalités classiques des SGBDR au profit de la simplicité :

- Pas de schéma et données non structurées
- Pas d'ACIDité complète

Les performances "restent bonnes" avec la montée en charge (**scalabilité**) en multipliant le nombre de serveurs

Divers paradigmes :

- Key-Value Stores
- Document-Stores (évolution des BD-XML)
- Wide Column Stores
- Graph DBMS
- Etc.

9

9

8. Big Data

8.2. Key-Value Stores

10

10

Key-Value Store (magasin clé-valeur)

Paradigme le plus simple

Peut être vu comme un simple tableau associatif clé-valeur (voir plusieurs tableaux)

- Au sein d'un tableau, **une clé est unique**.
- **La valeur associée à une clé peut être de tout type** : un *int*, une liste de *float*, 3 textes et un entier, 17 images, etc.

La clé est indexée en utilisant généralement une **table de hachage**
(en l'absence de collision, accès en $O(1)$)

Utilisations les plus fréquentes :

- Comme **SGBD NoSQL**
- Comme **serveur de cache**
- Dans des **systèmes embarqués** (simplicité de mise en œuvre et compacité)

Il est possible de **détourner le fonctionnement de certain SGBDR** pour les utiliser en KV-Store

11

11

Key-Value Store : principe



<https://aws.amazon.com/fr/nosql/key-value/>

Amazon DynamoDB peut également être utilisé en mode Document-Store

12

12

KV Store et recherches (1)

Contenu d'un KV-Store

user_id	jeu	score	date
1	Pacman	2400	02/04/18
1	Tetris	1345	03/05/17
2	Doom	1023456	17/07/18
2	Tetris	1455	20/09/18
2	Pacman	2345	06/05/17
3	Pacman	3240	07/05/17
3	Tetris	333	14/02/16
3	Donkey Kong	17895	21/08/18
3	Doom	990320	30/09/18
4	Doom	560333	01/01/19
...	...	...	...

Index sur les jeux

jeu	score	user_id
Donkey Kong	17895	3
Doom	1023456	2
Doom	990320	3
Doom	560333	4
Pacman	2400	1
Pacman	2345	2
Pacman	3240	3
Tetris	1345	1
Tetris	1455	2
Tetris	333	3
...	...	...

Comment récupérer les scores pour le jeu Doom ?

En l'absence d'index, il faut lire toutes les données ...

... mais... ajouter un index... ne serait-ce pas réintroduire le concept de relation ?

13

13

KV Store et recherches (2)

KV-Stores conventionnels : la clé est indexée en utilisant une table de hachage :

- En l'absence de collision, accès en $O(1)$
- La clé doit être totalement connue (et non partiellement)
- Les résultats ne sont pas naturellement triés

Ordered KV-Stores : la clé est indexée en utilisant par exemple un arbre B+ :

- Accès en $O(\log n)$
- La clé être partiellement connue
- Les résultats peuvent être naturellement triés

Il est possible de **détourner le fonctionnement de certain SGBDR** pour les utiliser en KV-Store

- PostgreSQL peut également être utilisé de la sorte avec le module Hstore
- Avec InnoDB, il est possible d'accéder à un enregistrement sans passer par SQL si on connaît la clé primaire.

14

14

KV Store comme système de cache

```
$cle = sha1('client#433');

$reponse_cache= $memcached->get($cle);

if($reponse_cache) $reponse=$reponse_cache;
else
{
    $r=$db->exec("select * from clients where id=$cle");
    $reponse=$r->fetch(PDO::FETCH_ASSOC);
    $memcached->set($cle, $reponse, MEMCACHE_COMPRESSED, 600);
    // 600 secondes en cache
}

foreach($reponse as $col => $valeur )
{
    echo $col.' : '.$valeur.'<br>';
}
```

App
Server



15

15

Les Key-Value Stores

Les plus connus :

- **Redis :**
 - Écrit en C (open source)
 - In-Memory
 - Heuristique HyperLogLogs pour les statistiques
 - Transactionnel
 - Tuto : try.redis.io
- **Memcached**
 - Principalement utilisé comme système de cache
 - Écrit en C (open source)
 - In-Memory
- **Amazon DynamoDB**
 - Est également un Document Store
 - Système propriétaire
- **Aerospike**
 - Écrit en C (open source)
 - In-Memory + persistance sur SSD

<https://db-engines.com/en/ranking/key-value+store>

Rank			DBMS	Database Model	Score		
May 2021	Apr 2021	May 2020			May 2021	Apr 2021	May 2020
1.	1.	1.	Redis	Key-value, Multi-model	162.17	+6.28	+18.69
2.	2.	2.	Amazon DynamoDB	Multi-model	70.07	-0.66	+5.35
3.	3.	3.	Microsoft Azure Cosmos DB	Multi-model	34.71	+1.19	+4.03
4.	4.	4.	Memcached	Key-value	24.50	-0.52	+0.57
5.	6.	6.	etcd	Key-value	9.42	+0.20	+2.14
6.	5.	5.	Hazelcast	Key-value, Multi-model	9.18	-0.09	+0.51
7.	7.	7.	Ehcache	Key-value	7.23	-0.18	+1.00
8.	8.	8.	Aerospike	Key-value, Multi-model	4.92	-0.41	-1.14
9.	9.	9.	Riak KV	Key-value	4.58	-0.50	-0.18
10.	11.	11.	Ignite	Multi-model	4.39	+0.02	-0.17

16

16

Témoignage (Redis)

Redis Cluster is a distributed implementation of Redis with the following goals

- High performance and linear scalability up to 1000 nodes
- Acceptable degree of write safety
- Availability

Disadvantages :

1. Requires at least 3 Masters with 2 slaves each to setup architecture.
2. Since Data is sharded based on the hash-slots assigned to each Master. If Master holding some slots is down, data to be written to that slot will be lost.
3. Clients connecting to Redis cluster should be aware of the cluster topology, causing overhead configuration on Clients.
4. Failover does not happen unless master have atleast one slave.
5. Slave promotion as a Master takes atleast 30–50 seconds, so data written to cluster in that time window will be lost.

<https://www.quora.com/What-are-the-disadvantages-of-Redis>

C'est juste un avis
qui date de 2016



17

17

Limitation des KV-Stores

En général, un KV-Store se préoccupe peu de la structure de la donnée (valeur) qui est référencée par la clé

Si uniquement une partie de cette donnée est requise/modifiée, c'est souvent l'intégralité de la donnée qui devra l'être :

- Plus de trafic réseau
- Plus de données à lire/écrire

Équivalent à un "SELECT * FROM" où seul quelques champs étaient en réalité nécessaires

Moins problématique pour un serveur de cache

18

18

8. Big Data

8.3. Document Stores

19

19

Document Store

SGBD orienté Documents

Peut être vu comme l'**évolution des bases de données XML**, sauf que la grande majorité des systèmes préfèrent le JSON à l'XML

- JSON est moins verbeux que XML: à contenu identique, un JSON occupe moins de place qu'un XML.
- Cette caractéristique permet également un traitement plus rapide.

```
<exempleXML>
  <titre>Mes supers albums</titre>
  <album>
    <artiste>Rammstein</artiste>
    <titre>Mutter</titre>
    <annee>2001</annee>
  </album>
  <album annee="2017">
    <artiste>Ultra Vomit</artiste>
    <titre>Panzer Surprise !</titre>
  </album>
</exempleXML>
```

269 octets

```
{ "exempleJSON" :
  {
    titre : "Mes supers albums",
    album : [{
      "artiste" : "Rammstein",
      "titre" : "Mutter",
      "annee" : 2001,
    }, {
      "artiste" : "Ultra Vomit",
      "titre" : "Panzer Surprise !",
      "annee" : 2017
    }
  ]
}
```

247 octets

20

20

Document Store

L'idée première -> un document est auto-suffisant :

- Son contenu est (semi)structuré
- Il n'est pas nécessaire de croiser ses données avec d'autres documents (pas de jointures)
- Une transaction se résume à l'écriture d'un document

Sur le principe, **un document store est assez proche d'un Key-Value store :**

- **à une clé unique est lié un document** (ce qui peut très bien être implémenté dans un KV-Store)
- Par contre, pour KV-Store, les valeurs stockées sont opaques pour le serveur, alors qu'**avec un Document Store, le serveur peut analyser (requêter) le contenu des documents.**

En fonction des systèmes, il est possible d'ajouter aux documents :

- **des métadonnées ;**
- **des index** sur certaines balises présentes dans les documents.

21

21

Document Store : bien ou pas bien ?



- **Scalabilité** (si adéquatement exploité)
- **Très adaptés à la POO** : les objets peuvent être sérialisés sous forme de document
- Les différents documents ne sont pas tenus d'être identiquement structurés ce qui offre une grande **flexibilité et simplicité**
- Pour afficher, par exemple, les caractéristiques d'un article, **on charge un seul fichier** plutôt que d'exécuter des jointures sur différentes tables

La jointure est un des grands ennemis de la scalabilité en lecture



- Introduit potentiellement une **#@!?! de redondance des données**
- **Les performances peuvent être catastrophiques** pour certains cas d'utilisation. Par exemple, quand il faut croiser les données d'une multitude de documents



22

22

Les Document Stores

Les plus connus :

- **MongoDB:**
 - Écrit en C++ (open source)
 - Support d'une version allégée du SQL pour les lectures
 - Connait un grand succès
- **Amazon DynamoDB**
 - Initialement Document Store, il est maintenant plus proche du Key-Value store
 - Système propriétaire
- **Couchbase**
 - Écrit en C, C++, Go et Erlang (open source)
 - Peut fonctionner en KV-Store
 - Réputé pour être plus performant que Mongo
www.datanami.com/2019/01/16/benchmarking-nosql-databases/
- **CouchDB**
 - Écrit en Erlang (open source)
 - Fondation Apache
 - Solution actuellement moins complète que les précédentes

<https://db-engines.com/en/ranking/document+store>

Rank	DBMS		Database Model	Score	
	Apr 2021	May 2020		May 2021	Apr 2021
1.	1.	MongoDB	Document, Multi-model	481.01	+11.04
2.	2.	Amazon DynamoDB	Multi-model	70.07	-0.66
3.	3.	Microsoft Azure Cosmos DB	Multi-model	34.71	+1.19
4.	4.	Couchbase	Document, Multi-model	30.23	-0.53
5.	6.	Firebase Realtime Database	Document	16.91	-0.02
6.	5.	CouchDB	Document, Multi-model	15.97	-0.04
8.	8.	Realm	Document	9.79	+0.66
7.	7.	MarkLogic	Multi-model	9.52	-0.40
9.	9.	Google Cloud Firestore	Document	6.96	+0.26
10.	10.	ArangoDB	Multi-model	4.38	-0.39

23

23

Document Store : Témoignage

<http://www.sarahmei.com/blog/2013/11/11/why-you-should-never-use-mongodb>

Hey t'es qui toi ?

Je suis Sarah Mei

... et ?

... fondatrice de RailsBridge (offre des formations Web/POO) et architecte pour Salesforce et j'ai un Master en ...

Oui oui.. dis-moi plutôt ce que tu fais dans mon PWP

Ha... je voulais juste dire que j'aime pas MongoDB

...

24

24

Document Store : Témoignage

<http://www.sarahmei.com/blog/2013/11/11/why-you-should-never-use-mongodb>

25

25

Document Store : Témoignage

<http://www.sarahmei.com/blog/2013/11/11/why-you-should-never-use-mongodb>

C'est cool MongoDB !
Un seul fichier pour charger toute sa page.

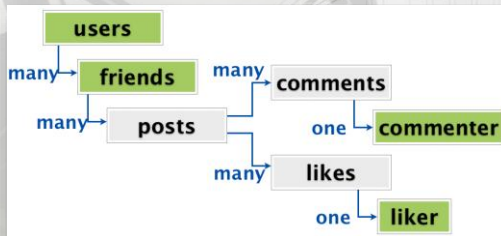
Oui c'est ce qu'on pensait...
... enfin au début

26

26

Document Store : Témoignage

<http://www.sarahmei.com/blog/2013/11/11/why-you-should-never-use-mongodb>



Problème :

Friends, commenter et liker sont également des users

```
{
  name: Joe,
  url: '...',
  stream:
  [{
    user: {name: Jane, url: '...'},
    title: 'today',
    body: 'go fly a kite',
    likes: [
      {user: {name: Lu, url: '...'}},
      {user: {name: Joe, url: '...'}}
    ]
  }
]}
```

Solution 1 :

duplication des contenus dans les contenus...

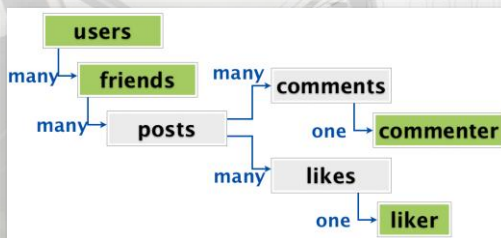
A chaque écriture... faut modifier presque tous les fichiers si on veut garder la cohérence des données...

27

27

Document Store : Témoignage

<http://www.sarahmei.com/blog/2013/11/11/why-you-should-never-use-mongodb>



Problème :

Friends, commenter et liker sont également des users

```
{
  name: Joe,
  url: '...',
  stream:
  [{
    user: {name: Jane, url: '...'},
    title: 'today',
    body: 'go fly a kite',
    likes: [
      {user: {name: Lu, url: '...'}},
      {user: {name: Joe, url: '...'}}
    ]
  }
]}
```

Solution 2 :

Faire des jointures de JSON

... et il faut le coder manuellement

28

28

Document Store : Témoignage

<http://www.sarahmei.com/blog/2013/11/11/why-you-should-never-use-mongodb>



On a eu que des problèmes d'incohérence de données...
...
plus la lourdeur du truc
...

Et au final vous avez fait quoi ?

On a perdu 1 mois à créer un code juste pour le transfert de données vers MySQL

Fallait pas se gourer au début ! Au moins pour un catalogue de films MongoDB c'est bien

Non, je ne trouve pas

29

29

Document Store : Témoignage

<http://www.sarahmei.com/blog/2013/11/11/why-you-should-never-use-mongodb>



```
graph TD
    tv_shows -- many --> seasons
    tv_shows -- many --> episodes
    seasons -- many --> reviews
    episodes -- many --> reviews
    episodes -- many --> cast_members
```

Tu fais comment pour lister la filmographie d'un acteur ?

Tu scans toute la DB..., et il y a moyen d'indexer un champ aussi

Et si deux acteurs ont le même nom ?

Tu te pends

Ou tu passes à un SGBDR

30

30

Document Store : Témoignage

<http://www.sarahmei.com/blog/2013/11/11/why-you-should-never-use-mongodb>

I've heard many people talk about dropping MongoDB in to their web application as a replacement for MySQL or PostgreSQL.

There are no circumstances under which that is a good idea.

Schema flexibility sounds like a great idea, but the only time it's actually useful is when the structure of your data has no value.



Je serais moins catégorique que Sarah.



Pour moi, il est possible de préférer un Document Store à un SGBDR classique si :

- Votre schéma ressemble à un arbre
- Vous avez "masse" données
- La redondance ne vous fait pas peur
- Et surtout... que vous n'ayez pas trop, voir pas du tout, de transactions inter-objets

31

31

Document Store : Témoignage

<http://www.sarahmei.com/blog/2013/11/11/why-you-should-never-use-mongodb>

D'après moi, et d'un point de vue plus général, un document store est **complémentaire** à un SGBDR.

Imaginez un site de petites annonces : vous stockez toutes les catégories, comptes clients et transactions dans un SGBDR et les descriptifs des annonces (textes, photos, mise en page, etc.) dans un Document Store.

En dehors de l'auteur et des catégories, les annonces n'ont aucun liens entre-elles, ce qui convient très bien à un Document-Store

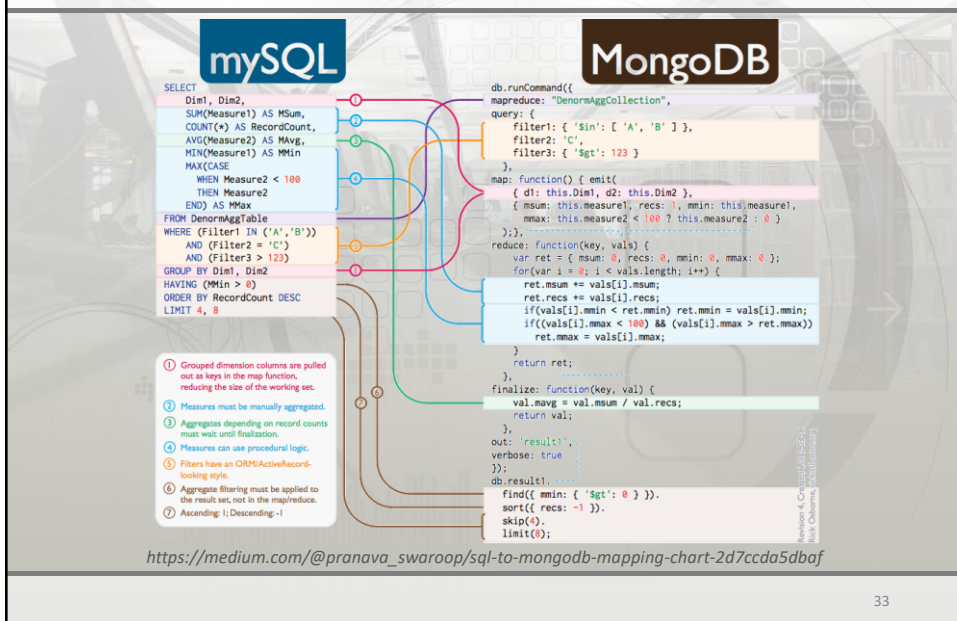
Notez que vous pouvez aussi utiliser les SGBDR en mode "Document Store". Les SGBDR modernes ont rajouté à leur dialecte SQL des fonctions de parsing d'XML et de JSON



32

32

Dur dur le SQL...



33

MongoDB témoignage 2

(2015) Suite à des soucis techniques : décision du passage

d'une solution MongoDB "on premise" à AWS pour la BD de "The Guardian"

Choix d'AWS PostgreSQL plutôt que AWS MongoDB (ou DynamoDB)

MongoDB Ops Manager -> requiert trop de temps de développement (et pas aidé...)

A l'époque (2018), DynamoDB n'offrait pas certains mécanismes de chiffrement

PostgreSQL peut être utilisé en Document Store (jsonb)

~2,3 millions de publications

Ils ont eu 10 mois d'avance sur le planning...

Another great thing about Postgres is how mature it is: every question we wanted to ask had in most cases already been answered on Stack Overflow. — [Philip McMahon](#) et al. in [The Guardian](#)

<https://betterprogramming.pub/why-the-guardian-switched-from-mongodb-to-postgresql-861b6cf01e1f>
<https://www.theguardian.com/info/2018/nov/30/bye-bye-mongo-hello-postgres>

34

34

8. Big Data

8.4. Wide Column Stores

35

35

Wide Column Store

Aussi appelé Extensible Record Store

Ne pas confondre avec les "Column-oriented DBMS" comme MariaDB ColumnStore

Wide Column Store le plus connu :



Table CLIENTS avec un SGBDR :

uuid	nom	prenom	age
Xyz...	Black	Sofie	22
ZhT...	Green	Mike	17
MTR...	Blue	Gary	43
Q5x...	Pink	Amanda	31

Un Wide Column Store peut se voir comme un **Key-Value Store en 2D** :

- La clé (uuid dans l'exemple)
- Une seconde clé (le nom de colonne)

Table CLIENTS avec un Cassandra :

Xyz...	nom	prenom	age
	Black	Sofie	22
ZhT...	nom	prenom	age
	Green	Mike	17
MTR...	nom	prenom	age
	Blue	Gary	43
Q5x...	nom	prenom	age
	Pink	Amanda	31

36

36

Wide Column Store

Si nous ajoutons une colonne à nos systèmes :

Table CLIENTS avec un SGBDR :

uuid	nom	prenom	age	Permis
Xyz...	Black	Sofie	22	B
ZhT...	Green	Mike	17	
MTR...	Blue	Gary	43	A, B
Q5x...	Pink	Amanda	31	B

Avec le SGBDR, il faut réallouer de l'espace

pour toutes les lignes :

- Occupe de l'espace inutile
- Peut prendre du temps

Table CLIENTS avec un Cassandra :

Xyz...	nom	prenom	age	Permis
	Black	Sofie	22	B
ZhT...	nom	prenom	age	
	Green	Mike	17	
MTR...	nom	prenom	age	Permis
	Blue	Gary	43	A, B
Q5x...	nom	prenom	age	Permis
	Pink	Amanda	31	B

Avec Cassandra, comme chaque ligne contient les noms des colonnes, il ne faut ajouter la nouvelle colonne qu'aux lignes qui en ont besoin :

- le schéma est dynamique (jusqu'à + de 2 milliards de colonnes)
- les noms des colonnes sont indexées

37

37

Wide Column Store : colonne composite

Prenons un exemple de schéma relationnel pour un système de blogs :

id_blog	nom_blog (unique)
1	Le blog de Kevin
2	Ariste Metal
3	Dj YoloSwag2000

id_post	id_blog	date	titre	texte
1	1	10/02/19	...	...
2	2	11/02/19	...	...
3	2	12/02/19	...	...
4	3	14/02/19	...	...
5	1	16/02/19	...	...

Principe de conversion Cassandra :

Partition key : "Le blog de Kevin"

Clustering column : date

Data column name : titre et texte

Data column value : ...

Colonne composite

38

38

Wide Column Store : colonne composite

id_blog	nom_blog (unique)	id_post	id_blog	date	titre	texte
1	Le blog de Kevin	1	1	10/02/19	...	...
2	Ariste Metal	2	2	11/02/19	...	...
3	Dj YoloSwag2000	3	3	12/02/19	...	...
		4	3	14/02/19	...	...
		5	1	16/02/19	...	...

Clustering column

Data column name

Partition key	{10/02/19, "titre"}	{10/02/19, "texte"}	{16/02/19, "titre"}	{16/02/19, "texte"}
Le blog de Kevin	...	...	...	...
Ariste Metal	{11/02/19, "titre"}	{11/02/19, "texte"}		
	...	...		
Dj YoloSwag2000	{12/02/19, "titre"}	{12/02/19, "texte"}	{14/02/19, "titre"}	{14/02/19, "texte"}
	...	...	...	...

Une ligne = Wide Row Structure

39

39

Ne pas confondre

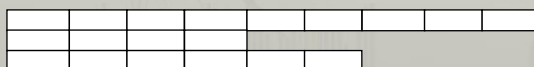
- Partitionnement vertical



- Column Store



- Wide Column Store



40

40

Wide Column Store

C'est le système **Google BigTable** qui est à l'origine de ce paradigme

Le plus connu : **Cassandra**

- toutes les données d'une ligne (Wide Row) sont stockées sur un même support physique, ce qui évite de morceler les données sur différents nœuds
- Les lignes sont dupliquées sur plusieurs nœuds (généralement 3)
- N'a pas été conçu pour un *use-case* spécifique
- Souvent utilisé pour :
 - Schéma-free tout en permettant une certaine structure
 - Scalabilité horizontale de premier plan
 - Shared-nothing (Cassandra)
 - Performances en écritures
 - Gérer des BD de plusieurs Po
- Il est souvent mis en avant que les "join" ne sont plus un problème car l'idée est de dupliquer les données (pré-join) pour ne pas faire de jointure...
- Le filtrage (Where) peut être "délicat" car les tuples ont leurs propres schémas

41

41

Wide Column Store

DBMS	Database Model	Score May 2021
Cassandra	Wide column	110.93
HBase	Wide column	43.24
Microsoft Azure Cosmos DB	Multi-model	34.71
Datastax Enterprise	Wide column, Multi-model	7.55
Microsoft Azure Table Storage	Wide column	4.29
Accumulo	Wide column	3.88
Google Cloud Bigtable	Wide column	3.11
ScyllaDB	Multi-model	2.14
HPE Ezmeral Data Fabric	Multi-model	0.86
Elassandra	Wide column, Multi-model	0.56

<https://db-engines.com/en/ranking/wide+column+store>

Apache Cassandra :

- écrit en Java et open source ;
- c'est le "fileSystem" qui est utilisé ;

Apache HBase :

- écrit en Java et open source ;
- utilise le HDFS (Hadoop Distributed File System)

Apache Accumulo :

- créé par la NASA et légué à Apache
- écrit en Java + quelques portions en C++
- système plus complexe, plus *secure* et plus rapide

ScyllaDB :

- *outsider* intéressant écrit en C++
- Discord est passé de Cassandra à ScyllaDB pour des raisons de performances

42

42

8. Big Data

8.5. Graph DBMS

43

43

Graph DBMS

Base de données orientée graphe

Est une base de données orientée objet dans laquelle les objets sont interconnectés entre eux pour former un graphe

Use-case :

Document Store
Wide Column Store
Key-Value Store

SGBDR

Graph DBMS

Relationnel

Concept :

Un nœud = un tuple

Les nœuds sont typés

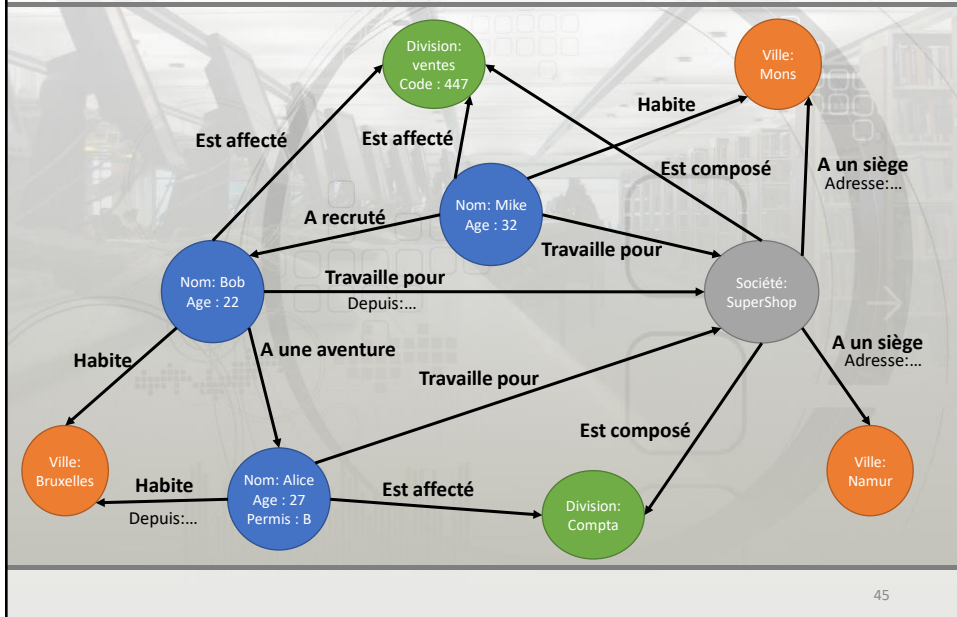
La structure (nombre de colonnes -> propriétés) d'un nœud lui est propre

Les relations sont typées et peuvent contenir des propriétés (propres à chaque relation)

44

44

Graph DBMS : exemple



Graph DBMS : spécificités

- Le graphe est parcouru via des pointeurs physiques
- Les algorithmes de théorie des graphes sont directement applicables (plus court chemin, arbre recouvrant, etc.)
- Sans index, l'accès direct à un nœud sur base de ses valeurs d'attribut n'est pas possible, par contre, à partir d'un nœud, le graphe est parcouru sans nécessiter d'index.
- Il est possible d'indexer les nœuds, les relations, voire même les propriétés ce qui offre les mêmes avantages et inconvénients qu'avec les SGBDR
- Il est conseillé d'élaborer la structure du graphe en fonction des requêtes nécessaires et non l'inverse (**idem pour les autres paradigmes NoSQL**)
- Modélisation plus naturelle des données (comme de la POO sans design patterns ☺)

Graph DBMS : les plus connus

Neo4J :

- écrit en Java et open source ;
- leader des Graph DBMS
- langage de navigation **Cypher**
- Transactionnel
- La version non payante ne tourne que sur un seul nœud

JanusGraph

- écrit en Java et open source ;
- Optimisé pour les clusters
- Fonctionnalités plus limitées

Rank	Rank		DBMS	Database Model	Score	
	Apr 2021	May 2020			May 2021	Apr 2021
1.	1.		Neo4J	Graph	52.23	+1.19
2.	2.		Microsoft Azure Cosmos DB	Multi-model	34.71	+1.19
3.	3.		ArangoDB	Multi-model	4.38	-0.39
4.	4.		OrientDB	Multi-model	4.19	-0.04
5.	5.		Virtuoso	Multi-model	3.44	+0.27
6.		8.	GraphDB	Multi-model	2.25	+0.01
7.		7.	JanusGraph	Graph	2.17	+0.07
8.		6.	Amazon Neptune	Multi-model	1.94	+0.04
	10.		TigerGraph	Graph	1.74	+0.17
	9.	10.	Stardog	Multi-model	1.71	+0.07

<https://db-engines.com/en/ranking/graph+dbms>

De nombreux systèmes implémentent des fonctionnalités "Graph" comme **Cosmos DB**, **Datasax** ou **ArangoDB**

Des plugins/moteurs de stockage orientés graphe existent pour les SGBDR comme le moteur **QQGRAPH** pour MariaDB.

47

Les 4 "top" paradigmes NoSQL

KV-Store

Clé	Valeur
-----	--------

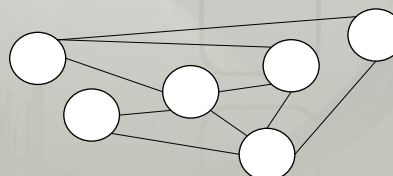
Document Store

Clé	Document
-----	----------

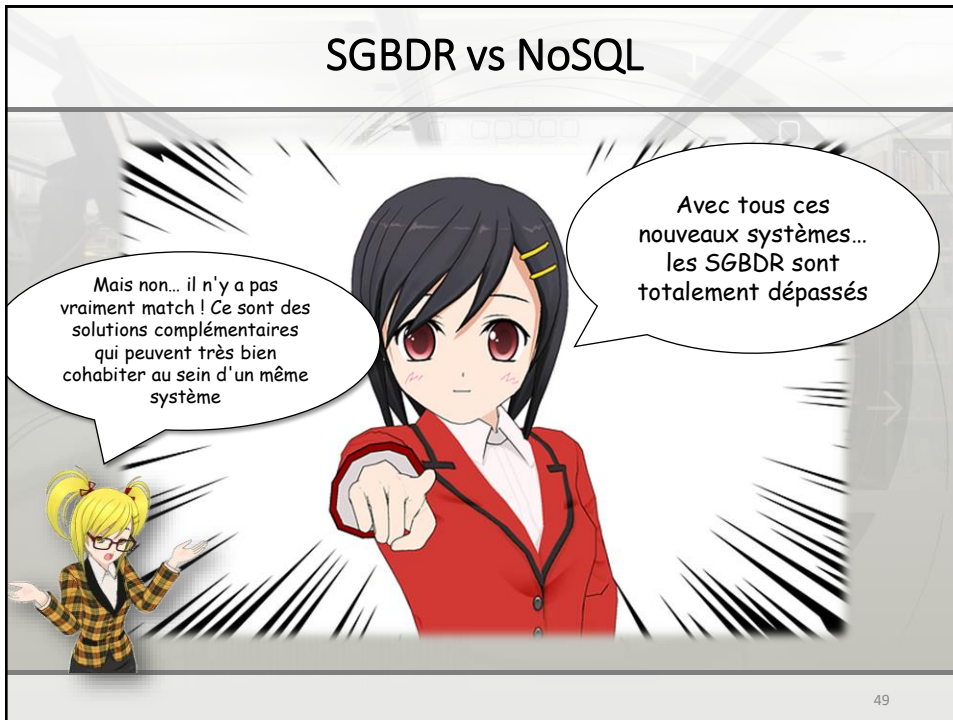
Wide Column Store

Clé	Col1	Col2	Col3	Col4	Col5
	Val1	Val2	Val3	Val4	Val5

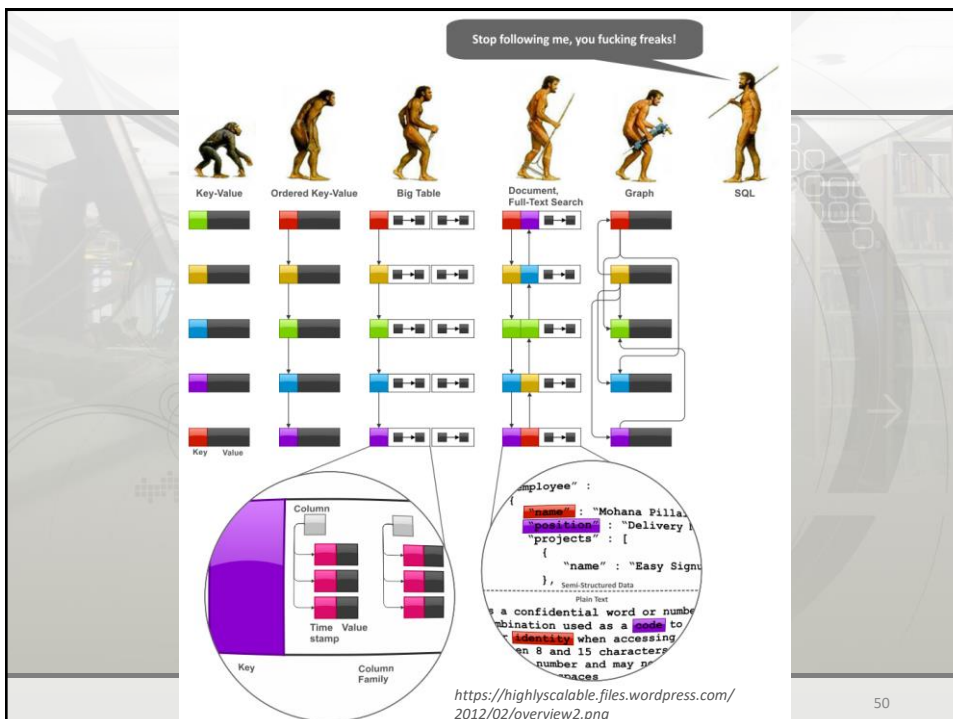
Graph-DB



48



49



50

Quelques Bench's

- <https://www.arangodb.com/2018/02/nosql-performance-benchmark-2018-mongodb-postgresql-orientdb-neo4j-arangodb/>
- https://www.researchgate.net/publication/303485422_Benchmarking_Top_NoSQL_Databases
- <https://www.datanami.com/2019/01/16/benchmarking-nosql-databases/>



N'oubliez pas de toujours avoir un regard critique sur les résultats des benchmarks

Les classements peuvent totalement s'inverser en fonction des types de tests et/ou des jeux de données

51

51

8. Big Data

8.6. Systèmes NoSQL plus spécifiques

52

52

Search Engines

Les moteurs de recherche sont classés dans la catégorie des SGBD NoSQL

Sont souvent utilisés en renfort pour se charger des requêtes de recherche complexe afin de décharger les serveurs plus critiques

De plus, les Search Engines offrent des fonctionnalités qui ne sont que (trop souvent) partiellement implémentées dans les autres systèmes (voir même absentes) :

- La prise en charge des recherches par expressions régulières
- L'indexation FULLTEXT
- Recherche géospatiale
- **Le support de la racinisation (Stemming)**
- La correction orthographique
- L'autocomplétion
- La recherche floue (fuzzy searches)

53

53

Search Engines

Elasticsearch :

- Écrit en Java (open source)
- Peut être utilisé en Document Store
- Basé sur Apache Lucene
- Également fort utilisé pour la fouille de logs

Splunk

- Produit propriétaire payant
- Plus axé analyse de données auto-générées (sécurité, IoT)
- Fait également du *reporting*

Apache Solr

- Écrit en Java (open source)
- Basé sur Apache Lucene
- En perte de vitesse au profit de Elasticsearch qui est plus complet

Sphinx

- Écrit en C++ (open source)
- Connu pour son intégration avec MySQL/MariaDB

May 2020	DBMS	Database Model	S May 2021
1.	Elasticsearch	Search engine, Multi-model	155.35
2.	Splunk	Search engine	92.11
3.	Solr	Search engine, Multi-model	51.19
4.	MarkLogic	Multi-model	9.52
8.	Algolia	Search engine	7.72
6.	Sphinx	Search engine	7.58
5.	Microsoft Azure Search	Search engine	6.05
7.	ArangoDB	Multi-model	4.38
11.	Virtuoso	Multi-model	3.44
10.	Amazon CloudSearch	Search engine	2.20

<https://db-engines.com/en/ranking/search+engine>

54

54

RDF Stores (1)

Ressource Description Framework

Également appelé Triplestore ou Named Graph

Les RDF stores ne stockent que des triplets :

sujet	prédicat	objet
_:a	foaf:name	"Alice Dufour"
_:a	foaf:mbox	"alice.dufour@yfaitchaud.be"
_:b	foaf:mbox	"bob.special@hotmail.com"
_:c	foaf:name	"Caroline Dujardin"
_:c	foaf:mbox	"greencaro@gmail.com"

Langage d'interrogation SPARQL :

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?name ?mbox
WHERE
{
  ?x      foaf:name      ?name
  ?x      foaf:mbox      ?mbox }
```

name	mbox
"Alice Dufour"	"alice.dufour@yfaitchaud.be"
"Caroline Dujardin"	"greencaro@gmail.com"

55

55

RDF Stores (2)

Champ d'application clé = le web sémantique

Exemple : *Till Lindemann est né à Leipzig le 4 janvier 1963*

Traduction en triplestore :

sujet	prédicat	objet
https://www.wikidata.org/wiki/Q154852	https://schema.org/itemType	https://schema.org/Person
https://www.wikidata.org/wiki/Q154852	https://schema.org/givenName	"Till"
https://www.wikidata.org/wiki/Q154852	https://schema.org/familyName	"Lindemann"
https://www.wikidata.org/wiki/Q154852	https://schema.org/birthDate	1963-01-04
https://www.wikidata.org/wiki/Q154852	https://schema.org/birthPlace	https://www.wikidata.org/wiki/Q2079
https://www.wikidata.org/wiki/Q2079	https://schema.org/itemType	https://schema.org/Place
https://www.wikidata.org/wiki/Q2079	https://schema.org/name	"Leipzig"

56

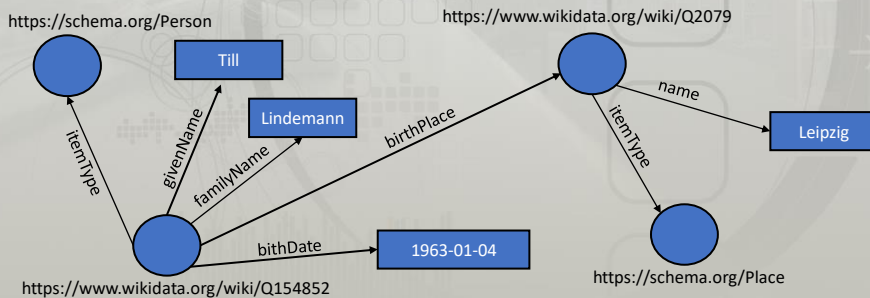
56

RDF Stores (3) – format RDFa

RDFa pour le web sémantique :

```
<div vocab="http://schema.org/" typeof="Person" href="https://www.wikidata.org/wiki/Q154852">
  <span property="givenName">Till</span> <span property="familyName">Lindemann</span> est né à
  <span property="birthPlace" typeof="Place" href="https://www.wikidata.org/wiki/Q2079">
    <span property="name">Leipzig</span>.
  </span>
  <span property="birthDate" content="1963-01-04"> le 4 janvier 1963</span>.
</div>
```

Graphe sémantique (named graph) :



57

57

Time Series

TSDB : Time Series Database

Un des paradigme NoSQL les plus récents

Aux entrées est systématiquement associé un horodatage

Conçu pour traiter des séries de données chronologiques. *Use-case* le plus courant = relevés de capteurs en IoT

Exemple de requête : **SELECT CAPTEUR_TENSION * CAPTEUR_COURANT**

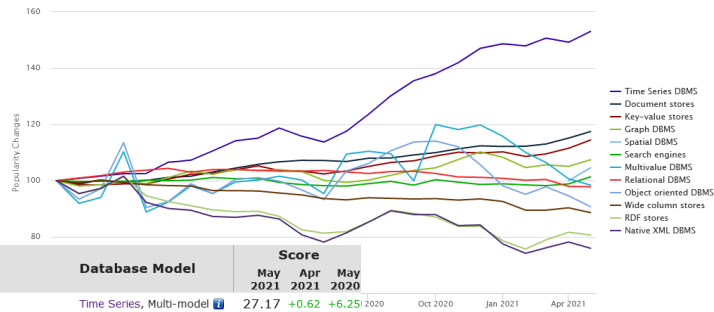
Cette requête va multiplier les valeurs de tension et de courant pour toute une plage de durée commune aux deux capteurs. Ce qui permettra de dessiner un beau graphique dans un second temps.

58

58

Time Series

Trend of the last 24 months



Rank			DBMS	Database Model	Score		
May 2021	Apr 2021	May 2020			May 2021	Apr 2021	May 2020
1.	1.	1.	InfluxDB	Time Series, Multi-model	27.17	+0.62	+6.25
2.	2.	2.	Kdb+	Time Series, Multi-model	8.26	+0.41	+2.89
3.	3.	3.	Prometheus	Time Series	5.76	+0.03	+1.45
4.	4.	4.	Graphite	Time Series	4.56	+0.03	+1.10
5.	5.	8.	TimescaleDB	Time Series, Multi-model	2.90	+0.13	+1.13
6.	7.	6.	Apache Druid	Multi-model	2.67	+0.04	+0.76
7.	6.	5.	RRDtool	Time Series	2.46	-0.25	-0.06
8.	8.	7.	OpenTSDB	Time Series	1.80	+0.05	-0.01
9.	9.	9.	Fauna	Multi-model	1.49	-0.03	+0.54
10.	10.	11.	GridDB	Time Series, Multi-model	1.03	+0.05	+0.59

https://db-engines.com/en/ranking_categories/time+series+dbms

https://db-engines.com/en/ranking_categories

59

59

Systèmes multiparadigmes

De nombreux systèmes sont multiparadigmes :

- Les **SGBDR natifs** qui ajoutent 1 ou plusieurs moteurs de stockage NoSQL
- Les **SGBD NoSQL** qui au départ étaient **mono-paradigme** et qui, au fil du temps, ont ajouté des moteurs de stockage additionnels
- Les **SGBD NoSQL nativement multiparadigmes**

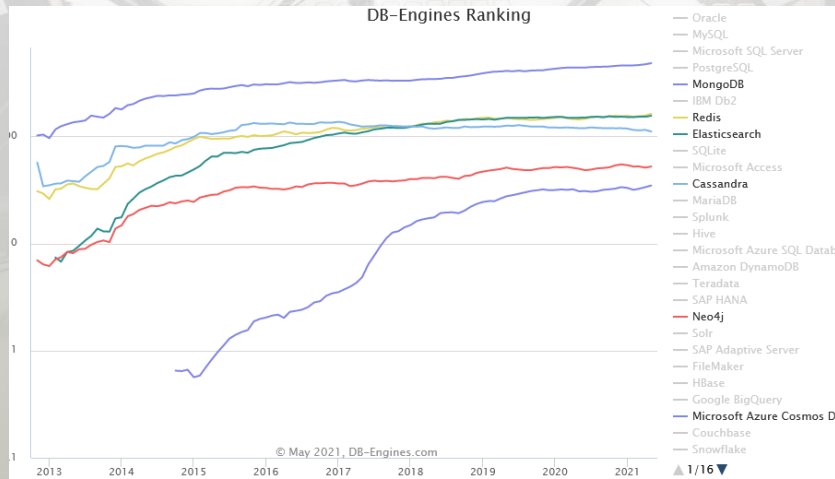
Cette dernière catégorie représente une minorité des systèmes mais parmi ceux-là, une solution récente (2015) prend de plus en plus de parts de marché :

Microsoft Azure Cosmo DB

60

60

Cosmo DB



61

61

Cosmo DB



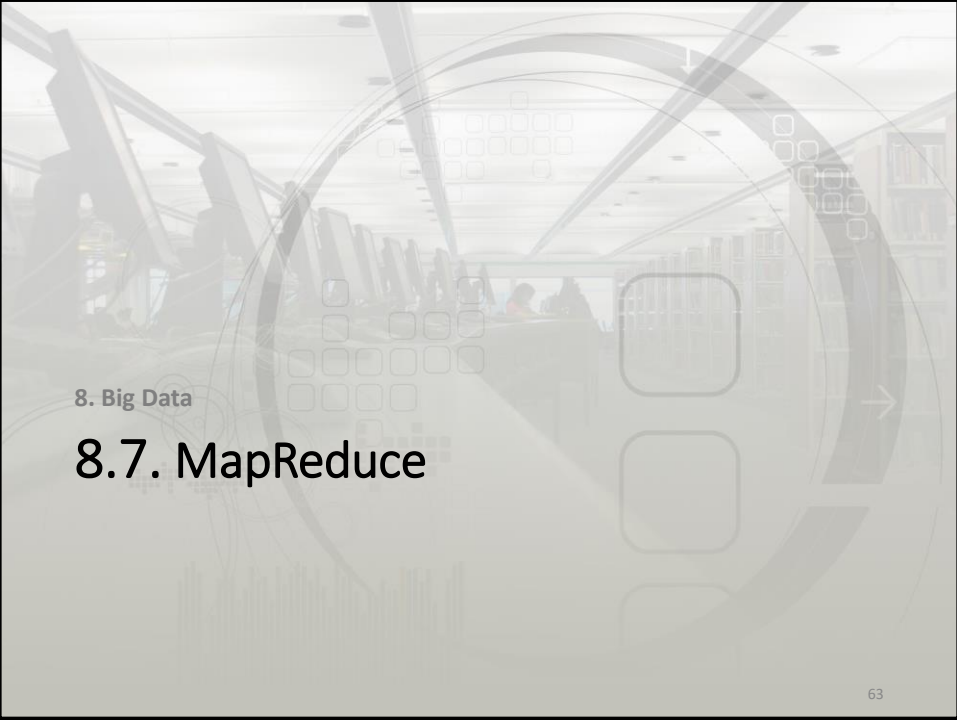
- Les API sont similaires aux autres acteurs NoSQL (Cassandra, MongoDB, etc.)
- BD distribuée sur tout le globe : **rapatriement automatique** des données vers le data center le plus proche afin de fournir **les latences les plus faibles**
- Support du SQL
- **ACIDité** des transactions et robustesse générale du système



- Solution propriétaire payante
- Uniquement disponible dans le cloud Microsoft
- Encore **peu d'évaluations "scientifiques"** de la solution

62

62

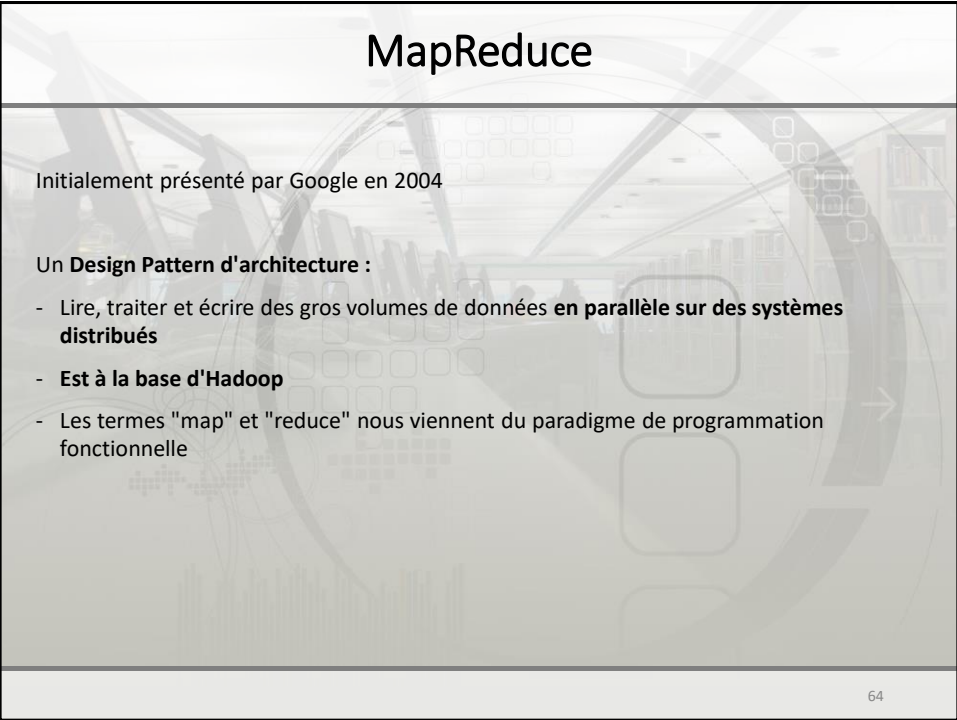


8. Big Data

8.7. MapReduce

63

63



MapReduce

Initialement présenté par Google en 2004

Un **Design Pattern d'architecture** :

- Lire, traiter et écrire des gros volumes de données **en parallèle sur des systèmes distribués**
- **Est à la base d'Hadoop**
- Les termes "map" et "reduce" nous viennent du paradigme de programmation fonctionnelle

64

64

MapReduce : exemple

Votre boss vous demande :

Compter le nombre de mots d'une lettre
Compter le nombre de mots de 2 lettres
Compter le nombre de mots de 3 lettres

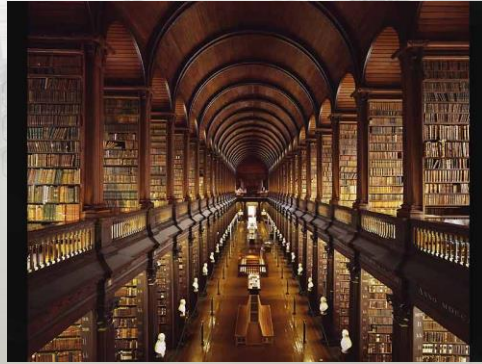
...

Compter le nombre de mots de 10 lettres

Livres : 500.000

Ressources 10.012 personnes

Durée max 1 semaine



<https://www.edilivre.com/bibliotheque-une-mission-impossible>

65

65

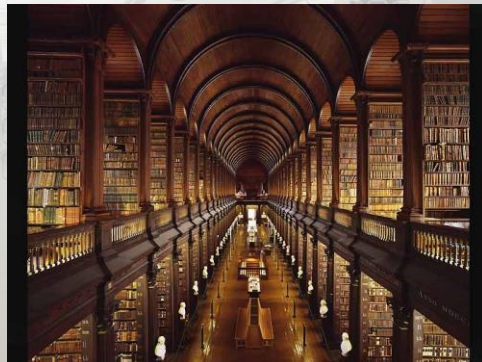
MapReduce : exemple

Solutions non admises :

- Démission
- Suicide
- Homicide

Solution :

- Prendre une journée de repos pour trouver une bonne solution
- **Jour 2** (après une journée de réflexion):
« Bordel j'ai perdu une journée »



<https://www.edilivre.com/bibliotheque-une-mission-impossible>

66

66

MapReduce : exemple

Vous divisez l'équipe en :

- 10.000 « mappers »
- 1 « grouper »
- 10 « reducers »
- 1 « master » (vous)

Chaque « **Mapper** » traite 50 livres

écrit chaque mot rencontré sur une feuille de papier en respectant la syntaxe :

Nombre de caractères , le mot

1,à
2,le
6,maison
1,à
8,saucisse
...

Fin du jour 4 : le « **grouper** » prend 10 papiers pour y classer tous les mots :

Feuille 1
À, a , a , à
l, à, a, V, a,
...

Feuille 2
Le, la , le , le
La , un , le ,
Le , ...

Feuille 3
Une, pet, une,
Rue, toi, une,
bit, ...

Feuille 10
...

Quand c'est terminé, il distribue une feuille à chacun des « reducers »

Le « **reducer** » compte le nombre de mots sur sa feuille et l'inscrit au verso

Fin du jour 5 : le « **master** » récolte les réponses et peut fournir la réponse à son boss

67

67

MapReduce : autre exemple

Source de données

A: Le loup est dans la bergerie
B: Les moutons sont dans la bergerie

C: Un loup a mangé un mouton, les autres loups (...) bergerie

D: trois moutons dans le pré, un mouton dans la gueule du loup

read()

read()

read()

Mapper 1

Mapper 2

Mapper 3

emit()

emit()

emit()

Fragments intermédiaires

(loup, 1)
(bergerie, 1)
(mouton, 1)
(bergerie, 1)

(loup, 2)
(mouton, 1)
(bergerie, 1)

(mouton, 2)
(pré, 1)
(gueule, 1)
(loup, 1)

(loup, 1)

(loup, 2)

(loup, 1)

Reducer 1

Reducer 2

Résult.

(loup, 1)

(loup, 2)

(loup, 1)

[1, 2, 1]

(loup, 4)

Résult.

<http://b3d.bdpedia.fr/calculdistr.html>

- Les *mapers* et *reducers* travaillent indépendamment
- Le *grouper* ne doit pas analyser toutes les données mais juste les clés
- Le *master* dispatch le travail

68

68

Hadoop

Framework open source écrit en **Java** maintenu par la fondation Apache

Le noyau Hadoop est constitué :

- d'une solution de stockage **HDFS (Hadoop Distributed File System)**
- d'une partie traitement implémentant le **MapReduce**

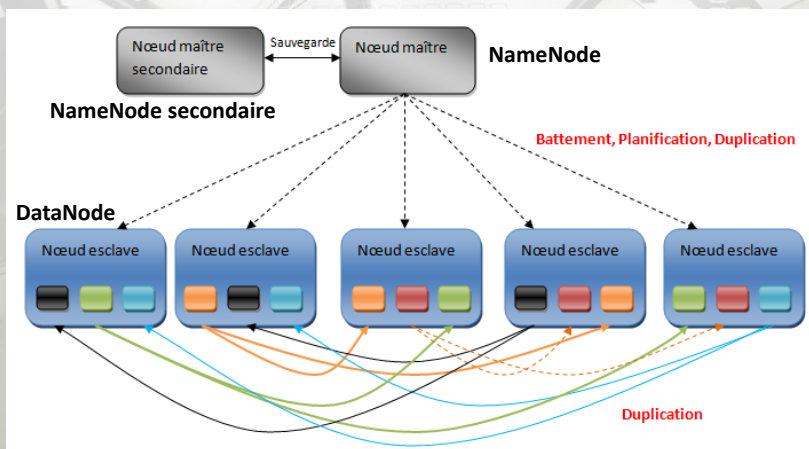
Souvent utilisé conjointement à d'autres systèmes en tant que solution de stockage et d'accès aux données :

- **Abstraction du stockage physique**
- **Scalabilité horizontale** : il suffit d'ajouter des nœuds pour accroître l'espace de stockage

69

69

Hadoop



<https://fr.wikipedia.org/wiki/Hadoop#/media/File:HDFS.png>

70

70

8. Big Data

8.8 NewSQL

71

71

NewSQL

SGBDR

NoSQL

NewSQL

Fonctionnalités



Scalabilité (horiz.)



La définition de ce que doit être un système
NewSQL est relativement floue

Attention aux effets
d'annonce !

De nombreux systèmes se disent être de type NewSQL
sans cocher toutes les cases de ce que doit réellement
offrir un tel système

72

72

Concrètement...

- Modèle relationnel
- Scalabilité horizontale
- Architecture distribuée de type *shared-nothing*
- Utilisation OLTP
- ACIDité des transactions
- Un minimum de verrous (voir aucun)
- Moteur de stockage propre (pas de HDFS)

73

73

Moteur de stockage NewSQL

De nombreux systèmes "scalables" se reposent sur Hadoop et son HDFS pour stocker les données <- ~~NewSQL~~

Un système NewSQL doit intégrer son propre moteur de stockage

- Permet certaines optimisations (car non généraliste)
- Offre des stratégies de partitionnement, de réplication et de rééquilibrage de charge plus sophistiquées.

Ce sont des systèmes de type In-Memory capables de déplacer automatiquement les "données froides" de la RAM vers le support de stockage.

- permet de ne pas être limité par la taille de la RAM
- OLTP (beaucoup de données froides)
- ~~OLAP (peu de données froides)~~

74

74

Paysage actuel et futur ?

- Nos bons vieux **SGBDR**
conçus il y a longtemps à l'époque du mono-cœur et du peu de RAM
- Des systèmes dédiés à l'**OLAP**
généralement basés sur du SGBDR
- La famille des systèmes **NoSQL**
conçus à l'époque du multicœur, des SSD et de la RAM moins limitée
- Les systèmes **NewSQL**
des idées d'il y a longtemps avec une conception moderne
- **L'avenir ???**
Des systèmes NewSQL capables d'être aussi performants pour de l'OLAP
que de l'OLTP (HATP, Hybrid transaction-analytical processing)